SHELL DE UNIX

1. Introducción a UNIX 4
1.1. Historia de UNIX 4
1.2. Introducción a los sistemas UNIX 6
1.3. Conexión a un sistema UNIX 6
1.4. Introducción al interfaz de UNIX 7
1.5. Mantenimiento de la clave 9
1.6. Sistema de ficheros de UNIX 10
1.6.1. Organización de archivos 11
1.6.2. Localización de comandos 13
1.6.3. Comodines y caracteres especiales 13
1.6.4. Ficheros especiales 14
1.6.5. Permisos sobre los ficheros y directorios 15
1.7. Procesos en UNIX 16
2. Intérpretes de comandos 18
2.1. bash 18
2.1.1. Caracteres especiales 18
2.1.2. Los alias 21
2.1.3. Configuración de opciones de bash 22
2.1.4. Redirección de E/S y tuberías 23
2.1.5. Variables de entorno 24
3. Comandos 27
3.1. Comandos de manejo de ficheros 27
3.1.1. Comando ls 27
3.1.2. Comando pdw 30
3.1.3. Comando cd 30
3.1.4. Comando mkdir 30
3.1.5. Comando rmdir 31
3.1.6. Comando cp 31
3.1.7. Comando rm 32
3.1.8. Comando mv 33
3.1.9. Comando ln 34
3.1.10. Comando chmod 38
3.1.11. Comando chown 39
3.2. Comandos de procesamiento de ficheros 40
3.2.1. Comando cat 40
3.2.2. Comando wc 41
3.2.3. Comando sort 42
3.2.4. Comando more 44
3.2.5. Comando less 45
3.2.6. Comando tail 45
3.2.7. Comando head 46

3.2.8. Comando grep ... 47
3.2.9. Comando find ... 49
3.2.10. Comando uniq ... 51
3.2.11. Comando comm ... 51
3.2.12. Comando cmp ... 52
3.2.13. Comando lpr ... 52
3.3. Comandos de manejo de unidades y dispositivos ... 53
3.3.1. Comando mount ... 54
3.3.2. Comando umount ... 55
3.3.3. Comando df ... 55
3.3.4. Comando mdir ... 56
3.3.5. Comando mcopy ... 56
3.3.6. Comando mkfs ... 57
3.4. Comandos de manejo de procesos ... 57
3.4.1. Comando ps ... 57
3.4.2. Comando kill ... 58
3.4.3. Comando top ... 59
3.5. Comandos de administración ... 59
3.5.1. Comando passwd ... 60
3.5.2. Comando shutdown ... 60
3.5.3. Comando adduser ... 61
3.5.4. Comando chsh ... 62
3.5.5. Comando chfn ... 62
3.6. Comandos de obtención de información ... 63
3.6.1. Comando who ... 63
3.6.2. Comando whoami ... 63
3.6.3. Comando tty ... 64
3.6.4. Comando id ... 64
3.6.5. Comando date ... 64
3.6.6. Comando cal ... 65
3.7. Comandos de control de sesión ... 65
3.7.1. Comando exit ... 65
3.7.2. Comando su ... 65
3.7.3. Comando login ... 66
3.8. Comandos de comunicación ... 66
3.8.1. Comando write ... 66
3.8.2. Comando mesg ... 67
3.8.3. Comando mail ... 67
3.9. Comandos de cálculo ... 67
3.9.1. Comando expr ... 67
3.9.2. Comando test ... 68
3.10. Comandos de escritura ... 70
3.10.1. Comando echo ... 71
3.10.2. Comando clear ... 71
3.10.3. Comando reset ... 72
3.10.4. Comando banner ... 72
3.11. Comandos de manejo de variables ... 72
3.11.1. Comando export ... 72
3.11.2. Comando set ... 73
3.11.3. Comando alias ... 73
3.12. Comandos de control de tiempo ... 74
3.12.1. Comando sleep ... 74
3.12.2. Comando time ... 74
3.12.3. Comando at ... 74
3.13. Comandos de ayuda ... 75
3.13.1. Comando man ... 75
3.13.2. Comando whatis ... 75
3.13.3. Comando apropos ... 76

3.14. Resumen de comandos ... 77
4. Editores de texto ... 83
4.1. Editor joe ... 83
4.2. Editor emacs ... 84
4.3. Editor vi ... 87
4.4. Entornos wpe y xwpe ... 89
5. Scripts en bash ... 91
5.1. Creación de scripts ... 91
5.2. Parámetros ... 93
5.3. Sentencias de control ... 95
5.3.1. Estructura iterativa for ... 95
5.3.2. Estructura iterativa while ... 95
5.3.3. Estructura iterativa until ... 96
5.3.4. Estructura alternativa simple (if) ... 96
5.3.5. Estructura alternativa múltiple (case) ... 97
5.4. Depuración de scripts ... 98
5.5. Scripts de iniciación ... 99
6. Bibliografía complementaria ... 101
6.1. Libros de Sistemas Operativos y UNIX ... 101
6.2. Libros de Linux en la Biblioteca ... 101
6.3. Libros de UNIX en la Biblioteca ... 101

1. Introducción a UNIX

1.1. Historia de UNIX

Los orígenes de los sistemas operativos UNIX se remontan a los años 60. Las empresas *Bell Labs* (*AT&T*), *General Electric* y el *MIT* trabajaban conjuntamente en un proyecto denominado *MULTICS* (*Multiplexed Information and Computing Services*), un sistema operativo multiusuario interactivo, proyecto que fracasó y tuvo que ser abandonado debido a la excesiva complejidad del mismo para los equipos de la época.

En 1969 *Ken Thompson* (*Bell Labs*) desarrolló una versión reducida de MULTICS para un máquina DEC PDP-7, que llamó irónicamente *UNICS* (*Uniplexed*...). Posteriormente (1970) se portó a una máquina PDP-11/20 y se le dio el nombre de ***UNIX***. Este sistema daba servicio a 2 usuarios simultáneamente, y estaba programado en un lenguaje de alto nivel llamado PL/1.

En 1973 *Dennis Ritchie* desarrolla (junto con *Thompson*) el lenguaje de programación *C*, y UNIX se reescribe en este lenguaje. A mediados de los 70 se desarrolla la versión 6 de UNIX para múltiples plataformas, y se proporciona de forma gratuita con fines didácticos a las universidades, lo cual le da una gran expansión. Diversas entidades programan sus propias versiones de UNIX.

A finales de los 70, *Thompson* desarrolla la versión *Berkeley Software Distribution (BSD)*. Por otra parte *AT&T* y *Sun Microsystem* fusionan sus versiones de UNIX y crean *System V*. *System V* y *BSD* sirven de punto de partida para multitud de distribuciones, cada vez más diferentes entre sí. Para tratar de poner orden entre las versiones de UNIX, durante los 80 se forman consorcios para la fijación de estándares. Surge entonces el estándar *POSIX*, que, a grandes rasgos, establece cómo debe funcionar un buen sistema operativo, y al que pretenden ajustarse las sucesivas versiones de UNIX. La Tabla 1 contiene algunas de las versiones de UNIX más conocidas.

Producto	**Fabricante**	**Basado en**	**Arquitecturas**
Solaris	*Sun Microsystems*	*AT&T*	*Sparc* y *x86*
SunOS (Solaris 1)	*Sun Microsystems*	*BSD*	*Sparc*
DEC OSF/1	*Digital Research*	*BSD*	*Alpha*
AIX	*IBM*	Diferente de *AT&T* y *BSD*	*RS6000*
IRIX	*Silicon Graphics*	*AT&T*	*Silicon Graphics*
SCO	*The Santa Cruz Operation*	*AT&T* con muchos agregados	*Intel x86*
HP-UX	*Hewlett-Packard*	*AT&T*	*Hewlett-Packard*
Linux	(Público)	*BSD* y *AT&T*	*Intel x86, Sparc, Alpha, PowerPC, PowerMac*
FreeBSD	(Público)	*BSD*	*Intel x86*
OpenBSD	(Público)	*BSD*	*Intel x86, Alpha, Sparc, HP, Amiga*

Tabla 1. Algunas versiones populares de UNIX

Entre todos estos sistemas surge también una versión reducida de UNIX, ***Minix***, desarrollada por *Andrew S. Tanenbaum* y distribuida de forma gratuita con fines didácticos. Minix funciona en computadores personales de la familia *x86* y *68xxx*, y fue creado desde cero, es decir, que no se basa en el código de las anteriores versiones de UNIX. Se trataba de obtener un sistema fácil de estudiar y gratuito, para utilizarlo en la enseñanza universitaria de sistemas operativos.

En 1990 *Linus Torvald* pretende mejorar este sistema (como pasatiempo) y crea ***Linux***, que se publica por primera vez en octubre de 1991 como una versión de prueba (0.02) para computadores *i386*. A partir de entonces el sistema empezó a crecer, gracias a la cooperación de desarrolladores de todo el mundo. Por esta razón, Linux es también gratuito y adquiere mucha popularidad, especialmente en los últimos años, ayudado también por la expansión de los computadores personales de gran potencia. Así, surgen multitud de distribuciones de Linux, siguiendo las recomendaciones POSIX, con características diferentes, y portados a múltiples plataformas: *x86*, *68k*, *Digital Alpha*, *Sparc*, *Mips* y *Motorola Power PC*. La Tabla 2 recoge las principales versiones de Linux en el mercado.

En realidad Linux no es dominio público (no se puede hacer cualquier cosa con él por el hecho de ser gratis), sino que sigue una licencia especial llamada ***GNU GPL (GNU General Public License)***. Esta licencia protege la filosofía Linux: software de todos y para todos. Cualquiera puede modificarlo y redistribuirlo sin consultar (aunque muchas partes del sistema poseen derechos de autor de sus creadores, por ejemplo, el núcleo, que pertenece a *Linus Torvalds*), pero con limitaciones. GPL permite, por ejemplo, modificar, redistribuir e incluso vender lo que hagamos, pero obliga a distribuir el código fuente junto con los programas para que puedan ser modificados y redistribuidos de nuevo.

Distribución	**Características**
Caldera OpenLinux	Viene acompañado por multitud de aplicaciones y utilidades.
Debian GNU/Linux	Mucho esfuerzo invertido en fiabilidad. Es mantenido por muchos voluntarios de todo el mundo. Instalación relativamente compleja.
RedHat	Disponible para *Intel*, *Alpha* y *Sparc*, pretendiendo mantener la total compatibilidad entre las versiones de esta arquitecturas.
Slackware Linux	Pretende dar soporte a todo el hardware disponible para procesadores Intel: discos, tarjetas, placas multiprocesador y optimización de código.
SuSE Linux	Incluye la herramienta *YaST* para facilitar la instalación y configuración del sistema, que incluye gran cantidad de programas y utilidades.
Linux Mandrake	Incorpora gran cantidad de aplicaciones. Funciona en procesadores de la familia Intel.
LinuxPPC	Portado a *PowerPC*
Linux Pro	Proporciona una gran cantidad de documentación adicional.
LinuxWare	Muchas facilidades de instalación, orientada a usuarios de DOS/Windows.
MkLinux	Portado a *PowerMac*
TurboLinux	Incluye aplicaciones fáciles de manejar, con documentación y soporte técnico. Disponible en *Intel*, *Alpha* y *PowerPC*.
Yggdrasil Linux	Facilidades de instalación *Plug&Play*.
DLX Linux	Pensado para funcionar desde un disquete de 3,5".
DOS Linux	Diseñado para ser instalado en un sistema DOS/Windows.
tomsrtbt Linux	Cabe en un único disquete de 3,5" y contiene varias herramientas para verificación y reparación de sistemas de ficheros.

Tabla 2. Principales versiones de Linux

NOTA:

Al nivel en que vamos a trabajar, UNIX y Linux, son *algo parecido*. Podemos olvidar la historia y hablar de UNIX como "*un tipo o estilo de sistema operativo*", y de Linux como "*una versión de un sistema operativo de tipo UNIX*". Así, estudiaremos UNIX y Linux a la vez, de forma que todo lo referente a UNIX será válido para Linux, aunque también veremos detalles presentes exclusivamente en Linux.

1.2. Introducción a los sistemas UNIX

UNIX es un sistema operativo *multiusuario* y *multitarea*, es decir, que varios usuarios distintos pueden ejecutar programas en el mismo computador, y que varios programas, del mismo o diferentes usuarios, pueden ejecutarse a la vez de forma concurrente o paralela. UNIX está construido de forma que los usuarios normales no puedan realizar operaciones que interfieran con los otros usuarios o que puedan dañar el sistema. Así, determinadas operaciones no están permitidas a determinados usuarios. Esto nos conduce a la existencia de diferentes tipos de usuarios. Además, los usuarios se organizan en grupos, de forma que, a veces, los permisos para efectuar determinadas operaciones se otorgan, no a usuarios individuales, sino a grupos de usuarios.

Inicialmente distinguiremos entre dos tipos de usuarios: usuarios normales y administradores. Los administradores (también reciben el nombre de *super-usuario* o *root*) tienen todos los derechos para realizar cualquier operación, pero los usuarios normales no. Los administradores son los que se encargan que configurar el sistema para que el resto de usuarios trabaje con las máximas garantías.

Para trabajar en una máquina UNIX no es necesario utilizar el teclado y la pantalla conectados al computador, sino que podemos acceder de forma remota a través de una red. Al teclado y pantalla de una máquina se le llama *consola*, y es muy frecuente utilizar la consola de una máquina para acceder hasta otra máquina remota y trabajar en ella. A través de la red viajan los comandos del teclado y la información de la pantalla, de forma que los programas se ejecutan en el computador UNIX, pero los resultados se solicitan y visualizan en otro sitio.

Cuando accedemos a un ordenador con UNIX, lo primero que debemos hacer es identificarnos. Para eso necesitamos un *nombre de usuario* (*username*, *login*) y una *clave de paso* (*password*) Con el nombre de usuario somos identificados, y con la clave nos aseguramos de que nadie utilice nuestro nombre de usuario. Cuando hacemos alguna operación importante, en el sistema queda registrada la información de la operación, fecha y el usuario que la realiza. Por esta razón es importante mantener nuestra clave en secreto, de forma que nadie pueda suplantarnos ni acceder a nuestra información (ficheros, programas, etc.).

1.3. Conexión a un sistema UNIX

Aparte de utilizar la consola propia del computador en el que está instalado el sistema Linux, existen multitud de formas de conectarse a una máquina Linux. La más habitual es a través de *telnet*. *telnet* es un protocolo de comunicación que nos permite conectarnos a la máquina Linux para ejecutar comandos y obtener los resultados en nuestra pantalla. Lo primero que necesitamos es un *cliente telnet*, que es un programa que nos permite realizar la conexión e intercambiar la información entre nuestro ordenador y el ordenador Linux, utilizando el protocolo *telnet*. Habitualmente este programa se llama también `telnet`. Por ejemplo, en Windows, tenemos el programa `c:\windows\telnet.exe`, y UNIX existe el comando *telnet*. De alguna forma debemos indicar con cuál ordenador queremos conectarnos, y eso lo hacemos especificando su *nombre* o su *dirección IP*. Esta identificación la pasamos como parámetro al programa *telnet*.

```
telnet polifemo
telnet 172.17.21.248
```

Sea cual sea la forma en que nos conectemos a la máquina Linux, el primer paso será identificarnos. La máquina Linux nos pedirá el nombre de usuario con un mensaje de bienvenida y la palabra *login*:

```
Linux 2.0.30 (polifemo.uhu.es) (ttyp0)
polifemo login:_
```

Aquí escribimos el nombre de usuario y pulsamos *Enter*. Ahora Linux pedirá que escribamos el *password*.

```
Linux 2.0.30 (polifemo.uhu.es) (ttyp0)
polifemo login: llorente
Password:_
```

Al teclear el *password* no veremos nada en pantalla, de forma que nadie pueda ver lo que escribimos, pero realmente sí se están introduciendo. Finalmente pulsamos *Enter*. Si todo es correcto, entraremos, si no, Linux mostrará un mensaje indicando que la clave no es correcta. En el caso de que escribamos mal el nombre de usuario, el error será el mismo, ya que el problema realmente es que la clave no corresponde al nombre de usuario.

Una vez entremos en el sistema podremos comenzar a trabajar con él.

Antes de apagar el ordenador debemos tener en cuanta que un computador UNIX **no se puede apagar** en cualquier momento, sino que debe seguirse un procedimiento "ordenado" de apagado (mediante los comandos apropiados: `shutdown`, `halt`, `reboot`, etc.) antes de desconectar la alimentación del computador. Si apagamos el computador Linux sin el procedimiento adecuado, nos arriesgamos a que el sistema de archivos sufra daños y pérdidas irreparables. En el caso de que nos hayamos conectado a través de la red (por ejemplo mediante *telnet*), únicamente necesitamos cerrar la conexión, ya que el computador Linux seguirá encendido aunque apaguemos nuestra consola. La forma correcta de abandonar el sistema en este caso es mediante los comandos `exit` o `logout`; después podemos apagar nuestro terminal. La siguiente pantalla muestra un ejemplo de una sesión completa realizada de forma remota mediante telnet:

```
Linux 2.0.30 (polifemo.uhu.es) (ttyp0)
polifemo login: llorente
Password:

Linux 2.0.30.
Last login: Fri Oct 15 12:20:22 on ttyp0 from jmal.uhu.es.
No mail.

polifemo:~$ exit
logout

[polifemo:23: remote disconnect]
```

1.4. Introducción al interfaz de UNIX

La interfaz elemental de UNIX (y, por supuesto, de Linux) es el *intérprete de comandos*, como ocurre en otros sistemas como MS-DOS. Un *intérprete de comandos* nos permite escribir órdenes que efectúan acciones sobre el sistema: ejecutar programas, manejar ficheros, etc. Es muy importante recordar que UNIX es **sensible a mayúsculas y minúsculas** (al contrario que MS-DOS), es decir, que no es lo mismo escribir `ls` y `LS`.

Una vez que entramos en el sistema, UNIX nos muestra el *prompt*. El *prompt* es un símbolo que nos indica que el sistema está listo para que escribamos comandos. Este símbolo suele ser alguno de los siguientes: ~, #, %, >, $. A veces antes del *prompt* aparece alguna información como el nombre del usuario, el nombre de la máquina, el directorio actual, etc. A partir de ahora supondremos que el símbolo del *prompt* es "$".

Los comandos UNIX tienen una estructura regular, y todos ellos tiene una sintaxis y funcionamiento similar:

```
comando flags argumento1 argumento2 argumento3 ...
```

- **comando** es el nombre del comando, y suele ser una abreviatura o acrónimo de la acción para la que se usa, normalmente en minúsculas.
- Los **flags** suelen ser opcionales. Son un conjunto de modificadores que hacer que el comando funcione con diferentes opciones, formatos, etc. Por lo general constan de el signo menos (-) seguido, sin espacios, de una letra (recordemos que mayúsculas y minúsculas son diferentes).
- Los **argumentos** suelen ser nombres de ficheros, y se refieren a los objetos (ficheros, etc.) sobre los que actuará el comando.

El número de *flags* y *argumentos*, así como su orden varía con cada comando. Veamos un ejemplo. El comando `ls` sirve para listar los archivos de un directorio (*ls* es el acrónimo de *list subdirectory*). Si escribimos simplemente el comando `ls`, obtenemos el listado de ficheros:

```
$ls
a.out*   mail/    mbox     min*     min.c    nsmail/  p/       tmp/
```

Así sólo se obtienen los nombres de los archivos del directorio. Podemos obtener información detallada utilizando el modificador `-l`:

```
$ls -l
total 20
-rwxr-xr-x   1 llorente users        4261 May  4 10:40 a.out*
drwx------   2 llorente users        1024 Jun  1 11:09 mail/
-rw-------   1 llorente users        2308 Jun  3 14:07 mbox
-rwxr-xr-x   1 llorente users        5643 Jun  3 14:54 min*
-rw-r--r--   1 llorente users        1296 Jun  3 14:54 min.c
drwx------   2 llorente users        1024 Apr  7  1999 nsmail/
drwxr-xr-x   2 root     root         1024 Jun  8 10:46 p/
drwxr-xr-x   2 llorente users        1024 Apr 15 15:06 tmp/
```

Por otra parte podemos utilizar el modificador `-a` para mostrar los archivos ocultos:

```
$ls -a
./                .less             .netscape/        mbox
../               .lessrc           .pinerc           min*
.Xauthority       .mc.hot           .tkdesk/          min.c
.addressbook      .mc.ini           .xsession-errors  nsmail/
.addressbook.lu   .mc.tree          a.out*            p/
.bash_history     .ncftp/           mail/             tmp/
```

También podemos combinar los modificadores. Normalmente es equivalente escribirlos por separado o juntos, y en cualquier orden (`-a -l` es lo mismo que `-al` y es lo mismo que `-la` y es lo mismo que `-l -a`):

`ls` también acepta como argumentos los nombres de archivos que se quieren procesar. Por ejemplo, si sólo queremos ver la información de los `min.c` y `a.out`:

```
$ls -l min.c a.out
-rwxr-xr-x   1 llorente users        4261 May  4 10:40 a.out*
-rw-r--r--   1 llorente users        1296 Jun  3 14:54 min.c
```

Para conocer la sintaxis y utilidad de un comando disponemos de un manual que podemos invocar mediante el comando `man` pasándole como argumento el nombre de otro comando. Por ejemplo, si queremos conocer la sintaxis de `ls`, escribimos `man ls`. Aparecerá un texto con completa información sobre el comando requerido. Podemos navegar por este texto utilizando la barra espaciadora, *Enter* y las teclas *B* y *Q*.

1.5. Mantenimiento de la clave

Ya sabemos que cada usuario dispone de una clave para que nadie pueda suplantarlo ni acceder a sus datos. El mantenimiento de esta clave es muy importante, y es tarea de cada usuario:

- Mantener su clave en secreto.
- Modificar periódicamente la clave.

El uso de una clave no es una garantía absoluta de que nadie puede utilizar nuestro nombre de usuario. Existen infinidad de métodos de ataque que permiten averiguar la clave de un usuario, especialmente cuando ésta es una palabra "fácil". Por eso, es recomendable cambiar periódicamente nuestra clave. Además, debemos tener en cuenta que hay claves muy fáciles de averiguar. Por ejemplo, nunca deberíamos utilizar claves del siguiente tipo:

- Nuestras iniciales, nuestro nombre junto a un número, etc.
- Fechas, especialmente nuestro año de nacimiento o cualquier fecha importante.
- Nombres de películas, canciones, actores, etc. populares.

Además, cuando cambiemos nuestra clave, no debemos utilizar una clave parecida a la anterior. La mejor idea es emplear combinaciones más o menos aleatorias de letras (mayúsculas y minúsculas), números y símbolos. El problema es que es difícil recordar este tipo de claves, pero son las más seguras. En muchos sistemas no se permite a los usuarios utilizar claves "fáciles" (*débiles*); en otros, simplemente, se advierte que una clave no es apropiada.

Por lo tanto una de nuestras primeras actividades en un sistema UNIX es establecer una buena clave. Cuando el administrador nos proporciona un nombre de usuario, también nos debe proporcionar una clave, pero lo mejor es que nosotros la cambiemos inmediatamente. Vamos a ver cómo hacerlo.

Se trata de utilizar un comando. En este caso el comando es `passwd`. El sistema nos pedirá, en primer lugar que introduzcamos la clave antigua (al teclearla no aparecerá nada en pantalla):

```
$ passwd
Changing password for llorente
Old password:_
```

Si no introducimos correctamente la clave antigua no podremos continuar, y el comando terminará:

```
$ passwd
Changing password for llorente
Old password:
Incorrect password for llorente.
The password for llorente is unchanged.
```

```
$_
```

En caso contrario, continuamos con el procedimiento. El siguiente paso es escribir la nueva clave (tampoco aparecerá nada en pantalla al teclear).

```
$ passwd
Changing password for llorente
Old password:
Enter the new password (minimum of 5, maximum of 8 characters)
Please use a combination of upper and lower case letters and numbers.
New password:_
```

Si escribimos una clave débil, nos mostrará un mensaje de error (p.e. *Bad password: too simple. Try again.*) y volverá a pedir la nueva clave. De lo contrario, es decir, si la clave es adecuada, el siguiente paso será repetir la clave (por si nos hemos confundido al teclear, ya que no vemos lo que escribimos):

```
$ passwd
Changing password for llorente
Old password:
Enter the new password (minimum of 5, maximum of 8 characters)
Please use a combination of upper and lower case letters and numbers.
New password:
Re-enter new password:_
```

Si ahora tecleamos la misma clave, habremos terminado y `passwd` mostrará un mensaje indicando el cambio de clave:

```
$ passwd
Changing password for llorente
Old password:
Enter the new password (minimum of 5, maximum of 8 characters)
Please use a combination of upper and lower case letters and numbers.
New password:
Re-enter new password:
Password changed.
$
```

Pero si nos confundimos y tecleamos algo diferente, el comando no cambiará la clave, mostrará un mensaje de error (p.e. *They don't match; try again.*) y nos pedirá otra vez que tecleemos la clave nueva (y después tendremos que repetirla).

1.6. Sistema de ficheros de UNIX

En UNIX, al igual que ocurre en MS-DOS y Windows, los ficheros se organizan en directorios, formando un árbol, pero, a diferencia de éstos, en UNIX no existen distintas "unidades de disco", sino que todo se organiza en el mismo árbol, con una única raíz. El acceso a los distintos dispositivos se hace colocando los árboles de cada dispositivo en alguna posición del árbol principal. Esta operación se denomina ***montar un dispositivo***. Cuando acabamos de utilizar los archivos guardados en ese dispositivo, debemos ***desmontar un dispositivo***, que consiste en

liberar la asignación de ese dispositivo a la ruta donde se ha montado. Es muy importante desmontar las unidades extraíbles, especialmente los disquetes, antes de extraer el dispositivo, porque, de lo contrario, podemos dañar el sistema de ficheros completo.

Por esta razón , los usuarios normales no pueden montar ni desmontar los dispositivos; esta es una tarea reservada al administrador.

Los dispositivos que se pueden montar son, en principio, únicamente los que están en la máquina UNIX. Cuando trabajamos de forma remota (por ejemplo mediante *telnet*), no podemos montar las unidades de nuestro computador local en el sistema de ficheros de la máquina UNIX remota.

1.6.1. Organización de archivos

En UNIX existen 3 tipos principales de ficheros o archivos:

- Directorios
- Ficheros ordinarios
- Ficheros especiales

Los ficheros ordinarios sirven para almacenar datos y programas. Los directorios sirven para crear la estructura de árbol, de forma que los directorios almacenan ficheros y otros directorios. Los ficheros especiales los veremos más adelante.

Los ficheros (de cualquier tipo) se identifican mediante un nombre. El nombre de un archivo puede contener cualquier combinación de caracteres, pero es recomendable no utilizar algunos caracteres que tienen significados especiales. Por lo general es mejor utilizar solamente letras y números. Debemos recordar que se diferencia entre mayúsculas y minúsculas.

Los nombres de ficheros suelen terminar con una extensión. La extensión está formada por un punto (.) y una secuencia de caracteres, y suele utilizarse para indicar el tipo de datos que contiene el archivo. Por ejemplo, los archivos que contienen código fuente de programas escritos en lenguaje C suelen tener extensión "`.c`". En este sentido, la extensión es un concepto diferente en MS-DOS y UNIX, ya que en MS-DOS la extensión es obligatoria (aunque esté vacía se supone que existe), y tiene un significado a veces ineludible; así los programas ejecutables en MS-DOS deben llevar obligatoriamente extensión de ejecutable (`.COM`, `.EXE`, `.BAT`, etc.). Además, en MS-DOS el carácter punto (.) sólo se puede emplear para separar el nombre de la extensión, pero en UNIX el punto se puede utilizar varias veces en el nombre.

Todo fichero o directorio debe estar situado dentro de algún directorio (excepto el directorio raíz, que es el origen). El camino de directorios que hay que seguir desde la raíz hasta encontrar un fichero se llama ruta (*path*). Para representar la ruta completa de un fichero, se escriben la secuencia de directorios que lo contienen, separados por una barra invertida (`/`, *slash*), seguido por el nombre del fichero. Por ejemplo, el fichero `libvga.config` que está en el directorio `vga` dentro del directorio `etc`, tiene la ruta `/etc/vga/libvga.config`. El directorio que contiene a otro directorio se denomina ***directorio padre*** o ***directorio superior***. Así, `/etc` es el directorio padre de `/etc/vga`.

Dentro de todos los directorios, excepto del directorio raíz (`/`), existen dos directorios especiales: "`.`" y "`..`"

- El directorio `.` es equivalente a la ruta del directorio que lo contiene. Es decir, que es lo mismo `/etc/vga/` que `/etc/vga/./`.
- El directorio `..` es equivalente a la ruta del directorio padre al que lo contiene. Es decir, que es lo mismo `/etc/` que `/etc/vga/../`.

Cuando nos queremos referir a un archivo dentro del árbol de directorios no siempre es necesario especificar la ruta competa del archivo. Para eso existe lo que de denomina ***directorio por defecto***, *directorio actual* o *directorio de*

trabajo. Cuando el directorio por defecto es *X*, decimos que ***estamos*** en el directorio *X*, o simplemente que estamos en *X*. En todo momento hay definido un directorio por defecto, de forma que si únicamente especificamos un nombre de archivo, se supone que la ruta hasta ese nombre de archivo es el directorio por defecto.

También podemos especificar directorios a partir del directorio actual. Esto es lo que se denomina una ***ruta relativa***, frente a una ***ruta absoluta***, que comienza por el directorio raíz (`/`). Por ejemplo, si el directorio por defecto es `/etc`, entonces son equivalentes `vga/libvga.config` y `/etc/vga/libvga.config`. También podemos utilizar `.` y `..` en rutas relativas. Por ejemplo, si el directorio por defecto es `/etc/ppp`, son equivalentes `../vga/libvga.config` y `/etc/vga/libvga.config`

El comando `pwd` (*print working directory*) escribe en pantalla la ruta del directorio por defecto. El comando `cd` (*change directory*) sirve para cambiar el directorio por defecto al que pasemos como argumento. Por ejemplo:

```
$ pwd
/home/llorente

$ cd .
$ pwd
/home/llorente

$ cd ..
$ pwd
/home

$ cd /etc
$ pwd
/etc
```

Cada usuario de un sistema UNIX tiene un directorio propio, llamado directorio *home*. El directorio *home* es el directorio donde se supone que trabaja el usuario, de manera que puede hacer cualquier cosa en su directorio, y ningún otro usuario puede acceder a él (a menos que explícitamente lo permita). Normalmente el directorio *home* de un usuario tiene el mismo nombre que el usuario y se coloca dentro del directorio `/home`, junto con los de los demás usuarios. Cada vez que un usuario entra al sistema, su directorio por defecto es su *home*. Además, utilizando el comando `cd` sin parámetros, el directorio por defecto volverá a ser el directorio *home*.

En todo sistema UNIX existen un conjunto de directorios que tienen un uso específico. Estos directorios pueden cambiar de unas versiones a otras, pero en general suelen ser los que muestra la Tabla 3.

Directorio	**Significado**
`/`	Directorio raíz. Contiene a todo lo demás.
`/bin`	contiene programas
`/dev`	Contiene los ficheros especiales relacionados con los dispositivos.
`/etc`	Contiene diversos archivos. Destacan los ficheros de claves (`passwd`, `shadow`) y algunos programas que sólo utiliza el administrador.
`/home`	Contiene los directorios *home* de los usuarios.
`/tmp`	Contiene archivos temporales.
`/usr`	Contiene archivos y programas varios.
`/usr/include`	Contiene los archivos de cabecera (`#include`) del lenguaje C.

Tabla 3. Algunos directorios estándar

1.6.2. Localización de comandos

Todos los comandos son programas que están en alguna posición en el árbol de directorios. Para evitar tener que escribir la ruta completa a cada comando, existe lo que se denomina ***path de búsqueda*** (o simplemente *path*) que consiste en un conjunto de rutas donde se supone que están los comandos. Así, si escribimos un comando sin especificar su ruta, el sistema buscará el comando en alguno de los directorios incluidos en el path. Si lo encuentra en alguna de las rutas, lo ejecuta; si no, generará un mensaje de error.

Cuando queremos ejecutar un programa o comando que se encuentre en el directorio actual también debemos indicar la ruta (o tener el directorio actual en el path), utilizando el nombre del directorio actual (`.`). Por ejemplo, si el comando se llama `mi_programa`, escribimos la siguiente línea:

```
$ ./mi_programa.c
```

1.6.3. Comodines y caracteres especiales

Los comodines (*wildcards*) son caracteres especiales que sirven para especificar patrones de nombres de ficheros (también llamados ***máscaras***). En muchas ocasiones necesitaremos especificar como argumentos a algún comando muchos nombres de ficheros que tienen alguna característica común. Por ejemplo, todos los ficheros que tengan extensión `.c`. Para este tipo de problema podemos utilizar los comodines, que son caracteres que representan a otros caracteres:

- La interrogación (?) representa a cualquier carácter (igual que en MS-DOS).
- El asterisco (*) representa a cualquier secuencia de caracteres, incluso una secuencia vacía (igual que en MS-DOS, pero puede colocarse en cualquier parte del patrón).
- Un conjunto de caracteres encerrados entre corchetes ([]) representa a cualquiera de los caracteres especificados entre los corchetes. Para utilizar el carácter `]` dentro de los corchetes, lo debemos poner como el primero de los caracteres justo detrás de `[`.
- Dos caracteres encerrados entre corchetes ([]) y separados por un guión (-) representa a cualquiera de los caracteres del rango delimitado por los caracteres especificados (incluidos éstos), según el orden ASCII. Para utilizar el guión dentro de los corchetes como carácter de la máscara, lo debemos poner al final (antes de `[`), o al principio (después de `[`).
- También podemos especificar un conjunto de caracteres que **no** deben formar parte del nombre de fichero. Para ello ponemos una admiración (`!`) justo después de `[`.

Todas estas posibilidades se pueden combinar. La Tabla 4 muestra algunos ejemplos.

Patrón	Significado (en el directorio por defecto)
*	Todos los archivos.
?	Archivos cuyo nombre sólo tenga un carácter (en el directorio por defecto).
.	Archivos que tengan un punto en alguna parte de su nombre.
[a-z]*	Archivos que comienzan por una letra minúscula.
*.c	Archivos con extensión ".c"
copia?.c	copia1.c, copia2.c, copiaA.c, copiaa.c, copia_.c, etc.
copia[123].c	copia1.c, copia2.c, copia3.c.
copia[123].*	copia1.*, copia2.*, copia3.*.
copia[1-5AB].c	copia1.c, copia2.c, copia3.c, copia4.c, copia5.c, copiaA.c, copiaB.c.
copia[!123].c	Todos los copia?.c, excepto copia1.c, copia2.c, copia3.c.
copia[123!].c	copia1.c, copia2.c, copia3.c, copia!.c.
copia[.c	El archivo copia[.c.
copia[]AB].c	copia].*, copiaA.*, copiaB.*.
copia[-AB].c	copia-.*, copiaA.*, copiaB.*.

Tabla 4. Ejemplos de uso de comodines para definir patrones de ficheros

Además de los comodines, existen otros caracteres especiales, es decir, que tienen algún significado especial para el intérprete de comandos, y, por lo tanto, a priori, no pueden ser utilizados para nombrar a los ficheros. Por ejemplo: espacios, >, &, $, etc. No obstante, y aunque no es recomendable, podemos utilizarlos. Para ello es necesario indicar que queremos utilizar esos símbolos como literales, obviando su significado especial. Tenemos dos posibilidades:

- Encerrar los nombre este comillas. Utilizamos las comillas simples (´).
- Indicar que un determinado símbolo no se utilice con su significado especial. Lo hacemos anteponiendo[1] el símbolo de barra invertida (\, *backslash*).

Por ejemplo, para referirnos al fichero con*asterisco podemos escribir ´con*asterisco´ o bien con*asterisco.

De todas formas, esta característica no es inherente a UNIX sino al intérprete de comandos, y la estudiaremos más adelante al profundizar en éste.

1.6.4. Ficheros especiales

Los ficheros especiales no sirven para almacenar información, sino para operaciones especiales. Consideraremos ficheros especiales:

- Los ***enlaces***. Sirven para que podamos acceder al contenido de un fichero o directorio desde distintos puntos del árbol de directorios. Tienen la apariencia de ficheros normales, pero el contenido de estos ficheros es compartido por otros archivos, de forma que si modificamos el contenido del fichero, queda modificado para todos los enlaces, ya que se refieren al mismo fichero.

[1] Al hecho de anteponer una barra invertida \ (o encerrar entre comillas) un carácter especial se le conoce vulgarmente con el nombre de "escapar" el carácter.

- Los ***dispositivos***. Sirven para manejar los distintos periféricos conectados al computador.

En UNIX todos los periféricos se manejan como si fueran ficheros. Por ejemplo, la pantalla se maneja como un fichero llamado `/dev/tty`, de forma que si escribimos información en este fichero, la información, en vez de almacenarse en el disco, se muestra en pantalla. Cada periférico tiene uno o varios ficheros especiales, que llamaremos dispositivos, y todos ellos se colocan en el directorio `/dev`. La Tabla 5 contiene algunos ejemplos de ficheros especiales.

Dispositivo	Significado
`/dev/tty`	Terminal (consola).
`/dev/hda`	Primer disco duro
`/dev/hda1`	Primera partición del primer disco duro
`/dev/hdb`	Segundo disco duro
`/dev/fd0`	Primera unidad de disco flexible.
`/dev/null`	Dispositivo nulo (descarta la información).
`/dev/ram`	Memoria RAM.

Tabla 5. Ejemplos de dispositivos

Cuando en UNIX se ejecuta cualquier programa, éste puede utilizar siempre tres dispositivos: la ***entrada estándar***, la ***salida estándar*** y la ***salida de errores***.

- La ***entrada estándar*** es el dispositivo por el que el programa recoge los datos de entrada, la información que debe procesar.
- La ***salida estándar*** es el dispositivo por el que el programa muestra los resultados, la información procesada.
- La ***salida de errores*** es el dispositivo por el que el programa muestra los errores durante el procesamiento de los datos.

Por defecto todos estos dispositivos se asocian con la consola, de forma que los datos se introducen por teclado, y los resultados y errores se muestran en pantalla.

1.6.5. Permisos sobre los ficheros y directorios

Ya sabemos que los usuarios normales no pueden acceder a los ficheros de los otros usuarios, a menos que se de permiso explícito. Veamos cómo funciona el mecanismo de permisos.

Cada fichero de UNIX (ficheros, directorios, dispositivos, etc.) tiene unos determinados *permisos*. Los permisos son de tres tipos, que pueden estar o no activados (si está activado, tenemos el permiso), y tienen un significado diferente para ficheros y directorios. Para ficheros:

- ***Permiso de lectura***: nos permite ver el contenido del fichero.
- ***Permiso de escritura***: nos permite modificar el contenido del fichero.
- ***Permiso de ejecución***: nos permite ejecutar el fichero (que se supone que es un programa).

Para directorios:

- ***Permiso de lectura***: nos permite conocer el contenido del directorio (listado de ficheros), pero no permite ver información detallada (tamaño, fecha, etc.) de los ficheros a menos que tengamos también permiso de

ejecución. Hay que distinguir entre poder saber qué ficheros hay (permiso de lectura sobre un directorio) y qué contiene cada fichero (permiso de lectura sobre cada fichero).

- ***Permiso de escritura***: nos permite modificar el contenido del directorio: añadir ficheros y eliminar ficheros (siempre que tenga permiso de escritura sobre los ficheros). Recordemos que para poder modificar los contenidos de los ficheros se requiere permiso de escritura sobre los ficheros.
- ***Permiso de ejecución***: nos permite que podamos cambiar el directorio por defecto a ese directorio, así como copiar ficheros del directorio (siempre que además tengamos los permisos de lectura de los ficheros).

Solemos referirnos a estos permisos de ficheros y directorios como *r*, *w* y *x* (lectura, escritura y ejecución, respectivamente). Así, si tenemos permisos *rx*, podemos *leer* y *ejecutar*, pero no *modificar*. Además, estos permisos se dan en función del usuario:

- ***Permisos para el usuario propietario***: indican los permisos (*r*, *w*, *x*) que tiene el usuario propietario del archivo. El propietario de un archivo es el usuario que lo creó por primera vez, aunque es posible cambiar de propietario con el comando adecuado (`chown`).
- ***Permisos para los usuarios del mismo grupo que el propietario***: indican los permisos (*r*, *w*, *x*) que tienen los usuarios que pertenecen al mismo grupo que el usuario propietario del archivo.
- ***Permisos para los otros usuarios***: indican los permisos (*r*, *w*, *x*) que tienen los usuarios que no pertenecen al grupo del usuario propietario.

Solemos referirnos a estos permisos como *U*, *G* y *O* (usuario –propietario–, grupo y otros, respectivamente). También se suelen representar con variantes de la cadena *rwxrwxrwx*, donde los permisos aparecen en el orden *UGO*, de manera que si el permiso existe se pone la letra correspondiente, y si el permiso no existe se pone un guión. Por ejemplo: *r--r--r--*, *rwxr-xr-x*, *rwx------*, etc.

Los permisos de un fichero pueden cambiarse, con el comando `chmod`, por usuario propietario y por el administrador, aunque no tengan permiso de escritura sobre el fichero.

Habitualmente, junto con los permisos se suele representar el tipo de fichero utilizando un carácter que se coloca antes de los permisos: directorio (`d`), fichero (`-`), dispositivo de caracteres (`c`), dispositivo de bloques (`b`), enlace simbólico[2] (`l`), etc. Por ejemplo, *drwxr-xr-x* son los permisos de un directorio, mientras que *-rwxr-xr-x* son los permisos de un fichero normal.

Además de estos permisos, existe otro: el permiso de establecimiento del propietario y del grupo, pero no los estudiamos.

1.7. Procesos en UNIX

Sabemos que UNIX es sistema operativo multiproceso, lo que quiere decir que en un solo ordenador se pueden ejecutar a la vez varios procesos (del mismo o de diferentes usuarios). Cada comando o programa que se está ejecutando es un proceso. Los procesos se identifican en la máquina UNIX mediante un número llamado *identificador de proceso (PID)*.

El intérprete de comandos es un proceso. Cada vez que ejecuta un comando crea un nuevo proceso para ese comando, y espera a que termine para aceptar nuevos comandos. Por lo tanto, cada comando que invocamos es un nuevo proceso.

[2] Estudiaremos los enlaces simbólicos en el apartado 3.1.9 *Comando ln*, en la página 33.

Existen varios comandos que permiten manejar los procesos. El comando `ps` sirve para listar los procesos que están ejecutándose en un momento determinado. Con él podemos averiguar no sólo qué procesos hay, sino también su PID, su estado y otras propiedades.

Normalmente no necesitamos conocer el PID de un proceso, pero a través de él podemos comunicarnos con un proceso mediante el *envío de señales*. Podemos considerar una señal como un mensaje con un número. Cuando enviamos una señal (comando `kill`) debemos identificar el proceso destinatario (mediante su PID) y el número de señal que queremos enviar. Cuando un proceso recibe una señal pueden ocurrir dos cosas:

- Si el número de señal era esperado por el proceso, éste efectúa alguna acción en respuesta a la señal, y después continúa con su actividad normal.
- Si el número de señal no era esperado por el proceso, éste termina (muere) de forma inmediata.

Por lo tanto, una de las utilidades del envío de señales es matar procesos.

2. Intérpretes de comandos

El *intérprete de comandos* es la interfaz elemental de UNIX y nos permite escribir órdenes que efectúan acciones sobre el sistema: ejecutar programas, manejar ficheros. Un intérprete de comandos no es más que un programa que espera a que escribamos una orden, y entonces la ejecuta. En este sentido, es un interface, una capa sobre el sistema operativo que sirve para facilitar el trabajo (es este caso de ejecutar comandos) al usuario. Por esta razón, los intérpretes de comandos reciben el nombre de ***shell*** (caparazón).

En realidad el término shell es más general que el de intérprete de comandos, ya que un shell pude ejecutar comandos sin necesidad de interpretar un texto. Por ejemplo, en MS-DOS el popular ***Comandante Norton***, su equivalente en Linux, ***Midnight Comander***, el ***Explorer*** de Windows, etc. son ejemplos de shell que no son intérpretes de comandos.

Existen multitud de intérpretes de comandos para UNIX, cada uno con sus opciones y sintaxis particular, aunque todos ellos son semejantes. Algunos ejemplos: `sh`, `csh`, `ash`, `bash`, `ksh`.

Cuando un usuario es dado de alta en un sistema UNIX, además del nombre de usuario, grupo, directorio *home*, etc., se le asigna un shell por defecto, que es el que se utiliza cada vez que el usuario entra en el sistema. Este shell puede cambiarse utilizando el comando `chsh`. Además, puesto que el shell no es más que un programa, desde nuestro shell por defecto podemos invocar otro shell cualquiera como un comando más.

2.1. bash

bash (*Bourne Again Shell*) es el más popular de los shell incluidos con Linux, por lo que nos centraremos en su estudio. *bash* lee los comandos de la entrada estándar (teclado). Una orden (comando o comandos introducidos en la misma línea) está formada por una línea de caracteres terminada con la pulsación de la tecla *Enter* (*Intro*, retorno de carro). Dentro de la orden debe aparece uno o más comandos. Podemos incluir varios comandos separándolos con algunos caracteres especiales reservados a tal efecto. Mientras los comandos de una orden se ejecutan, *bash* espera, y una vez terminados, nos vuelve a mostrar el *prompt* para que escribamos una nueva orden.

Para facilitar la introducción de órdenes, bash mantiene un historial de las últimas órdenes introducidas. Pulsando los cursores arriba (↑) y abajo (↓) accedemos a las órdenes anteriores. Además, podemos modificar las órdenes escritas moviéndonos por la orden con las flechas izquierda (←) y derecha (→) y modificando el texto.

Por último, *bash* permite utilizar la tecla de tabulación para completar automáticamente el nombre de un archivo que ya exista en el disco. Por ejemplo, para teclear el comando "`ls /etc/passwd`", nos basta con teclear "`ls /etc/pa`" y pulsar el tabulador. En el caso de que con la fracción del nombre tecleada *bash* no pueda determinar de qué archivo se trata (por que no exista ningún archivo con esa raíz, o porque existan varios), emitirá un pitido. Si volvemos a pulsar el tabulador, nos mostrará una lista de los posibles archivos que casan con la raíz introducida, pero deberemos completar el nombre a mano (al menos hasta resolver la ambigüedad).

2.1.1. Caracteres especiales

bash reserva un conjunto de caracteres para utilizarlos como símbolos con un significado especial (Tabla 6). En primer lugar tenemos los comodines, cuyo significado no depende exclusivamente del shell, sino que se utilizan de forma más o menos idéntica en cualquier sistema UNIX. Antes de ejecutar un comando (un programa en general), el

shell sustituye los parámetros con comodines por el conjunto de nombres de ficheros que casen con dichas máscaras, de manera que el comando no recibe como parámetro la máscara, sino el conjunto de ficheros ya localizados[3].

Todos los caracteres especiales pueden utilizarse también como caracteres normales (por ejemplo para que formen parte de un nombre de fichero). Para ello es necesario utilizar otros caracteres especiales, cuya misión es indicar que otros caracteres se utilicen como literales (escaparlos):

- La barra invertida (`\`) hace que el carácter que aparezca a continuación no tenga significado especial.
- Las comillas normales simples (´) hacen que los caracteres encerrados entre comillas no tengan significado especial, sino que se utilicen como literales.
- Las comillas dobles (`"`) hacen que los caracteres encerrados entre ellas no tienen significado especial (igual que las comillas normales simples), excepto si los símbolos son cualquiera de los siguientes: `$`, `` ` `` o `\`.

Por ejemplo, para utilizar el nombre "*`abc*def$ghi\jkl mno`*" tenemos las siguientes posibilidades:

- Utilizando la barra invertida delante de cada carácter especial: `abc\*def\$ghi\\jkl\ mno`.
- Encerrando el nombre entre comillas normales simples: `´abc*def$ghi\jkl mno´`.
- Encerrando el nombre entre comillas dobles y empleando una barra invertida delante de `$` y `\`: `"abc*def\$ghi\\jkl mno"`.

También podemos, por supuesto, anular el significado especial de los símbolos de anulación de significado especial:

- `'\'` introduce una barra inversa.
- `\\` introduce una barra inversa.
- `\´` introduce una comilla simple
- `´"´` introduce unas comillas dobles
- `\´\\\´` introduce una barra inversa encerrada entre comillas simples
- etc.

Otros caracteres especiales sirven para redirección de entradas y salidas. Los estudiamos más adelante en el apartado *Redirección de E/S y tuberías*.

El carácter de comilla inversa simple (`` ` ``) sirve para sustituir en un comando un conjunto de caracteres que representan un comando por el resultado de ese comando. En el siguiente ejemplo, `pwd` devuelve el directorio actual; a ese resultado se le encadena `/fd*`, y eso se utiliza como máscara para listar los archivos con `ls`.

```
$ ls `pwd`/fd*
```

El carácter de dólar (`$`), antepuesto a una cadena de caracteres sirve para que la esa cadena se sustituya por el valor de la variable identificada con la cadena. Por ejemplo, sabiendo que HOME es una variable que contiene la ruta del directorio *home* de un usuario, podemos listar el contenido de ese directorio mediante la siguiente instrucción:

[3] En el caso de que una máscara no case con ningún nombre de archivo, se interpreta dicha máscara como un nombre literal, pasándose al comando como parámetro la máscara tal cual.

```
$ ls $HOME
```

El uso de `{}` también está relacionado con las variables. Estudiaremos las variables en la sección *2.1.5 Variables de entorno* en la página 24.

Los símbolos `&` y `;` se utilizan para separar comandos. En una misma orden (línea de comandos) pueden introducirse varios comandos, separados por alguno de estos símbolos:

- Si utilizamos `;` la ejecución de los comandos será secuencial (primero el de la izquierda y luego el de la derecha).
- Si utilizamos `&` la ejecución de los comandos será concurrente (el comando de la izquierda y el de la derecha se lanzan a la vez).

En el primer ejemplo se lista en primer lugar el contenido del directorio *home* y a continuación el del directorio */bin*; en el segundo ejemplo el orden de listado es impredecible, e incluso se pueden mezclar ambos listados.

```
$ ls $HOME ; ls /bin
$ ls $HOME & ls /bin
```

`&` suele utilizarse para ejecutar comandos en segundo plano. Por ejemplo, la siguiente orden ejecuta el listado en segundo plano, y devuelve el *prompt* de forma inmediata[4] para que podamos escribir una nueva orden sin tener que esperar a que el comando `ls` termine:

```
$ ls /bin &
```

[4] El texto del *prompt* se mezclará con la salida del comando, por lo que probablemente dé la sensación de que no ha aparecido.

Carácter	Significado
Comodines	El uso de los comodines no depende de *bash* . `*`, `?`, `[]` y `!` suelen tener el mismo significado en cualquier shell.
\	Hace que el carácter que aparezca a continuación no tenga significado especial.
´	Hace que los caracteres encerrados entre comillas no tengan significado especial.
"	Los caracteres encerrados entre comillas no tienen significado especial, excepto si son `$`, ` o \
`	Los caracteres encerrados entre las comillas invertidas con un comando y se sustituyen por el resultado de sus ejecución.
espacio	Separador de argumentos.
`{}`	Las llaves sirven para encerrar nombres de variables.
`()`	Los paréntesis se emplean para especificar prioridades en la ejecución de comandos.
`$`	Lo que viene a continuación es el nombre de una variable, y se sustituye por su valor.
`;`	Separa dos comandos que se ejecutan uno a continuación del otro (primero el de la izquierda).
`&`	Separa dos comandos que se ejecutan a la vez (de forma concurrente).
>	Redirecciona la salida estándar a un archivo nuevo.
>>	Redirecciona la salida estándar para añadir a un archivo existente.
<	Redirecciona la entrada estándar.
|	Separa dos comandos que se ejecutan a la vez (de forma concurrente), de forma que la salida estándar del comando de la izquierda se comunica con la entrada estándar del comando de la derecha mediante una tubería.

Tabla 6. Resumen de caracteres especiales en bash

2.1.2. Los alias

Los alias son un mecanismo que proporciona el shell para facilitarnos la escritura de comandos. Un alias se algo parecido a una variable, que sirve para dar otros nombres a los comandos. Por ejemplo podemos definir un nuevo comando "`dir`" cuyo equivalente es "`ls`":

```
alias dir='ls'
```

A partir de este momento podemos utilizar `dir` igual que `ls`. Pero la utilidad de los alias va más allá, ya que podemos darle cualquier valor al alias, por ejemplo, para ejecutar un comando con parámetros:

```
alias dir='ls -al'
```

Ahora podemos utilizar `dir` como `ls`, añadiéndole cuantos parámetros sean necesarios, pero no será necesario que especifiquemos `-al` porque ya estará incluido. Pero podemos ir más allá todavía, y es que podemos dar al alias el mismo nombre que el comando:

```
alias ls='ls -al'
```

De esta manera lo que conseguimos es darle a un comando un conjunto de opciones por defecto, de manera que no tendremos que especificarlas cada vez que ejecutemos el comando.

Finalmente, para ver el conjunto de alias definidos, podemos escribir `alias` sin parámetros[5]:

```
polifemo:~$ alias
alias d='dir'
alias dir='/bin/ls $LS_OPTIONS --format=vertical'
alias joe='joe -asis'
alias ls='/bin/ls $LS_OPTIONS'
alias v='vdir'
alias vdir='/bin/ls $LS_OPTIONS --format=long'
```

Vemos en este ejemplo cómo el comando `ls` está definido como un comando con una ruta completa, más un conjunto de opciones algunas de las cuales se toman de una variable de entorno.

Así, la secuencia correcta de interpretación de un comando consiste en: sustitución de alias, sustitución de variables, sustitución de comodines, ejecución. Podemos ver el aspecto del comando realmente ejecutado mediante las opciones que proporciona el shell, y que estudiamos a continuación.

Para que un alias sea sustituido por su valor correcto, debe aparecer como primera palabra de una orden, ya que de lo contrario se interpreta, no como comando, sino como parámetro, y entonces no se sustituye por su valor de alias.

2.1.3. Configuración de opciones de bash

El comportamiento del shell *bash* ante determinadas circunstancias puede ser modificado mediante la configuración de un conjunto de opciones mediante el comando `set`. Este comando lo estudiaremos más adelante como utilidad para listar las variables de entorno definidas, pero ahora lo vamos a utilizar como método de configuración.

bash proporciona un conjunto de opciones que pueden ser activadas o desactivadas. Por defecto aparecen desactivas, pero podemos activarlas mediante el comando `set`. Por ejemplo `set -x` activa la opción "`x`". Luego, con `set +x` podemos volver a desactivar la opción "`x`".

La opción –x, cuando es activada hace que *bash* muestre, antes de ejecutar un comando, la verdadera secuencia de parámetros que se ha pasado al comando. Esto incluye la sustitución de alias por el comando y opciones verdaderos, la sustitución de los nombres de variables por sus valores verdaderos, sustitución de máscaras y comodines por listas de ficheros, etc. De esta manera podemos analizar lo que realmente se ejecuta cuando invocamos un comando, lo cual nos puede dar una idea precisa de por qué ocurren algunas cosas "misteriosamente".

Por ejemplo, cuando invocamos `ls *.c`, realmente estamos haciendo mucho más que eso. Vemos, en la línea que comienza por `+` el verdadero comando, que incluye la ruta completa al comando, las opciones definidas mediante el alias y la sustitución de la máscara `*.c` por el conjunto de ficheros que casan.

```
polifemo:~$ ls *.c
+ /bin/ls --8bit --color=tty -F -b -T 0 busqueda.c cgi.c cgi00.c libreria.c prueba.c
busqueda.c   cgi.c        cgi00.c      libreria.c   prueba.c
```

[5] Estudiaremos el comando `alias` con el resto de comandos más adelante.

Igualmente, en el siguiente comando se puede ver la sustitución de una variable por su valor:

```
polifemo:~$ echo Mi directorio es $HOME
+ echo Mi directorio es /home/llorente
Mi directorio es /home/llorente
```

Otra opción interesante de bash es la `-v`. Es parecida a `-x`. Hace que antes de ejecutar un comando se muestre el comando que se va a ejecutar, pero a diferencia de `-x`, con `-v` no se realiza ninguna sustitución, sino que se muestra el comando tal cual se escribió. Esto no tiene ninguna utilidad mientras escribimos comandos, ya que volvemos a ver lo que acabamos de escribir. Pero resulta realmente interesante cuando ejecutamos *scripts* (programas hechos como secuencias de comandos), ya que entonces podemos hacer una traza de lo que se está ejecutando en un script, que de otra manera no podemos ver. Estudiaremos los scripts y esta opción en el capítulo 5, *Scripts en bash*, en la página 91.

La opción –f permite hacer que bash no realice ninguna sustitución de máscaras con comodines por nombres de ficheros que casen, sino que pasa a los comandos los comodines como caracteres normales. Por ejemplo:

```
polifemo:~$ set -f
polifemo:~$ ls *
/bin/ls: *: No such file or directory
```

2.1.4. Redirección de E/S y tuberías

Cuando ejecutamos un comando en *bash*, por defecto, el comando utilizará como entradas y salidas estándar los mismos dispositivos que *bash*, es decir, la consola (pantalla y teclado). Sin embargo, podemos hacer que estos dispositivos sean otros, por ejemplo, para que el resultado de un comando, en lugar de mostrarse en pantalla, se almacene en un fichero, se envíe a la impresora, se descarte, etc. Esto se denomina ***redirección*** de entradas y salidas, y para hacerlo debemos indicar, junto con el comando, el conjunto de redirecciones necesarias.

- Para ***redireccionar la entrada estándar*** utilizamos el carácter menor que (<) seguido del nombre de fichero o dispositivo de donde de tomará la información de entrada.
- Para ***redireccionar la salida estándar*** utilizamos el carácter mayor que (>) seguido del nombre de fichero o dispositivo a donde irá a parar la información de salida. Tenemos dos posibilidades: crear un nuevo fichero con el contenido de la salida, o bien añadir la salida a un fichero que ya existe. En este último caso utilizamos dos signos mayor que (>>) en vez de uno antes del nombre del fichero.
- Para ***redireccionar la salida de errores*** utilizamos los caracteres `&>` (o también `>&`) seguidos del nombre de fichero o dispositivo a donde irá a parar la información de los errores.

La Tabla 7 muestra algunos ejemplos de redirección.

A veces conviene que la salida de un programa se utilice como entrada de otro. Por ejemplo, nos puede interesar listar los archivos de un directorio, pero haciendo una pausa cada vez que se llene la pantalla[6]. Podemos hacer la siguiente secuencia:

```
$ ls >temporal
```

[6] El comando `more` sirve para hacer que una fichero se muestra por páginas, haciendo una pausa cada vez que se llena una pantalla.

```
$ more <temporal
```

Esta es una operación muy habitual; por eso existe un mecanismo que nos permite reunir los dos comandos en una única orden. En vez de utilizar un fichero auxiliar (`temporal` en el ejemplo) se utiliza lo que se denomina una ***tubería*** (*pipe*). Una tubería es como un fichero en el que un comando escribe y el otro lee, pero no tiene una representación real en el disco, sino que se crea en memoria de forma automática, y se destruye una vez terminada la orden. Para utilizar una tubería utilizamos el símbolo de raya vertical (`|`, *pipe*). El ejemplo anterior quedaría:

```
$ ls | more
```

Se pueden encadenar varios comandos con varias tuberías. Todos los comandos se ejecutan de forma concurrente, de forma que la salida de cada comando a la izquierda de un `|` se utiliza como entrada en el comando inmediatamente a la derecha de `|`.

Comando	Significado
`ls >mifichero`	Lista los archivos del directorio y los escribe en el fichero `mifichero`. Si este fichero ya existe, el contenido anterior se pierde.
`ls >>mifichero`	Lista los archivos del directorio y los añade al fichero `mifichero`. Si este fichero no existe se crea.
`ls &>mifichero`	Lista los archivos del directorio en pantalla y escribe los posibles errores en el fichero `mifichero`. Si este fichero ya existe, el contenido anterior se pierde.
`ls &>/dev/null`	Lista los archivos del directorio en pantalla y descarta los posibles errores.
`more <mifichero`	Muestra en pantalla el contenido de `mifichero` haciendo una pausa en cada página.
`ls | more`	Muestra en pantalla la lista de ficheros haciendo una pausa en cada página.

Tabla 7. Ejemplos de redirección

2.1.5. Variables de entorno

Las variables de entorno son un conjunto de variables que están presentes en el sistema UNIX y que algunos programas, así como el propio shell, utilizan para intercambiar información, conocer datos de configuración, etc. Una variable se identifica mediante una cadena de caracteres, y su valor es siempre otra cadena de caracteres (recordemos una vez más que se diferencia entre mayúsculas y minúsculas).

Para definir el valor de una variable utilizamos una asignación (no necesitamos ningún comando). Por ejemplo, para definir la variable `TEMPORAL` con el valor `/tmp` hacemos:

```
$ TEMPORAL=/tmp
```

Para utilizar la variable, anteponemos el símbolo $. Por ejemplo, la siguiente orden muestra el contenido del directorio almacenado en la variable `TEMPORAL`:

```
$ ls $TEMPORAL
```

Al definir variables podemos utilizar otros caracteres especiales, como en cualquier comando. Por ejemplo:

```
$ ESTE_DIR=`pwd`
$ ls ${ESTE_DIR}/*.c
```

En este ejemplo se han utilizado las llaves (`{}`) para delimitar el nombre de la variable. Si se hubieran omitido (`ls $ESTE_DIR/*.c`), el shell podría pensar que el nombre de la variable incluye, al menos, el barra de directorio (`/`), y entonces el valor de la misma no sería el esperado.

Si intentamos utilizar el valor de una variable no definida, obtendremos como resultado una cadena vacía (nunca un error).

Para utilizar el símbolo `$` en un comando sin que denote una variable, debemos encerrarlo entre comillas simples (´) o anteponer una barra inversa (\).

Las variables definidas en un shell son locales a ese shell. Para hacer que los programas lanzados por un shell conozcan el valor de esas variable es necesario *exportarlas*, con el comando `export` (ya lo estudiaremos).

Existen algunas variables predefinidas de uso muy común, que están reflejadas en la Tabla 8. La más importante de todas ellas es la variable **`PATH`**, que almacena un conjunto de rutas de búsqueda para localizar comandos (*path de búsqueda*). Así, no es necesario escribir la ruta completa a un comando, siempre que esa ruta esté incluida entre las rutas del la variable `PATH`. `PATH` almacena un conjunto de rutas separadas por dos puntos (`:`).

Comando	Significado
`PATH`	Rutas de búsqueda de comandos.
`HOME`	Ruta al directorio *home*.
`PS1`	*Prompt* principal.
`PS1`	*Prompt* secundario.
`SHELL`	Comando utilizado como shell.
`LOGNAME`	Nombre utilizado en el *login* (nombre de usuario).
`HOSTNAME`	Nombre completo de la máquina UNIX

Tabla 8. Variables de entorno más importantes

Podemos añadir nuevas rutas al path utilizando el valor de la variable. Por ejemplo, para añadir el directorio `/tmp` al path:

```
$ PATH=${PATH}:/tmp
```

Otras variables importantes son las que definen el *prompt*. Hasta ahora sabemos que el prompt sirve para que el shell nos indique que está listo para recibir comandos, y que suele ser un símbolo (p.e. `$`). En realidad existen varios prompt. El principal es el que conocemos. Pero a veces algunos comandos, al ejecutarlos piden más información por teclado, y entones se muestra un prompt secundario, terciario, etc.

El aspecto de todos los prompt se almacena en las variables PS1, PS2, etc., siendo PS1 el principal. En estas variables se almacena la cadena de caracteres que se muestra como prompt, aunque existen algunos códigos que tienen funciones especiales (como si fueran caracteres especiales). La Tabla 9 recoge algunos de los códigos más

habituales utilizados en las variables de prompt. Todos comienzan por el símbolo \, por lo que al definir la variable habrá que utilizar doble barra o encerrar el valor entre comillas. El siguiente ejemplo construye un prompt principal formado por el nombre de la máquina, seguido de dos puntos, el directorio de trabajo y un símbolo `$` o `#`:

```
PS1=\\h:\\w\\$
```

Código	Significado
`\h`	Nombre del computador Linux.
`\w`	Ruta al directorio por defecto.
`\t`	La hora actual en formato HH:MM:SS.
`\d`	La fecha actual en formato "*DiaSemana Mes Día*" (por ejemplo "Tue May 26").
`\u`	El nombre del usuario.
`\$`	El símbolo `$` si es un usuario normal o `#` si es el administrador.

Tabla 9. Códigos para las variables prompt

3. Comandos

En este apartado vamos a estudiar detalladamente los principales comandos presentes en UNIX, agrupándolos en las siguientes categorías:

- Manejo de ficheros.
- Procesamiento de ficheros.
- Manejo de unidades y dispositivos.
- Manejo de procesos.
- Manejo de variables.
- Administración del sistema.
- Otros.

Para cada comando que estudiemos, haremos una descripción del formato de las opciones del comando, utilizando las convenciones de la Tabla 10:

Código	Significado
[parámetros]	El parámetro o parámetros encerrados son opcionales.
{opción1|opción2}	Debe especificarse alguna de las opciones encerradas entre las llaves (una o varias).
`parametro...`	Los puntos suspensivos indican que el parámetro puede ser una lista de parámetros.

Tabla 10. Convenciones para describir la sintaxis de los comandos

Para cada comando podemos obtener información desde la línea de comando mediante el modificador `--help` (sólo en Linux) y mediante el comando `man` (en cualquier UNIX).

3.1. Comandos de manejo de ficheros

En esta categoría estudiamos los comandos relacionados con el manejo de ficheros (manipulación de los nombres de fichero, no de sus contenidos).

3.1.1. Comando ls

`ls` significa *list subdirectory*. Sirve para mostrar los nombres de los ficheros y directorios que hay en un directorio. La sintaxis es:

SINTAXIS

```
ls [modificadores] [ficheros...]
```

Los modificadores permiten especificar diversos formatos y opciones. Los ficheros son la lista de máscaras de los ficheros que hay que listar. Si se omite la máscara, se listarán todos los ficheros. Así, el comando sin parámetros muestra una lista en varias columnas de los ficheros del directorio actual, ordenados alfabéticamente por columnas (de arriba abajo y luego de izquierda a derecha; mayúsculas y minúsculas se ordenan por separado, primero las mayúsculas):

```
polifemo:/boot$ ls
System.map  any_d.b     chain.b     os2_d.b
any_b.b     boot.b      config
```

Podríamos haber indicado una máscara, para listar sólo algunos de los ficheros:

```
polifemo:/boot$ ls *.b
any_b.b  any_d.b  boot.b   chain.b  os2_d.b
```

O también una lista de máscaras:

```
polifemo:/boot$ ls a*.b b*
any_b.b  any_d.b  boot.b
```

En estos listado no se muestran todos los archivos. Los archivos que comienzan por un punto (`.`) se consideran archivos "especiales", y no se listan normalmente, a menos que especifiquemos lo contrario. Esto lo hacemos con el modificador `-a`:

```
polifemo:/boot$ ls -a
./          System.map  any_d.b     chain.b     os2_d.b
../         any_b.b     boot.b      config
```

Observamos que los nombres de directorios se distinguen de los nombres de ficheros por que aparecen seguidos de una barra inclinada (`/`). Dependiendo de la versión de Linux y el tipo de terminal que utilicemos es posible que cada fichero o directorio listado aparezca de un color diferente. Este color indica alguna propiedad del fichero (tipo, permisos, etc.). Con `ls`, además del nombre de los ficheros, podemos obtener información sobre los mismos, con el modificador `-l`:

```
polifemo:/boot$ ls -l
total 157
-rw-r--r--   1 root     root       143835 Jun 24  1997 System.map
-rw-r--r--   1 root     root          204 May 25  1996 any_b.b
-rw-r--r--   1 root     root          204 May 25  1996 any_d.b
-rw-r--r--   1 root     root         4416 May 25  1996 boot.b
-rw-r--r--   1 root     root           88 May 25  1996 chain.b
-rw-r--r--   1 root     root         5147 Jun 24  1997 config
-rw-r--r--   1 root     root          192 May 25  1996 os2_d.b
```

La información que aparece en las columnas es la siguiente (de izquierda a derecha): permisos (y tipo), número de enlaces que existen sobre el fichero, el propietario, el grupo del fichero, el tamaño en bytes, la fecha de modificación y el nombre. Además, al principio del listado aparece el número total de bloques de disco (de 1 kilobyte normalmente, aunque también pueden ser de 512 bytes) que ocupan los ficheros listados.

El código de colores no puede utilizar siempre, ya que es muy habitual utilizar terminales monocromo para conectarnos a las máquinas UNIX. Por eso, en vez del código de colores, podemos utilizar un código de caracteres

para determinar el tipo de los ficheros listados. Esto lo hacemos con el modificador `-F` (a veces se utiliza esta opción por defecto, aunque no la especifiquemos):

```
polifemo:/lib$ ls -F
cpp@                    libcurses.so.1@         libm.so.5@
ld-linux.so@            libcurses.so.1.0.0*     libm.so.5.0.9*
ld-linux.so.1@          libdl.so@               libncurses.so@
ld-linux.so.1.9.5*      libdl.so.1@             libncurses.so.1.9.9e@
ld.so*                  libdl.so.1.9.5*         libncurses.so.3.0@
libc.so.4@              libe2p.so.2@            libncurses.so.3.0.0*
libc.so.4.7.6*          libe2p.so.2.3*          libss.so.2@
libc.so.5@              libext2fs.so.2@         libss.so.2.0*
libc.so.5.4.33*         libext2fs.so.2.3*       libtermcap.so.2@
libcom_err.so.2@        libgdbm.so.1@           libtermcap.so.2.0.8*
libcom_err.so.2.0*      libgdbm.so.1.7.3*       libuuid.so.1@
libcurses.so.0@         libm.so.4@              libuuid.so.1.1*
libcurses.so.0.1.2*     libm.so.4.6.27*         modules/
```

El código se añade al nombre del fichero: `*` indica un fichero ejecutable, `/` indica un directorio, `@` indica un enlace simbólico[7], etc. Cuando se trata de un archivo normal, no se añade nada.

La Tabla 11 resume los principales modificadores del comando `ls`.

Modificador	Resultado
`-1`	Lista los ficheros en una única columna (un fichero por línea).
`-l`	Formato largo: muestra información de los ficheros (premisos, propietario, fecha, etc.)
`-R`	Recursivo: para cada directorio, muestra también su contenido de forma recursiva.
`-a`	Todos los ficheros: incluye los ficheros que comienzan por un punto (`.`).
`-A`	Casi todos los ficheros: incluye los ficheros que comienzan por un punto (`.`), excepto los directorios "." y "`..`".
`-B`	Sin *backups*: no muestra los ficheros de *backup*[8] (copia de seguridad), que lo los ficheros que terminan en "~".
`-d`	Lista el nombre de un directorio en lugar de su contenido.
`-t`	Ordena los ficheros listados por la fecha y hora de modificación.
`-S`	Ordena los ficheros listados por su tamaño.
`-X`	Ordena los ficheros listados por su extensión.
`-r`	Ordena los ficheros de forma inversa.
`-U`	No ordena los ficheros de ninguna forma; los lista en el orden en que aparecen en el disco.
`-s`	Muestra el tamaño de un archivo a la izquierda de su nombre, medido en bloques.
`-F`	Muestra un carácter al final del nombre, que indica el tipo de fichero (como el código de colores).
`-i`	Muestra el inodo (la zona de disco) en que se ubica la información del fichero.

Tabla 11. Modificadores para el comando `ls`

[7] Estudiaremos los enlaces simbólicos en el apartado 3.1.9 *Comando ln*, en la página 33.

[8] Muchas aplicaciones en UNIX que manipulan documentos de datos, cuando salvan un documento, guardan la copia anterior del archivo, con el mismo nombre, pero acabado en "~".

Existen varios comandos derivados de `ls` que listan los ficheros con algunas opciones predeterminadas: `dir`, `vdir`, etc.

3.1.2. Comando pdw

`pdw` (*print working directory*) muestra la ruta completa del directorio actual. No admite modificadores ni parámetros.

```
polifemo:~$ pwd
/home/llorente
```

3.1.3. Comando cd

`cd` (*change directory*) sirve para cambiar el directorio por defecto, permitiendo navegar por el árbol de directorios. Admite como parámetro el nombre del directorio al que queremos cambiar:

SINTAXIS

```
cd [destino]
```

Como directorio de destino debemos especificar una ruta absoluta o relativa al directorio actual. Si omitimos el parámetro, `cd` cambia al directorio *home*. En el siguiente ejemplo observamos cómo cambiamos de directorio, viendo el resultado en el *prompt* y mediante `pwd`. Observamos también que el directorio del usuario (en este caso `/home/llorente`) se representa también con el símbolo "~":

```
polifemo:~$ cd /usr/include
polifemo:/usr/include$ pwd
/usr/include
polifemo:/usr/include$ cd ..
polifemo:/usr$ cd .
polifemo:/usr$ cd /
polifemo:/$ cd /home
polifemo:/home$ cd llorente
polifemo:~$ pwd
/home/llorente
polifemo:~$ cd /tmp
polifemo:/tmp$ cd
polifemo:~$ cd /bin
polifemo:/bin$ cd ~
polifemo:~$
```

NOTA:

No debemos olvidar la separación entre el comando (`cd`) y el parámetro. Los usuarios de MS-DOS suelen utilizar a menudo "`cd..`" sin separar el comando de los parámetros, lo cual es correcto en MS-DOS (para ese caso particular), pero no en UNIX.

3.1.4. Comando mkdir

El comando `mkdir` (*make directory*) sirve para crear nuevos subdirectorios, dando lugar a la estructura del árbol de directorios. Podemos crear varios directorios en la misma orden, especificando una lista; los directorios se crearán en el mismo orden de la lista.

SINTAXIS

```
mkdir [opciones] directorios...
```

Los nuevos directorios se pueden especificar mediante una ruta absoluta o relativa al directorio actual. Solamente puede crearse un nivel de directorio de una sola vez. Por ejemplo, si no existe el directorio `/tmp/1`, no podemos crear `/tmp/1/2`. Antes debemos crear `/tmp/1`, o bien utilizar la opción `-p`.

```
polifemo:~$ mkdir /tmp/1/2
mkdir: cannot make directory `/tmp/1/2': No such file or directory

polifemo:~$ mkdir /tmp/1 /tmp/1/2
polifemo:~$ mkdir -p /tmp/1b/2b/3b
```

La Tabla 12 resume los principales modificadores del comando `mkdir`.

Modificador	Resultado
`-p`	Crea recursivamente: antes de crear un directorio, crear su directorio padre (si no existe).
`-m`	Permite especificar los permisos del nuevo directorio.

Tabla 12. Modificadores para el comando `mkdir`

3.1.5. Comando rmdir

El comando `rmdir` (*remove directory*) sirve para borrar subdirectorios vacíos. Podemos borrar varios directorios en la misma orden, especificando una lista. Si un directorio no está vacío, no puede ser borrado. un directorio está vacío cuando únicamente contiene las entradas "`.`" y "`..`", que no se pueden borrar.

SINTAXIS

```
rmdir [opciones] directorios...
```

Podemos utilizar el parámetro -p para hacer borrados recursivos, siempre que la ruta indicada no contenga otros ficheros. Por ejemplo, si hacemos `rmdir -p /tmp/1/2/3`, se intentará eliminar `/tmp/1/2/3`; si no está vacío o no se puede borrar, terminará; si no, luego tratará de borrar `/tmp/1/2`, luego `/tmp/1`, y finalmente `/tmp`.

La Tabla 13 resume los principales modificadores del comando `rmdir`.

Modificador	Resultado
`-p`	Borra recursivamente: después de borrar un directorio, intentar borrar el directorio padre.

Tabla 13. Modificadores para el comando `rmdir`

3.1.6. Comando cp

El comando `cp` (*copy*) sirve para copiar ficheros. Admite como parámetros un fuente y un destino, de forma que el archivo especificado como fuente se copia en la ruta de destino.

SINTAXIS

```
cp [opciones] origen... destino
```

Si la ruta de destino no incluye un nombre de archivo (sólo son nombres de directorios), el nombre del fichero de destino será el mismo que en el origen. Por otro lado, si la ruta de origen es una máscara o una lista de rutas o máscaras, el destino sólo puede ser el nombre de un directorio. Todos los archivos de origen se copiarán al directorio de destino.

Ejemplos:

```
polifemo:~$ cp fichero1 fichero2
polifemo:~$ cp fichero1 /tmp/fichero2
polifemo:~$ cp *.c /tmp
polifemo:~$ cp fichero1 fichero2 /tmp
```

Los archivos recién copiados tienen la fecha actual y como propietario el usuario que realiza la copia, a menos que utilicemos el modificador `-p`. Otras posibles opciones (modificadores) del comando `cp` se resumen en la Tabla 14.

Modificador	Resultado
`-i`	Interactivo: pregunta antes de sobrescribir un fichero existente.
`-l`	En vez de copiar, crea enlaces.
`-s`	En vez de copiar, crea *enlaces simbólicos*.
`-b`	Crea una copia de seguridad de los archivos que sobrescribe.
`-P`	Copia el archivo de origen en el directorio de destino incluyendo la ruta completa de origen.
`-p`	Mantiene la fecha, permisos, propietario y grupo de los archivos origen.
`-R`	Recursivo: copia también los ficheros que estén dentro de directorios.
`-u`	Actualizar: no sobrescribe ficheros a menos que el origen sea más moderno que el destino.
`-v`	Muestra el nombre de cada fichero antes de copiarlo.

Tabla 14. Modificadores para el comando `cp`

3.1.7. Comando rm

El comando `rm` (*remove*) sirve para borrar ficheros. Admite como parámetro una lista de rutas o máscaras para ser borrados.

SINTAXIS

```
rm [opciones] ficheros...
```

Cuando no tenemos permiso de escritura sobre los archivos que vamos a borrar, `rm` nos pedirá confirmación. Si queremos omitir la confirmación podemos utilizar el modificador `-f`. por el contrario, si queremos confirmación para cada fichero (aunque tengamos permiso de escritura) utilizamos `-i`.

Algunos ejemplos de uso de `rm`:

```
polifemo:~$ rm .c *.h
polifemo:~$ rm -f /tmp/*
```

Podemos borrar directorios completos con la opción de recursividad `-R`. El siguiente comando elimina todo el contenido de un directorio (y los subdirectorios) sin confirmación: `rm -fR *`. Debemos tener mucha precaución al utilizar este comando, puesto que su acción no puede recuperarse.

NOTA IMPORTANTE:

No debemos confundir `rm` con el comando de renombrar (`mv`). Muchos usuarios de MS-DOS escriben instintivamente *rm* con la intención de renombrar, ya que se relaciona con el comando de MS-DOS `rename`. **Lo que borremos accidentalmente no puede ser recuperado**.

La Tabla 15 recoge las principales opciones de `rm`..

Modificador	Resultado
`-i`	Interactivo: pide confirmación antes de borrar cada fichero.
`-f`	No pide confirmación ni produce error si un fichero no existe.
`-r`	Recursivo: borrar los ficheros dentro de los subdirectorios y elimina también los subdirectorio.
`-R`	Igual que `-r`.
`-v`	Muestra el nombre de cada fichero antes de borrarlo.

Tabla 15. Modificadores para el comando rm

3.1.8. Comando mv

El comando `mv` (*move*) sirve para cambiar el nombre de ficheros o directorios, o para moverlos dentro del árbol de directorios. Debemos especificar uno o varios archivos de origen y una ruta de destino. Si sólo hay un origen, el destino puede ser un nombre de fichero –con su ruta correspondiente– y `mv` renombrará y/o moverá el archivo; en cambio, si hay varias rutas como origen, o el origen contiene alguna máscara, el destino sólo puede ser un directorio, y `mv` sólo actúa moviendo los archivos a la ruta indicada como destino.

SINTAXIS

```
mv [opciones] origen... destino
```

Las opciones que admite este comando se resumen en la Tabla 16. Son semejantes a las que se utilizan para el comando `cp`.

Modificador	Resultado
`-i`	Interactivo: pregunta antes de sobrescribir un fichero existente en el directorio de destino.
`-b`	Crea una copia de seguridad de los archivos que sobrescribe.
`-u`	Actualizar: no mueve ficheros a menos que el origen sea más moderno que el destino.
`-v`	Muestra el nombre de cada fichero antes de moverlo.

Tabla 16. Modificadores para el comando mv

A continuación vemos los siguientes ejemplos de uso de este comando:

- Renombrar el fichero `copia1` a `copia2`.
- Mover y renombrar el fichero `copia1` a `copia2` en el directorio padre.
- Mover varios ficheros al directorio temporal.
- Mover varios ficheros del directorio temporal al directorio actual.

```
polifemo:~$ mv copia1 copia2
polifemo:~$ mv copia1 ../copia2
polifemo:~$ mv *.tiff /tmp
polifemo:~$ mv /tmp/*.tiff .
```

3.1.9. Comando ln

El comando `ln` (*link*) sirve para crear enlaces. Podemos considerar un enlace como otro nombre para acceder al mismo fichero o directorio. En realidad son un tipo especial de ficheros que no contienen datos, sino una referencia a otro fichero. Los enlaces pueden ser de dos tipos:

- **Enlaces duros** (*hard*): la entrada de directorio (el nombre del enlace) referencia a la zona del disco en la que se encuentran los datos del archivo enlazado. En esa zona del disco, además de los datos se almacena el número de enlaces que tiene ese archivo. El fichero original es también un enlace. Así, al borrar un enlace (aunque sea el del fichero original) se quita la entrada del directorio (el nombre del enlace), pero los datos contenidos no se eliminan del disco. En su lugar se decrementa el número de enlaces restantes y, si no quedan más enlaces, entonces se cuando elimina el contenido del fichero. Cada vez que se añade un enlace para ese fichero, se incrementa el número de enlaces.
- **Enlaces simbólicos**: la entrada de directorio (el nombre del enlace) es fichero nuevo que contiene datos (ocupa espacio en disco), y esos datos son una cadena de texto que referencia al fichero original. El fichero original no funciona como otro enlace simbólico. Si borramos o renombramos o movemos el fichero original, los enlaces simbólicos quedarán inconsistentes, apuntando a un objeto que ya no existe. (Se parece a los accesos directos de Windows).

El comando `ln` admite como parámetro los nombres del fichero original y del enlace:

SINTAXIS

```
ln [opciones] origen... [destino]
```

Como origen especificamos uno o varios ficheros o directorios, o una máscara. Como destino especificamos el nombre del enlace (si el origen es un único archivo o directorio), o bien un directorio donde situar los enlaces creados (en este caso el origen son varios archivos o directorios, o una máscara, y los enlaces se crean con los mismos nombres que el origen, en el directorio de destino). Si omitimos el destino, se supone que es el directorio actual, por lo que el origen debe encontrarse en un directorio diferente del actual.

El principal modificador que podemos utilizar es `-s`, que sirve para que se creen enlaces simbólicos en lugar de enlaces duros.

Por defecto, `ln` no sobrescribe los nombres de ficheros que existan para crear, en su lugar, enlaces. Necesitamos utilizar la opción `-f` para que esto ocurra. Además, con la opción `-i` nos preguntará antes de sobrescribir algún fichero existente.

En circunstancias normales los usuarios no podemos crear enlaces duros a directorios (el sistema sí puede hacerlo). Para directorios sólo podemos hacer enlaces simbólicos. Estos enlaces a directorios se manejan siempre como si fueran realmente directorios, excepto por que para eliminarlos no debemos utilizar `rmdir`, sino `rm`, ya que realmente no son directorios sino un tipo especial de ficheros.

Para identificar los enlaces simbólicos podemos utilizar (entre otras muchas cosas) el comando `ls` con la opción `-l`, ya que junto con los permisos de los ficheros aparece el carácter que identifica el tipo de fichero, que en el caso de los enlaces es "`l`". Por ejemplo: supongamos que en un directorio existen dos entradas: el directorio "`dir1`" y un enlace simbólico a éste, llamado "`otro_dir1`"; al hacer `ls -l` observamos que `dir1` es de tipo directorio, y `otro_dir1` es un enlace que apunta, precisamente, a `dir1` (esto lo vemos junto al nombre, a la derecha de la línea):

```
polifemo:~$ mkdir dir1
polifemo:~$ ln -s dir1 otro_dir1
polifemo:~$ ls -l
total 1
drwxr-xr-x   2 llorente users        1024 Oct 15 09:59 dir1/
lrwxrwxrwx   1 llorente users           4 Oct 15 09:59 otro_dir1 -> dir1/
```

Los enlaces duros no se pueden identificar, ya que son realmente del mismo tipo que sus originales, y no hay manera de distinguirlos de éstos (son idénticos a todos los efectos). Lo que sí podemos es averiguar cuántos enlaces recaen sobre la misma zona de disco (cuántos enlaces apuntan a un fichero). También lo podemos hacer con `ls -l`, ya que la segunda columna indica el número de enlaces que se mantiene para un fichero. En el siguiente ejemplo utilizamos también la opción -a para que se muestren también los directorios "`.`" y "`..`":

```
polifemo:~$ ls -al
total 7
drwx--x--x   3 llorente users        1024 Oct 15 10:08 ./
drwxr-xr-x  16 root     root         5120 Oct 14 11:27 ../
drwxr-xr-x   2 llorente users        1024 Oct 15 10:01 dir1/
lrwxrwxrwx   1 llorente users           4 Oct 15 09:59 otro_dir1 -> dir1/
```

Observamos que:

- El enlace `otro_dir1` sólo tiene 1 enlace, ya que nadie más enlaza con él.
- El directorio `dir1` tiene dos enlaces: el suyo propio, y el directorio "`dir1/.`", que es un enlace duro a `dir1` creado por el sistema cuando se creó (p.e. con `mkdir`) el directorio `dir1` (¡no se cuentan los enlaces simbólicos!).
- El directorio "`.`" tiene 3 enlaces: el propio "`.`", el de "`/home/llorente`" y el de "`/home/llorente/dir1/..`".
- El directorio "`..`" tiene múltiples enlaces: el propio "`..`" (su ruta completa es "`/home/llorente/..`"), el de "`/home`" , el de "`/home/.`" y el precedente del "`..`" de otros 13 directorios `home` de otros usuarios (de la forma "`/home/`*`USUARIO`*`/..`").

Los distintos enlaces duros a un mismo archivo comparten la siguiente información (aparte de los datos contenidos en fichero, su tamaño, etc.):

- El tipo de archivo.
- El propietario.

- El grupo.
- Los permisos.
- La fecha de modificación, acceso, etc.

La Tabla 17 resume las principales opciones que admite el comando `ln`. Muchas de ellas son semejantes a las utilizadas en otros comandos.

Modificador	Resultado
`-s`	Indica que los enlaces que se deben crear serán simbólicos (y no duros)
`-f`	Indica que se sobrescriban los ficheros existentes en el destino si es necesario.
`-i`	Interactivo: pregunta antes de sobrescribir un fichero existente en el directorio de destino.
`-b`	Crea una copia de seguridad de los archivos que sobrescribe.
`-v`	Muestra el nombre de cada fichero antes de crear enlace.

Tabla 17. Modificadores para el comando `ln`

Algunos ejemplos de uso de `ln`:

- Crear un enlace simbólico de todos los dispositivos de tipo *tty* en un subdirectorio llamado `ttys`.
- Crear un enlace al directorio de dispositivos llamado `acceso_a_dispositivos`.
- Hacer un enlace de todos los ficheros que tenga el usuario *fjgarcia* en su directorio `practicas` a nuestro directorio `practica_copiada`, sobrescribiendo los ficheros que pudiera haber en éste directorio.
- Hacer un enlace del fichero `/home/profesor/notas_practicas` al fichero `mi_susupenso`.

```
polifemo:~$ ln -s /dev/tty* ttys
polifemo:~$ ln -s /dev/tty acceso_a_dispositivos
polifemo:~$ ln -f /home/fjgarcia/practica/* ~/practica_copiada
polifemo:~$ ln /home/profesor/notas_practicas mi_suspenso
```

Debemos prestar especial atención cuando construyamos enlaces simbólicos entre objetos ubicados en directorios diferentes utilizando rutas relativas, ya que el enlace buscará el objeto original siguiendo la ruta especificada en el momento de la creación, pero interpretándola como relativa a la ubicación actual del enlace. Veamos este problema con un ejemplo.

Supongamos que tenemos dos directorios: `dir1` y `dir2`. Dentro de `dir1`, tenemos el fichero `mi_fichero`, y ahora queremos crear un enlace simbólico al fichero, desde el directorio `dir2`:

Creación de la estructura

```
polifemo:~$ mkdir dir1
polifemo:~$ mkdir dir2
polifemo:~$ cat >dir1/mi_fichero
Este es el contenido de mi_fichero
^D
```

El siguiente intento de crear el enlace es incorrecto. A pesar de que `ln` crea un enlace (al hacer `ls -l` aparece el enlace con la ruta indicada al crearlo), no podemos ver el contenido del fichero original:

```
polifemo:~$ ln -s dir1/mi_fichero dir2
polifemo:~$ cd dir2
polifemo:~/dir2$ ls -l
total 0
lrwxrwxrwx   1 llorente users     15 Jan 25 11:14 mi_fichero -> dir1/mi_fichero
polifemo:~/dir2$ cat mi_fichero
cat: mi_fichero: No such file or directory
```

La razón de que sea incorrecto, es que el enlace busca el archivo original como "`dir1/mi_fichero`" desde su ubicación actual (`~/dir2/`), es decir, que interpreta que el original es "`~/dir2/dir1/mi_fichero`". Donde en realidad se encuentra el original (utilizando una ruta relativa desde el directorio donde se ubica el enlace) es en "`../dir1/mi_fichero`". Luego esta es la ruta que deberíamos haber empleado al crear el enlace, a pesar de que, dado el directorio en el que nos encontrábamos en el momento de crear el enlace, esta última ruta no conduce al original (lo que ocurre es que sí es correcta desde la ubicación final del enlace). En consecuencia, la forma correcta de crear el enlace es la siguiente:

```
polifemo:~$ ln -s ../dir1/mi_fichero dir2
polifemo:~$ cd dir2
polifemo:~/dir2$ ls -l
total 0
lrwxrwxrwx   1 llorente users     18 Jan 25 11:24 mi_fichero -> ../dir1/mi_fichero
polifemo:~/dir2$ cat mi_fichero
Este es el contenido de mi_fichero
polifemo:~/dir2$
```

Recordemos que este problema sólo se plantea cuando trabajamos con enlaces simbólicos; para los enlaces duros la ruta de enlace que debemos especificar es la que sea correcta en el momento de la creación.

La ruta del enlace simbólico hacia su destino no se modifica al cambiar de ubicación el enlace. Siguiendo con el ejemplo, si cambiamos el enlace `mi_fichero` a otra ubicación (por ejemplo un directorio `dir3` dentro de `dir2`), el enlace dejará de ser correcto:

```
polifemo:~/dir2$ mkdir dir3
polifemo:~/dir2$ mv mi_fichero dir3/
polifemo:~/dir2$ cd dir3/
polifemo:~/dir2/dir3$ ls -l
total 0
lrwxrwxrwx   1 llorente users     18 Jan 25 11:24 mi_fichero -> ../dir1/mi_fichero
polifemo:~/dir2/dir3$ cat mi_fichero
cat: mi_fichero: No such file or directory
```

En cualquier caso, para evitar posibles problemas, es mejor utilizar rutas absolutas. Por ejemplo:

```
polifemo:~$ ln -s ~/dir1/mi_fichero dir2
polifemo:~$ cd dir2
polifemo:~/dir2$ ls -l
total 0
lrwxrwxrwx   1 llorente users     29 Jan 25 11:45 mi_fichero -> /home/llorente/dir1/mi_fichero
polifemo:~/dir2$ cat mi_fichero
Este es el contenido de mi_fichero
```

3.1.10. Comando chmod

El comando `chmod` (*change mode*) sirve para cambiar los permisos de un fichero o directorio. Hemos estudiado los permisos en la sección 1.6.5 *Permisos sobre los ficheros y directorios* (página 15). `chmod` admite como parámetro, primero los nuevos permisos, y luego los archivos (un fichero, una máscara o una lista) a los que hay que aplicar los permisos:

SINTAXIS

```
chmod [opciones] permisos ficheros...
```

Los permisos se pueden especificar de dos formas: simbólica o numéricamente. En ***notación simbólica***, los permisos se especifican con la siguiente sintaxis:

```
[{u|g|o|a}]{+|-|=}{r|w|x|s}
```

En concreto: cero o más letras del conjunto {*u, g, o, a*}, un símbolo de {+, –, =}, y una letra de {*r, w, x, s*}.

- La primera letra identifica los permisos a cambiar: propietario (*u*), grupo (*g*), otros usuarios (*o*), o bien todos los permisos (*a*). Si se omite esta parte, se supone *a*.
- El símbolo identifica cómo se cambiarán los permisos: añadir un permiso (+), restringir un permiso (–), o bien establecer un permiso único (=).
- La última letra identifica el permiso asignado: de lectura (*r*), escritura (*w*), de ejecución (*x*), o de establecimiento de propietario (*s*). Si se trata de este último (*s*), la primera letra sólo puede identificar al usuario (*u*) o al grupo (*g*).

Por ejemplo:

- Añadir permiso de lectura a todos los usuarios: `a+r`.
- Restringir permiso de escritura a los usuarios que no son del grupo: `o-w`.
- Establecer permisos `rx` (lectura y ejecución) al propietario: u=`rx`.
- Añadir permisos de escritura al propietario y el grupo: `ug+w`.
- Añadir permisos de modificación de propietario y grupo: `ug+s`.

Podemos incluir varias claves a la vez separándolas mediante comas ***sin dejar espacios entre cada clave***. Por ejemplo, para establecer permisos de lectura y ejecución al propietario y ejecución a todos utilizamos la siguiente clave: `a=x,u+r`.

En ***notación numérica***, los permisos se especifican mediante un número octal de hasta 4 cifras, donde cada cifra (número entre 0 y 7) representa los permisos de establecimiento de propietario, propietario, grupo y otros, respectivamente, de forma el valor 0-7 indica los permisos según la clave de la Tabla 18. En el caso del establecimiento de propietario, las claves son distintas, pero no lo vamos a estudiar.

Número	Valor binario	Permiso
0	000	---
1	001	--x
2	010	-w-
3	011	-wx
4	100	r--
5	101	r-x
6	110	rw-
7	111	rwx

Tabla 18. Claves numéricas para los permisos de acceso

Si se especifican menos de 4 cifras, se supone que las primeras son 0, es decir que 174 es equivalente a 0174. Los permisos de establecimiento de propietario no suelen utilizarse, por lo que normalmente se emplean sólo 3 dígitos, para los permisos de acceso a la información, que son los más comúnmente utilizados. Veamos algunos ejemplos:

- Otorgar todos los permisos: `777`.
- Permiso de lectura a todos, y de lectura, ejecución y escritura al propietario: `744`.
- Permiso de lectura y ejecución a todos: `555`.
- Permiso de lectura y escritura al usuario, de ejecución y lectura al grupo y de ejecución al resto: `651`.

Los modificadores que admite el comando `chmod` se resumen en la Tabla 19.

Modificador	Resultado
-v	Muestra el cambio que realiza en los permisos cada fichero procesado.
-c	Muestra el cambio que realiza en los permisos cada fichero procesado sólo si realiza algún cambio.
-f	No muestra errores cuando no sea posible cambiar los permisos.
-R	Recursivo: procesa también los ficheros que estén dentro de subdirectorios.

Tabla 19. Modificadores para el comando `chmod`

3.1.11. Comando chown

El comando `chwon` (*change owner*) sirve para cambiar el propietario de un fichero. Originalmente el propietario de un fichero es su creador. `chwon` también permite cambiar el grupo al que pertenece el fichero (el comando `chgrp` sirve también para cambiar el grupo, pero no lo estudiaremos).

SINTAXIS

```
chown [opciones] [propietario][{.|:}grupo] ficheros...
```

El nuevo propietario y grupo se especifica mediante su nombre. Podemos modificar el propietario, el grupo o las dos cosas a la vez. El nombre del propietario y el grupo se separan mediante un punto (`.`) o dos puntos (`:`), sin

espacios entre ellos. Si sólo quedemos cambiar el grupo, debemos poner el punto o los dos puntos antes del nombre del grupo.

La Tabla 20 contiene las principales opciones del comando. Veamos ahora algunos ejemplos de uso de `chown`.

```
chown llorente /tmp/*.c /tmp/*.h
chown llorente:users /tmp/*.c
chown :users /tmp/inicio.c
chown root.root /tmp/inicio.c
```

Modificador	Resultado
`-R`	Recursivo: cambia el propietario/grupo a los ficheros de los subdirectorios.
`-v`	Muestra el nombre de los ficheros que procesa.
`-c`	Muestra el nombre de los ficheros que procesa, sólo si hace algún cambio.
`-f`	No muestra mensajes de error cuando no pueda cambiar el propietario/grupo.

Tabla 20. Modificadores para el comando chown

3.2. Comandos de procesamiento de ficheros

En UNIX existe un gran número de comandos diseñados para el procesamiento de ficheros de texto, de manera que con el uso combinado de ellos se puede conseguir cualquier resultado. Los comandos que vamos a estudiar en este apartado utilizan normalmente ficheros de texto, aunque eso no quiere decir que no se puedan utilizar con ficheros binarios.

3.2.1. Comando cat

El comando `cat` (*concatenate*) sirve para encadenar varios ficheros. El resultado se muestra por pantalla (por la salida estándar). Admite como parámetro la lista de ficheros que hay que encadenar. Estos se enviarán a la salida estándar en el mismo orden en que aparecen en la lista. Si sólo se especifica un fichero, el resultado será que se mostrará por pantalla ese fichero (este es un de los usos más habituales de `cat`). Si no se especifica ningún fichero, `cat` tomará el contenido de la entrada estándar (teclado).

SINTAXIS

```
cat [opciones] [ficheros...]
```

`cat` se utiliza a menudo para componer pequeños ficheros de texto. El comando "`cat >nuevo_texto`" crea el fichero `nuevo_texto` con el contenido de lo que a continuación escribamos. Lo que hace es tomar la entrada estándar (teclado en este caso) y lo pasa a la salida estándar, que en este caso está redireccionada hacia el fichero `nuevo_texto`. Cuando utilizamos el teclado para componer la entrada a un comando, para indicar el final del texto que escribimos tenemos que pulsar ^*D* (la tecla *Ctrl* más la tecla *D*). Si no estamos al principio de la línea (hemos escrito algo antes de pulsar ^D), tendremos que pulsar ^D dos veces. Por ejemplo (^D no aparece en pantalla):

```
polifemo:~$ cat >nuevo_texto
Este es el texto que va a contener                (estas dos lineas las escribimos nosotros)
```

```
el fichero.
^D                                                    (pulsamos ^D)
polifemo:~$ cat nuevo_texto
Este es el texto que va a contener
el fichero.
```

El comando `cat` admite los modificadores de la Tabla 21.

Modificador	Resultado
`-n`	Hace que se numeren las línea mostradas (empezando en 1).
`-b`	Hace que se numeren las línea mostradas, pero no se numeran las líneas en blanco (vacías).
`-s`	Elimina las múltiples líneas en blanco contiguas por una única línea vacía.
`-A`	Muestra los caracteres no imprimibles con notación "^", y a los caracteres cuyo código ASCII sea mayor que 127 les antepone "M-"

Tabla 21. Modificadores para el comando `cat`

3.2.2. Comando wc

El comando `wc` (*word count*) sirve para contar las líneas, palabras y caracteres que contiene un fichero. Por defecto cuanta estas tres cosas. Para cada fichero procesado por el comando, escribe (en la salida estándar) 4 columnas que indican el número de líneas, palabras y caracteres (en este orden) y el nombre del fichero procesado. Además, si se procesan varios ficheros, al final se muestra la cuenta total.

SINTAXIS

```
wc [opciones] [ficheros...]
```

Para `wc` una palabra es el conjunto de caracteres que hay entre dos blancos (espacio, tabulación, retorno de carro, etc). Con las opciones `-c`, `-w` y `-l` podemos hacer que en vez de las tres cuentas sólo se muestre una o dos, tal y como se indica en la Tabla 22. En cualquier caso el nombre de cada fichero procesado aparecerá a la derecha de las cuentas, y los números aparecen en el orden líneas, palabras, caracteres (independientemente del orden de los modificadores utilizados). El siguiente ejemplo muestra el número de líneas y palabra de los ficheros `*.c`.

```
polifemo:~$ wc -wl *.c
      3      11 f1.c
    533    1118 data.c
     24     121 x_2.c
      0       0 empty.c
    560    1250 total
```

Si no se indica ningún fichero, contará el contenido de la entrada estándar.

Modificador	Resultado
-c	Sólo muestra el número de caracteres (número de bytes).
-w	Sólo muestra el número de palabras.
-l	Sólo muestra el número de líneas.

Tabla 22. Modificadores para el comando wc

3.2.3. Comando sort

El comando `sort` sirve para ordenar alfabéticamente las líneas de un fichero, para mezclar las líneas de varios ficheros y para comprobar si un fichero está ordenado. La selección entre estas tres operaciones se hace mediante los modificadores `-c` y `-m`.

SINTAXIS

```
sort [opciones] [ficheros...]
```

- Si no se utiliza `-c` ni `-m`, sirve para ordenar: toma los ficheros especificados como parámetro, los fusiona, ordena el resultado y lo muestra por la salida estándar.
- Con `-c`, `sort` se utiliza para comprobar si un fichero está o no ordenado. Si lo está, no hace nada; en caso contrario termina con error[9].
- Con `-m`, `sort` se utiliza para mezclar los ficheros especificados. No es más que otras forma de ordenar, pero requiere que los ficheros especificados estén individualmente ordenados. Como ventaja, este método es más rápido que la ordenación normal.

El criterio de ordenación por defecto es el orden alfabético basado en el orden ASCII, aunque podemos utilizar otros criterios. El modificador `-f` hace que no se distinga entre mayúsculas y minúsculas, `-n` indica que la ordenación es numérica (y no alfabética), `-b` ignora los espacios antes de la primera letra, etc. La Tabla 23 contiene los principales modificadores.

Habitualmente los ficheros que ordenemos tendrán campos, es decir, estarán formado una tabla organizada en filas (líneas) y columnas (separadas mediante algún carácter, por ejemplo, un tabulador). Entonces nos puede interesar ordenar esa tabla (fichero) por algún campo que no tiene necesariamente que ser el primero de cada línea. Para ello debemos indicarle a `sort` dos cosas:

- El carácter que separa los campos. Para esto utilizamos la opción `-t` seguida del carácter de separación (hay que dejar un espacio después de `-t` y antes del carácter).
- El número de campo por el que ordenamos (la clave de ordenación). Esto lo indicamos con la opción `+###` y `-###`, donde `###` representa el número de campo (comenzando en 0). `+###` indica el primer campo de ordenación y `-###` el posterior al último (si se omite éste, se supone hasta el final de la línea). Es decir que la clave son los campos incluidos entre `+###` y `-###`, sin incluir éste. El número de campo se especifica como uno o dos números separados por un punto (`.`); lo que queda a la izquierda es el número de campo; lo que queda a la derecha (la parte opcional) es el número de carácter dentro del campo (si se omite el número de carácter, se debe omitir el punto). Detrás del número (sin separación) se puede utilizar un carácter para especificar opciones de ordenación (numérica, ignorar mayúsculas/minúsculas, etc),

[9] Termina con estado 1.

La opción `-o` seguida (con una separación) de un nombre de fichero, hace que el resultado se envíe a ese fichero en lugar de a la salida estándar. Esto permite que se pueda utilizar ese mismo fichero como fichero de entrada, haciéndose una copia temporal para utilizarla como entrada.

Por último, la opción -r ordena de forma inversa.

Modificador	Resultado
`-b`	Ignora los blancos (espacio, tabuladores, etc.) al principio de las líneas.
`-d`	Ignora los caracteres que no sean letras o números.
`-f`	Ignora la diferencia entre mayúsculas y minúsculas.
`-M`	Supone que la clave de ordenación representa los meses del año en formato "`JAN`", "`FEB`",... "`DEC`". Además ignora la diferencia entre mayúsculas y minúsculas.
`-n`	Supone que la clave de ordenación representa un número (entero o decimal).
`-r`	Invierte el orden.
`-o`	Permite especificar a continuación un fichero de salida en lugar de la salida estándar.
`-t`	Permite especificar a continuación el carácter de separación entre campos.
`+###`	### es el primer campo de ordenación (numerado desde 0).
`-###`	### es el campo posterior al último campo de ordenación (numerado desde 0).

Tabla 23. Modificadores para el comando `sort`

Ejemplos:

- Listar los ficheros del directorio y ordenarlos según su propietario (con `ls -l` obtenemos un listado en el que la tercera columna es el propietario):

```
polifemo:~$ ls -l|sort +2 -3
total 11
-rw-r--r--   1 llorente users          99 Oct 19 14:35 nuevo_texto
-rw-r--r--   1 llorente users        7869 Jul 10  1997 termcap
-rw-r-x--x   2 llorente users          41 Oct 19 14:47 fichero1*
-rw-r-x--x   2 llorente users          41 Oct 19 14:47 fichero2*
-rwsrwsrwx   1 root     root            0 Oct 19 11:41 p*
```

- Listar los ficheros del directorio y ordenarlos según el mes de creación y después según su tamaño:

```
polifemo:~$ ls -l | sort +5M -6 +4n -5
total 11
-rw-r--r--   1 llorente users        7869 Jul 10  1997 termcap
-rwsrwsrwx   1 root     root            0 Oct 19 11:41 p*
-rw-r-x--x   2 llorente users          41 Oct 19 14:47 fichero1*
-rw-r-x--x   2 llorente users          41 Oct 19 14:47 fichero2*
-rw-r--r--   1 llorente users          99 Oct 19 14:35 nuevo_texto
```

- Crear un fichero con plantas y otro con animales y ordenarlos juntos:

```
polifemo:~$ cat >animales
```

```
oso                                                    (esto lo escribimos nosotros)
pez
elefante
rana
^D                                                     (pulsamos ^D)
polifemo:~$ cat >plantas
pino                                                   (esto lo escribimos nosotros)
encina
rosa
alcornoque
^D                                                     (pulsamos ^D)
polifemo:~$ sort plantas animales
alcornoque
elefante
encina
oso
pez
pino
rana
rosa
```

3.2.4. Comando more

El comando `more` sirve para mostrar un fichero haciendo una pausa en cada página (pantalla). Si especificamos varios ficheros, se mostrarán en el mismo orden, haciendo pausa e imprimiendo una línea de título antes de comenzar cada uno.

SINTAXIS

```
more [opciones] [ficheros...]
```

Para avanzar por el texto paginado, utilizamos las siguientes teclas:

- *Barra espaciadora*: avanzar una página.
- *Enter*: Avanzar una línea.
- *q*: terminar.

Cada vez que avanzamos en el fichero mostrado, las nuevas líneas aparecen en la parte inferior de la pantalla, mientras las anteriores se desplazan hacia arriba, a menos que empleemos la opción `-p`, que hace que las páginas se reemplacen unas a otras sin que se produzca desplazamiento (esto es más rápido). Otras opciones que permite este comando se resumen en la Tabla 24.

Modificador	Resultado
`-###`	Indica el número `###` de líneas por página.
`+###`	Indica el número `###` de línea por el que hay que comenzar.
`+/`	Seguido de una cadena, hace que se comience a mostrar el fichero por la primera aparición de la cadena.
`-p`	Hace que las páginas se muestren sin que se produzca desplazamiento (*scroll*)

Tabla 24. Modificadores para el comando `more`

3.2.5. Comando less

El comando `less` sirve para visualizar el contenido de un fichero poco a poco. Es semejante a `more`, pero con más funcionalidad. La principal ventaja sobre `more` es que se puede navegar por el texto tanto hacia delante como hacia atrás.

SINTAXIS

```
less [opciones] [ficheros...]
```

Para avanzar y retroceder por el texto paginado podemos utilizar, además de las teclas de `more`, los cursores. Además existen multitud de teclas y opciones para todo tipo opciones, que podemos consultar con `man less`. Destaca la posibilidad de numerar las líneas, lo que hacemos con `-N`.

El comando `man` utiliza `less` para mostrar su información.

Modificador	Resultado
`-N`	Muestra el número de línea junto a la línea.

Tabla 25. Modificadores para el comando `less`

3.2.6. Comando tail

El comando `tail` sirve para mostrar por la salida estándar las últimas líneas de un fichero. Por defecto muestra las 10 últimas líneas, pero podemos seleccionar otro número.

SINTAXIS

```
tail [opciones] fichero...
```

La opción más habitual es `-n`, que permite especificar a continuación el número de líneas que hay que mostrar. Por ejemplo, para ver las 3 últimas líneas de un fichero:

```
polifemo:~$ tail -n 3 mi_fichero
```

Si especificamos varios ficheros a la vez, se mostrarán en el mismo orden, con un título antes de cada uno, a menos que empleemos la opción `-q`. Ejemplo:

```
polifemo:~$ tail -n 2 animales plantas
==> animales <==
elefante
rana

==> plantas <==
rosa
alcornoque
polifemo:~$ tail -q -n 2 animales plantas
elefante
rana
rosa
```

```
alcornoque
```

La Tabla 26 recoge los principales modificadores que se pueden utilizar con `tail`.

Modificador	Resultado
`-c`	Permite especificar a continuación el número de caracteres del final que hay que mostrar (en vez de líneas).
`-q`	No muestra títulos antes de cada fichero cuando hay varios ficheros.
`-n`	Permite especificar a continuación el número de líneas que hay que mostrar (por defecto 10).

Tabla 26. Modificadores para el comando `tail`

3.2.7. Comando head

El comando `head` sirve para mostrar por la salida estándar las primeras líneas de un fichero. Por defecto muestra las 10 primeras líneas, pero podemos seleccionar otro número.

SINTAXIS

```
head [opciones] fichero...
```

La opción más habitual es `-n`, que permite especificar a continuación el número de líneas que hay que mostrar. Por ejemplo, para ver las 3 primeras líneas de un fichero:

```
polifemo:~$ head -n 3 mi_fichero
```

Si especificamos varios ficheros a la vez, se mostrarán en el mismo orden, con un título antes de cada uno, a menos que empleemos la opción `-q`. Ejemplo:

```
polifemo:~$ head -n 2 animales plantas
==> animales <==
oso
pez

==> plantas <==
pino
encina
polifemo:~$ head -qn 2 animales plantas
oso
pez
pino
encina
```

La Tabla 27 recoge los principales modificadores que se pueden aplicar a `head`.

Modificador	Resultado
`-c`	Permite especificar a continuación el número de caracteres del principio que hay que mostrar (en vez de líneas).
`-q`	No muestra títulos antes de cada fichero cuando hay varios ficheros.
`-n`	Permite especificar a continuación el número de líneas que hay que mostrar (por defecto 10).

Tabla 27. Modificadores para el comando `head`

3.2.8. Comando grep

El comando `grep` (*get regular expression and print*) sirve para mostrar las líneas de un fichero que contienen un patrón, es decir que contienen un texto. Si especificamos varios ficheros, se buscará en todos ellos; si no especificamos ninguno se buscará en la entrada estándar.

SINTAXIS

```
grep [opciones] patrón [ficheros...]
```

El funcionamiento normal del comando es mostrar en la salida estándar las líneas que contienen el patrón (este funcionamiento se puede alterar mediante las opciones de la Tabla 30). Además, si se encuentra el patrón alguna vez, el comando termina con éxito, y si no, termina con error.

El patrón puede ser una cadena literal o un patrón con comodines. Los comodines básicos que podemos usar son los recogidos en la Tabla 28.

Carácter	Significado
`.`	Cualquier carácter
`[]`	Cualquier carácter del conjunto o rango encerrado (p.e. `[0-9A-Z]`)
`^`	Comienzo de línea
`$`	Final de línea
<	Comienzo de palabra (\<)
>	Final de palabra (\>)

Tabla 28. Comodines básicos que se pueden utilizar en `grep`

Cada comodín sólo sustituye a un carácter, aunque se pueden combinar con otros símbolos para ampliar las posibilidades, como muestra la Tabla 29. Si queremos buscar un carácter que funciona como comodín (por ejemplo, queremos buscar un punto), podemos anteponer una barra inversa delante del carácter. De esta forma eliminamos el valor especial del comodín; pero la barra inversa es un carácter especial del shell, por lo que ésta debe aparecer entre comillas, o bien estar precedida de otra barra inversa.

Carácter	Significado
*x**	*x* se puede repetir de 0 a *n* veces (*x* es un literal o un comodín).
x?	*x* se puede repetir 0 o 1 veces (*x* es un literal o un comodín).
x+	*x* se puede repetir de 1 a *n* veces (*x* es un literal o un comodín).
x{*n*}	*x* se repite *n* veces (*x* es un literal o un comodín y *n* un número).
x{*n*,}	*x* se repite *n* o más veces (*x* es un literal o un comodín y *n* un número).
x{,*n*}	*x* se puede repetir de 0 a *n* veces (*x* es un literal o un comodín y *n* un número).
x{*n*,*m*}	*x* se puede repetir de *n* a *m* veces (*x* es un literal o un comodín y *n* y *m* números).

Tabla 29. Comodines de repetición que se pueden utilizar en `grep`

También podemos agrupar subexpresiones utilizando los paréntesis y utilizar la operación OR lógica con el símbolo de la tubería (`|`).

La comparación de patrones hace distinción entre mayúsculas y minúsculas a menos que empleemos la opción `-i`. Por defecto `grep` muestra las línea que contienen el patrón, pero podemos obtener otros comportamientos utilizando las opciones adecuadas:

- Para las líneas que no contienen el patrón utilizamos `-v`.
- Para ver el número de líneas que contienen el patrón utilizamos `-c`.
- Para ver los archivos que contienen el patrón en alguna línea utilizamos `-l`.

Si la expresión que queremos buscar comienza por un guión (-), podría confundirse con una opción del parámetro. Entonces podemos utilizar la opción `-e` justo antes de la expresión para indicar que lo que viene a continuación no es una opción, sino la expresión. Por ejemplo, para localizar el patrón "`-v-`" en los ficheros debemos escribir:

```
grep -e -v- *
```

Otros ejemplos:

- Buscar nombres de usuario que comiencen por "`a`":

```
grep "^a" /etc/passwd
```

- Mostrar el fichero `informe` sin las líneas que contengan las palabras "TOP SECRET":

```
grep -v "TOP SECRET" informe
```

- Mostrar los ficheros en C que contienen líneas vacías:

```
grep -l "^$" *.c
```

La Tabla 30 resume las principales opciones del comando `grep`.

Modificador	Resultado
`-i`	No hace distinción entre mayúsculas y minúsculas.
`-v`	Muestra las líneas que no contienen el patrón.
`-c`	Muestra el número de líneas que contienen el patrón.
`-l`	Muestra los archivos que contienen el patrón en alguna línea, en lugar de las líneas.
`-e`	Permite especificar que lo que viene a continuación es una expresión (y no una opción).
`-n`	Muestra el número de línea en que aparece la expresión en cada fichero.
`-s`	No muestra mensajes de error cuando algún fichero no se puede leer.
`-q`	No muestra ninguna salida; sólo termina con éxito o con error.

Tabla 30. Modificadores para el comando `grep`

Existen variantes de `grep`, como por ejemplo `fgrep` y `egrep`, que tienen opciones adicionales.

3.2.9. Comando find

El comando `find` sirve para buscar ficheros en el árbol de directorios. El criterio de búsqueda es muy flexible, permitiendo buscar por nombre, tamaño, fecha, etc. También podemos especificar la rama o ramas del árbol de directorios por las que buscar, y permite programar la acción a realizar con los ficheros localizados.

SINTAXIS

```
find rutas... [criterios] [acción]
```

- `rutas` es el conjunto de directorios donde comenzar a buscar. `find` buscará en las rutas indicadas y en todos sus subdirectorios.
- criterios es el criterio de búsqueda, y consiste en una opción que indica la forma de buscar (por nombre, por fecha, etc.) y el patrón de comparación (la máscara de nombre, la fecha, etc.). La Tabla 31 recoge los posible criterios. Podemos combinar varios criterios, uniéndolos con los operadores AND y OR. Para hacerlo con AND, basta con escribir más de un criterio de búsqueda; para hacerlo con OR, anteponemos `-o` al criterio. Además, podemos anteponer una admiración (`!`) a un criterio para que funcione al contrario (busca los ficheros que no cumplan la condición). Además, podemos especificar el orden en que se evalúan las condiciones utilizando paréntesis. Puesto que los paréntesis son un carácter especial para el shell, debemos anteponer a cada paréntesis una barra invertida (`\`), encerrarlos entre comillas, etc.
- `acción` es la tarea que debe ejecutar `find` cada vez que localice un archivo que verifique las condiciones de búsqueda. La Tabla 32 contiene las principales acciones posibles.

Si se omiten los criterios, todos los ficheros de los directorios indicados se consideran encontrados. En los criterios que comparan con números podemos anteponer al número "`-`" para expresar que la comparación es cierta si el numero es menor que el indicado, o "`+`" para expresar que la comparación es cierta si el numero es mayor que el indicado. (Por defecto la comparación es verdadera si el número es igual al indicado.)

Selector	Significado (criterio de búsqueda)
`-name` *`nombre`*	Busca ficheros cuyo nombre coincida con *`nombre`*.
`-perm` *`modo`*	Busca ficheros cuyos permisos coincidan con *`modo`*. Los permisos se especifican con el formato numérico (ver apartado 1.6.5 *Permisos sobre los ficheros y directorios* en la página 15). Si *`modo`* va precedido de un signo menos (-) busca ficheros que tengan al menos los permisos indicados, y si va precedido de un signo más (+), busca ficheros que tengan alguno de los permisos indicados.
`-type` *`x`*	Busca ficheros cuyo tipos sea el indicado con *`x`*, donde *`x`* puede ser: `f` para buscar ficheros normales, `d` para buscar directorios, `l` para buscar enlaces simbólicos, `c` para buscar dispositivos de caracteres, etc.
`-links ###`	Busca ficheros que tengan el número *`###`* de enlaces duros. Si usamos *-###* significa menos de ### enlaces. Si usamos *+###* significa más de ### enlaces.
`-size ###`	Busca ficheros que tengan un tamaño de *`###`* bloques. Para asegurarnos de que el tamaño de bloque es de un kilobyte podemos poner ###`k`. Además, si usamos *-###* significa menos de ### bloques y si usamos *+###* significa más de ### bloques.
`-user` *`nombre`*	Busca ficheros que pertenezcan al usuario *`nombre`*.
`-atime ###`	Busca ficheros que hayan sido accedidos hace *`###`* días. Si usamos *-###* significa menos de ### días. Si usamos *+###* significa más de ### días.
`-mtime ###`	Busca ficheros que hayan sido modificados hace *`###`* días. Si usamos *-###* significa menos de ### días. Si usamos *+###* significa más de ### días.
`-newer` *`nombre`*	Busca ficheros más recientes que el fichero *`nombre`*.

Tabla 31. Principales criterios de búsqueda para el comando `find`

Selector	Significado (criterio de búsqueda)
`-print`	Imprimir por la salida estándar el nombre del archivo localizado.
`-exec` *`comando`*	Ejecutar el *`comando`* indicado. Para indicar en el comando el nombre del fichero localizado, utilizamos {}, que se sustituirá por dicho nombre. Además, para indicar el fin del comando, debemos poner un punto y coma.(`;`) precedido de una barra invertida porque `;` es un carácter especial para el shell). Por ejemplo `-exec cat {} \;` hace que se muestre el contenido del fichero localizado.

Tabla 32. Principales acciones posibles para `find`

Veamos algunos ejemplos de uso de `find`:

- Buscar ficheros que comiencen por "`a`" en el directorio actual y en `/tmp` (y subdirectorios de ambos) y mostrar dónde esta:

```
find . /tmp -name "a*" -print
```

- Localizar el fichero `passwd` en el disco y mostrar dónde esta:

```
find / -name passwd -print
```

- Mostrar el contenido de los ficheros modificados esta semana en los directorios del usuario `llorente` que ocupen menos de 2 bloques de disco:

```
find /home/llorente -mtime -7 -size -2 -exec cat {} \;
```

- Borrar los ficheros que tengan el nombre `*.c` y tengan más de 30 días, y también aquellos que sean del usuario *fgarcia*:

```
find / \( -mtime +30 -name "*.c" \) -o -user fgarcia -exec rm -f {} \;
```

NOTA IMPORTANTE: Hay que tener cuidado con el uso de máscaras en los criterios de búsqueda de nombres de fichero, ya que las máscaras se podrían sustituir por los nombres de archivos del directorio actual que encajen en las máscaras, de manera que se buscarían esos nombres de ficheros del directorio actual, en lugar de los nombres de fichero que indiquen las máscaras. Para evitar el problema, encerramos entre comillas las máscaras (como en los ejemplos anteriores), de manera que la máscara llega tal cual al comando, y no es interpretada por el shell antes de ser entregada al comando.

Este problema puede observarse también en otros comandos, además de en `find`.

3.2.10. Comando uniq

El comando `uniq` sirve para eliminar de un fichero las líneas repetidas. El fichero debe estar previamente ordenado. Podemos especificar el fichero de entrada y el fichero de salida (no puede ser el mismo). Si se omiten, se utilizarán la entrada y salida estándar.

SINTAXIS

```
uniq [opciones] [entrada [salida]]
```

Si sólo se especifica uno de los ficheros de entrada o salida, éste será el de entrada, tal y como se aprecia en la sintaxis. Las opciones que admite `uniq` sirven para modificar su comportamiento, como se describe en la Tabla 33.

Modificador	Resultado
`-u`	Hace que sólo se envíen a la salida las líneas no repetidas.
`-d`	Hace que sólo se envíen a la salida las líneas repetidas (sólo envía una copia).
`-c`	Muestra a la izquierda de cada línea el número de veces que se repite.

Tabla 33. Modificadores para el comando `uniq`

3.2.11. Comando comm

El comando `comm` sirve para separar las líneas comunes y no comunes de dos ficheros (`fichero1` y `fichero2`) previamente ordenados, y genera en la salida estándar un informe con 3 columnas: líneas que sólo están en `fichero1`, líneas que sólo están en `fichero2` y líneas que están en los dos ficheros.

SINTAXIS

```
comm [opciones] fichero1 fichero2
```

Las opciones -1, -2 y -3 hacen que la columna correspondiente no se muestre, como se indica en la Tabla 34.

Modificador	Resultado
-1	Hace que no se muestre la columna primera (líneas que sólo están en el primer fichero).
-2	Hace que no se muestre la columna segunda (líneas que sólo están en el segundo fichero).
-3	Hace que no se muestre la columna tercera (líneas que están en ambos ficheros).

Tabla 34. Modificadores para el comando comm

3.2.12. Comando cmp

El comando `cmp` (*compare*) sirve para comparar dos ficheros, no necesariamente ordenados. Como salida, imprime un mensaje que contiene el número de byte (carácter) dentro del fichero y el número de línea donde aparece la primera diferencia entre los ficheros comparados (las líneas y caracteres se numeran comenzando en 1). Si los ficheros son iguales, no imprime nada.

SINTAXIS

```
cmp [opciones] fichero1 fichero2
```

`cmp` se detiene ante la primera diferencia. Con la opción `-l` podemos visualizar todas las diferencias en tres columnas que contienen: posición de la primera diferencia (byte), código ASCII en octal del carácter que aparece en el primer fichero en esa posición, y código ASCII en octal del carácter que aparece en el segundo fichero en esa posición. La Tabla 35 contiene otros modificadores que podemos utilizar con `cmp`.

Modificador	Resultado
-l	Imprime la lista completa de diferencias indicando en 3 columnas la posición y los caracteres que difieren.
-s	No imprime nada cuando los ficheros son diferentes, sino que termina con error.

Tabla 35. Modificadores para el comando cmp

3.2.13. Comando lpr

El comando `lpr` (*line printer*) sirve para enviar documentos a la impresora. Admite una lista de ficheros para imprimir; si no se especifica ninguno, se imprime la entrada estándar.

SINTAXIS

```
lpr [opciones] [ficheros...]
```

Con la opción `-#` seguida (sin separación) de un numero indicamos el número de copias. Otras opciones se resumen en la Tabla 36.

Debemos recordar que ***la impresora que se utiliza es la que está conectada a la máquina Linux***, no la que tengamos en la consola que estemos utilizando (si nos hemos conectado de forma remota).

Modificador	Resultado
-#*nnn*	*nnn* es el número de copias.
-P*xxxxxx*	*xxxxxx* es el nombre de la impresora que hay que utilizar.
-r	Hace que los ficheros indicados se eliminen una vez finalizada la impresión.
-m	Hace que el sistema nos envíe un mensaje de correo electrónico cuando el trabajo haya terminado.

Tabla 36. Modificadores para el comando lpr

3.3. Comandos de manejo de unidades y dispositivos

Vamos a estudiar en este apartado los comandos que nos permiten acceder a discos y disquetes. Ya sabemos que el UNIX todas las unidades de disco son accedidas como si formaran una única unidad, organizada en un único árbol de directorios. Como es posible que se añadan nuevas unidades una vez arrancado el sistema (por ejemplo, si introducimos un disquete o un CD-ROM), es necesario indicar a UNIX en qué parte del árbol de directorios queremos situar el árbol contenido en la unidad de disco incorporada (montar la unidad), y también hay que advertir al sistema de que vamos a retirar el contenido de la unidad (desmontar la unidad). Si no hacemos correctamente estas operaciones (por ejemplo, retiramos un disquete sin desmotarlo previamente), podemos dañar irreparablemente el árbol de directorios y perder la información. Por esta razón, sólo se permite al administrador trabajar con las unidades de disco.

Antes de comenzar a estudiar los comandos debemos conocer cómo se denominan las distintas unidades de disco en UNIX. Esta denominación puede cambiar de unas versiones de UNIX a otras. La Tabla 37 recoge la denominación en los sistemas Linux convencionales.

NOTA:

Los nombres de las unidades son los dispositivos, unos ficheros especiales que se ubican en el directorio /dev. Los dispositivos son de dos tipos (de bloques, como un disco; y de caracteres, como un terminal). Las entradas de /dev asocian a cada nombre de dispositivo dos números que sirven para el sistema localice el hardware correspondiente. Cuando nos refiramos a un dispositivo, nosotros especificaremos la entrada adecuada del directorio /dev.

Linux está bastante ligado al uso de MS-DOS/Windows, por lo que existen varios comandos relacionados con el uso de disquetes con formato MS-DOS/Windows.

Dispositivo	Resultado
hda	Primer disco duro (IDE).
hda1	Primera partición del primer disco duro (IDE).
hda2	Segunda partición del primer disco duro (IDE).
hda3	Tercera partición del primer disco duro (IDE).
...	
hdb	Segundo disco duro (IDE).
hdb1	Primera partición del segundo disco duro (IDE).
...	
hdc	Tercer disco duro (IDE).
hdd	Cuarto disco duro (IDE).
...	
fd0	Primera unidad de disco flexible.

Dispositivo	Resultado
fd1	Segunda unidad de disco flexible.
...	

Tabla 37. Algunos dispositivos de unidades de disco en Linux

3.3.1. Comando mount

El comando `mount` nos permite montar una unidad al sistema de ficheros. Debemos indicar qué unidad montamos y en qué directorio del árbol queremos situar el sub-árbol contenido en dicha unidad.

SINTAXIS

```
mount [opciones] [dispositivo directorio]
```

- `dispositivo` es alguna de las entradas de `/dev` que identifican algún dispositivo de disco, por ejemplo los recogidos en la Tabla 37.
- `disrectorio` es algún directorio que no esté en uso. No tiene necesariamente que estar vacío, pero mientras se haga uso del dispositivo (mientras siga montado), no podremos acceder al contenido anterior de ese directorio; una vez desmontado el dispositivo, el directorio quedará como estaba.

Si no especificamos dispositivo ni directorio, `mount` imprimirá un listado de los dispositivos actualmente montados.

Linux soporta varios sistemas de ficheros (formatos de discos de distintos sistemas operativos), por lo que es posible que también tengamos que indicar el tipo de formato que tiene el contenido de la unidad montada. Esto lo hacemos mediante la opción `-t` seguida (con una separación) de una palabra clave que describe el tipo de sistema de ficheros: `minix`, `ext`, `ext2`, `xiafs`, `hpfs`, `msdos`, `umsdos`, `vfat`, `proc`, `nfs`, `iso9660`, `smbfs`, `ncpfs`, `affs`, `ufs`, `romfs`, `sysv`, `xenix`, `coherent`. Normalmente Linux detecta automáticamente el tipo de sistema de ficheros, pero a veces es necesario indicarlo explícitamente.

Normalmente los dispositivos sólo pueden ser montados por el administrador, aunque todos los usuarios pueden acceder al contenido de los dispositivos (siempre que tengan los permisos adecuados sobre los directorios de montaje). En este caso, una opción interesante es la de montar los dispositivos como sólo lectura con la opción `-r`, de forma que su contenido no se puede modificar.

El siguiente ejemplo monta la primera partición del primer disco duro en el directorio `/mnt`:

```
mount /dev/hda1 /mnt
```

En el siguiente caso, montamos un disquete del sistema operativo Minix en `/mnt`:

```
mount -t minix /dev/fd0 /mnt
```

La Tabla 38 contiene las opciones estudiadas para el comando `mount`.

Modificador	Resultado
-r	Monta el dispositivo como sólo lectura.
-t	Permite especificar a continuación el tipo de sistema de ficheros contenido en la unidad.

Tabla 38. Modificadores para el comando mount

3.3.2. Comando umount

El comando `umount` sirve para desmontar un dispositivo previamente montado con `mount`. Debemos especificar el dispositivo de que se trata, o bien el directorio dónde se montó.

SINTAXIS

```
umount [opciones] {dispositivo|directorio}
```

Un dispositivo no puede se desmontado cuando está ***ocupado*** (*busy*), es decir, cuando contiene ficheros abiertos, o cuando existan procesos cuyo directorio por defecto pertenezca al dispositivo.

No estudiaremos las opciones de este comando. Los siguientes ejemplos desmontan los ejemplos de montaje vistos con el comando `mount`:

```
umount /dev/hda1
umount /dev/fd0
```

3.3.3. Comando df

El comando `df` (*disk free*) sirve para visualizar el estado de los discos que se encuentran montados en el sistema: tipo, capacidad

SINTAXIS

```
df [opciones]
```

Por ejemplo, el siguiente listado indica que el sistema de ficheros de `polifemo` está formado por dos discos (`sda2`, montado como raíz, y `sda3`, montado como `/home`). El primer disco tiene una capacidad de 1.982.006 bloques de 1024 bytes[10] (es decir, 1,89 GB), del que se han ocupado 684.010 bloques (el 36%) y quedan libres 1.195.545 bloques. El segundo disco tiene una capacidad de 2.191.875 bloques (2,09 GB), del que se han ocupado 1.218.246 (el 59%) y quedan libres 860.324.

```
polifemo:~$ df
Filesystem         1024-blocks  Used Available Capacity Mounted on
/dev/sda2            1982006  684010  1195545     36%   /
/dev/sda3            2191875 1218246   860324     59%   /home
```

[10] A veces se utilizan bloques de 512 bytes a menos que se indique lo contrario mediante alguna opción

Todo esto quiere decir que disponemos de 840 MB libres para escribir archivos dentro del directorio `/home` (y sus subdirectorios) más 1,14 GB para escribir en el resto de directorios del disco.

3.3.4. Comando mdir

El comando `mdir` (*MS-DOS directory*) sirve para visualizar el contenido de un disquete (o en general de cualquier disco) de MS-DOS/Windows sin necesidad de montar ni desmontar la unidad de disquetes (la unidad no debe estar montada para que funcione). El listado se muestra con el mismo aspecto a como lo hace el comando `dir` de MS-DOS/Windows.

SINTAXIS

```
mdir [opciones] directorio
```

El directorio se especifica mediante una mezcla de formato entre Linux y MS-DOS/Windows:

- El nombre de la unidad se especifica mediante la letra de MS-DOS/Windows, seguida de dos puntos (`:`).
- La barra de separación de directorios no es inversa (`\`), sino que es la que se emplea en UNIX (`/`).

El siguiente ejemplo visualiza el contenido del directorio copia que se encuentra en el disquete MS-DOS introducido en la primera unidad de disquetes:

```
mdir a:/copia/*.*
```

Los modificadores que acepta el comando se resumen en la Tabla 39.

Modificador	Resultado
`-w`	Muestra el listado en formato ancho (como `/w` en MS-DOS/Windows)
`-a`	Incluye los ficheros ocultos.
`-f`	No muestra información sobre el espacio libre en el disco (así es más rápido).

Tabla 39. Modificadores para el comando `mdir`

3.3.5. Comando mcopy

El comando `mcopy` (*MS-DOS copy*) sirve para copiar ficheros entre el árbol de directorios de Linux y un disquete (disco en general) con formato MS-DOS/Windows sin necesidad de montar ni desmontar la unidad de disquetes (la unidad no debe estar montada para que funcione). La copia se puede realizar desde o hacia un disco MS-DOS/Windows.

SINTAXIS

```
mcopy [opciones] origen... [destino]
```

La composición de nombres de ficheros/directorios para las rutas MS-DOS/Windows sigue las mismas reglas que para el comando `mdir`.

Podemos especificar varios ficheros de origen, y en este caso, el destino será un directorio. También podemos especificar un fichero como origen y un fichero como destino, y entonces se copiará con el nuevo nombre. Finalmente podemos especificar sólo un fichero de origen, y el destino será el directorio por defecto.

Las opciones que admite `mcopy` son las resumidas en la Tabla 40.

Modificador	Resultado
`-t`	Copia ficheros de texto (hace conversión de `CR` a `CR+LF`, y viceversa).
`-n`	No pide confirmación antes de sobrescribir ficheros Linux.
`-o`	No pide confirmación antes de sobrescribir ficheros MS-DOS/Windows.

Tabla 40. Modificadores para el comando `mcopy`

3.3.6. Comando mkfs

El comando `mkfs` (*make file system*) sirve para crear discos con formato UNIX, aunque en Linux se pueden especificar otros sistemas de ficheros.

SINTAXIS

```
mkfs [opciones] dispositivo
```

Dispositivo identifica alguno de los dispositivos de disco (con posibilidad de escritura). La Tabla 37 (en la página 54) recoge algunos de los dispositivos de disco en Linux.

Con el parámetro `-t` podemos indicar a continuación el tipo de sistema de ficheros (como en el caso de `mount`). Además, con la opción `-c` hacemos que se verifique la superficie del disco en busca de bloques dañados.

Modificador	Resultado
`-t`	Permite especificar a continuación el tipo de sistema de ficheros contenido en la unidad.
`-c`	Comprueba los bloques defectuosos.

Tabla 41. Modificadores para el comando `mkfs`

3.4. Comandos de manejo de procesos

Ya sabemos (ver sección *1.7 Procesos en UNIX* en la página 16) que cada comando o programa que se está ejecutando en un sistema UNIX es un proceso y que los procesos se identifican mediante un número llamado *identificador de proceso (PID)*.

Estudiamos ahora los principales comandos que permiten manejar estos procesos.

3.4.1. Comando ps

El comando `ps` (*process status*) sirve para mostrar un listado de los procesos que actualmente se están ejecutando.

SINTAXIS

```
ps [-][opciones]
```

Las opciones de este comando se deben especificar sin ningún guión (-).Las versiones antiguas de `ps` sí aceptaban el guión; por eso ahora es opcional (aunque si lo usamos, nos mostrará un mensaje de advertencia).

Por defecto `ps` emite una lista únicamente de los procesos pertenecientes al usuario que ejecuta `ps`, con la siguiente información para cada proceso:

- PID: Número identificador de proceso.
- TTY: Nombre del terminal desde donde se solicita la ejecución (y que controla la ejecución).
- STAT: Estado del proceso: una letra con el siguiente significado: `R`: ejecutando, `S`: dormido (en pausa), `T`: parado (esperando algún recurso), `Z`: terminado.
- TIME: Tiempo que lleva ejecutándose el proceso (desde la última vez que pasó a ejecutarse).
- COMMAND: Nombre del comando que ejecuta (con los parámetros pasados).

Por ejemplo:

```
polifemo:~$ ps
  PID TTY STAT  TIME COMMAND
 2500  p1 S    0:00 -bash
 2897  p1 R    0:00 ps
```

Las opciones de `ps` se resumen en la Tabla 42.

Modificador	Resultado
`l`	Formato largo: muestra más información sobre cada proceso.
`u`	Muestra información sobre el usuario propietario del proceso y el tiempo de ejecución.
`s`	Muestra información sobre las señales que maneja cada proceso.
`m`	Muestra información sobre las memoria que maneja cada proceso.
`a`	Muestra también procesos de otros usuarios.
`h`	Elimina la línea de encabezado (con los títulos de las columnas).
`r`	Sólo muestra los procesos que se están ejecutando (no los parados).

Tabla 42. Modificadores para el comando `ps`

3.4.2. Comando kill

El comando `kill` sirve para enviar señales a procesos. Si el proceso no está preparado para recibir la señal, terminará inmediatamente su ejecución (morirá). La señal 9 (SIGKILL) siempre mata al proceso. Normalmente `kill` se emplea para matar procesos, de ahí su nombre.

SINTAXIS

```
kill [opciones] procesos...
```

Los procesos se identifican mediante su *PID*. Para averiguar el PID de un proceso tenemos varias opciones:

- Utilizar el comando `ps`.
- Al ejecutar un comando en segundo plano (poniendo `&` en la línea de comandos), el intérprete de comandos nos indica el *PID* del proceso que queda en segundo plano.

La principal opción que podemos utilizar es `-s`, que nos permite a continuación especificar el número de señal a enviar (mediante su nombre –en mayúsculas– o su número). Para ver las señales disponibles podemos utilizar la opción `-l`.:

```
polifemo:~$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL
 5) SIGTRAP      6) SIGIOT       7) SIGBUS       8) SIGFPE
 9) SIGKILL     10) SIGUSR1     11) SIGSEGV     12) SIGUSR2
13) SIGPIPE     14) SIGALRM     15) SIGTERM     17) SIGCHLD
18) SIGCONT     19) SIGSTOP     20) SIGTSTP     21) SIGTTIN
22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO
30) SIGPWR
```

Por defecto `kill` utiliza la señal `SIGTERM`, habitualmente utilizada para abortar procesos. Como `-s` es la opción más habitual, también podemos omitirla, escribiendo el nombre o número de la señal precedido de un guión. Los siguientes ejemplos envían la señal de muerte al proceso cuyo *PID* es el 2543 (todos los comandos son equivalentes):

```
kill -9 2543
kill -s 9 2543
kill -SIGKILL 2543
kill -s SIGKILL 2543
```

Un usuario normal sólo puede enviar señales a procesos que sean de su propiedad (comandos que él mismo haya ejecutado).

La Tabla 43 contiene las principales opciones disponibles para este comando.

Modificador	Resultado
`-l`	Muestra un listado de los nombres de señal y sus números.
`-`*XXXXX*	`-`*XXXXX* es el nombre o número de señal que hay que enviar.
`-s`	Permite especificar a continuación especificar el nombre o número de señal que hay que enviar.

Tabla 43. Modificadores para el comando `kill`

3.4.3. Comando top

`top` es un comando de Linux parecido a `ps`. Muestra información sobre los procesos activos en la máquina, con la diferencia de que el listado se refresca automáticamente a intervalos regulares de tiempo.

3.5. Comandos de administración

Los comandos que vamos a estudiar en esta sección están destinados a la administración del sistema. Algunos sólo pueden ser utilizados por el administrador; otros sirven para la administración personal de cada usuario.

3.5.1. Comando passwd

El comando `passwd` (*password*) permite a un usuario cambiar su clave de paso. Además, el administrador puede cambiar la clave de paso de otro usuario.

SINTAXIS

```
passwd [usuario]
```

El proceso de cambio de la clave se describió en la sección *1.5 Mantenimiento de la clave*, en la página 9.

3.5.2. Comando shutdown

El comando `shutdown` (*shut down*) sirve para iniciar el procedimiento ordenado de apagado del computador. Ya sabemos que los sistemas UNIX no pueden ser apagados sin antes haber sido detenidos. Con `shutdown`, además de detener el sistema para permitir el apagado, podemos informar a los usuarios conectados de la inminente detención, para que puedan finalizar sus trabajos ordenadamente.

SINTAXIS

```
shutdown [opciones] hora [mensaje]
```

- `hora` es la hora en que se comenzará la detención del sistema. Desde la ejecución del comando y hasta iniciar la detención no se permite que ningún usuario nuevo entre en el sistema. La hora se puede especificar de forma absoluta o relativa. El formato absoluto es `hh:mm` (p.e. `16:25`, las cuatro y veinticinco de la tarde); el formato relativo es `+mm` (p.e. `+15`, dentro de 15 minutos). También podemos utilizar la palabra `now`, equivalente a `+0`.
- mensaje es un mensaje que se envía a cada usuario conectado informándole de la inminente desconexión. Si consta de varias palabras deberemos encerrarlo entre comillas.

Por ejemplo, si necesitamos instalar una unidad de CD-ROM, tendremos que apagar el ordenador, y podemos usar el siguiente comando para apagar en un plazo de 10 minutos:

```
shutdown +10 "El sistema será interrumpido para instalar una unidad de CD-ROM"
```

Las opciones que admite este comando se resumen en la Tabla 44.

Modificador	Resultado
`-k`	Simulacro: envía el mensaje a los usuarios, pero no detiene el sistema.
`-r`	Hace que después de detener el sistema se vuelva a arrancar automáticamente.
`-c`	Permite cancelar una orden anterior de apagado. No se puede especificar de nuevo la hora, pero sí se puede enviar un mensaje a los usuarios para indicarles que el apagado se ha cancelado.

Tabla 44. Modificadores para el comando `shutdown`

Existen otros comandos que permiten detener el sistema: `halt` y `reboot`. Todos ellos deben ser utilizados de forma exclusiva por el administrador.

3.5.3. Comando adduser

El comando `adduser` (*add user*) sirve para abrir cuentas a nuevos usuarios, de forma que puedan entrar en el sistema. `adduser` no admite parámetros y sólo puede ser utilizado por el administrador.

SINTAXIS

```
adduser
```

Cuando ejecutamos este programa, nos irá pidiendo los datos del nuevo usuario: nombre de *login*, número de usuario, número de grupo, directorio *home*, shell, y fecha de expiración de la cuenta. Además, se pueden rellenar otros datos personales, como nombre completo, despacho, teléfono, etc. Finalmente habrá que asignarle una clave inicial al nuevo usuario. En muchos casos `adduser` nos proporcionará un valor por defecto, probablemente el más adecuado, por lo que en la mayoría de los casos nos bastará con pulsar *Enter* y aceptar el valor propuesto.

Veamos el siguiente ejemplo, en el que damos de alta un usuario al que llamaremos `user133`. Se resalta en negrita el texto tecleado por el administrador:

```
polifemo:~# adduser

Login name for new user (8 characters or less) []: user133
User id for user133 [ defaults to next available]:
Initial group for user133 [users]:
user133's home directory [/home/user133]:
user133's shell [/bin/bash]:
user133's account expiry date (MM/DD/YY) []:

OK, I'm about to make a new account. Here's what you entered so far:

New login name: user133
New UID: [Next available]
Initial group: users
Additional groups: [none]
Home directory: /home/user133
Shell: /bin/bash
Expiry date: [no expiration]

This is it... if you want to bail out, hit Control-C.  Otherwise, press
ENTER to go ahead and make the account.

Making new account...

Changing the user information for user133
Enter the new value, or press return for the default

        Full Name []: Usuario 133 de sistemas operativos
        Room Number []: Laboratorio de informatica II
        Work Phone []:
        Home Phone []:
        Other []:

Changing password for user133
Enter the new password (minimum of 5, maximum of 8 characters)
Please use a combination of upper and lower case letters and numbers.
New password:
Re-enter new password:
Password changed.
Done...
```

La información del nuevo usuario queda registrada en los archivos `/etc/passwd` (datos de las cuentas y personales) y `/etc/shadow` (claves de los usuarios, encriptadas).

3.5.4. Comando chsh

El comando `chsh` (*change shell*) nos permite cambiar el shell por defecto asociado a nuestra cuenta de usuario. Además, el administrador puede cambiar el shell de cualquier usuario.

SINTAXIS

```
chsh [-s shell] [usuario]
```

Con el parámetro `-s` podemos especificar el nuevo shell (ruta hasta el programa que funcionará como nuevo shell. Si no especificamos el parámetro, funcionará de forma interactiva, como `adduser`. En el siguiente ejemplo, el administrador cambia el shell del usuario `llorente` para que sea `/bin/csh`.

```
polifemo:~# chsh llorente
Changing the login shell for llorente
Enter the new value, or press return for the default

        Login Shell [/bin/bash]: /bin/csh
```

También podría haberse hecho utilizando `-s`:

```
polifemo:~# chsh -s /bin/csh llorente
```

3.5.5. Comando chfn

El comando `chfn` (*change full name*) sirve para modificar nuestros datos personales, que se introdujeron por primera ver al dar de alta al usuario con `addsuer`. Además, el administrador puede cambiar los datos personales de cualquier usuario, especificándolo como parámetro.

SINTAXIS

```
chfn [opciones] [usuario]
```

Los datos personales son: nombre completo, despacho, teléfono del despacho, teléfono de cada y otros. Con las opciones de la Tabla 45 podemos especificar los datos que hay que cambiar. Si no especificamos ninguna opción, el programa funciona de forma interactiva, como `adduser`. Por ejemplo:

```
polifemo:~# chfn llorente
Changing the user information for llorente
Enter the new value, or press return for the default

        Full Name []: Jesús M. Alvarez Llorente
        Room Number []: 101
        Work Phone []: 2321
        Home Phone []: 959222222
        Other []:
```

Modificador	Resultado
-f	Permite especificar a continuación el nombre completo (*full name*).
-r	Permite especificar a continuación el número de despacho (*room number*).
-w	Permite especificar a continuación el teléfono del despacho (*work phone*).
-g	Permite especificar a continuación el teléfono de cada (*home phone*).
-o	Permite especificar a continuación otra información (*other*).

Tabla 45. Modificadores para el comando chfn

3.6. Comandos de obtención de información

Los comandos de este apartado sirven para obtener diversa información, como la fecha y hora, el nombre de usuario, la lista de usuarios que están trabajando en la máquina, etc.

3.6.1. Comando who

El comando `who` sirve para mostrar el listado de usuarios conectados en un momento dado a la máquina UNIX.

SINTAXIS

```
who [opciones]
```

Si no se especifican opciones, `who` muestra un listado con la siguiente información, para cada usuario: Nombre de *login*, nombre del terminal desde el que se conecta, hora de entrada, dirección desde la que se conecta. Por ejemplo:

```
polifemo:~$ who
llorente ttyp0    Oct 22 10:18 (jmal.uhu.)
ggaleano ttyp2    Oct 22 11:13 (ggg.uhu.)
root     ttyp1    Oct 22 11:03 (:0.0)
```

La principal opción de `who` es "`-m`", que también podemos escribir como "`am i`". En este caso, `who` funciona como el comando `whoami`.

3.6.2. Comando whoami

El comando `whoami` (*who am I?*) sirve para obtener nuestro nombre de usuario. No admite parámetros.

SINTAXIS

```
whoami
```

Ejemplo:

```
polifemo:~$ whoami
llorente
```

3.6.3. Comando tty

El comando `tty` sirve para obtener el nombre de dispositivo correspondiente al terminal en el que estamos trabajando (concretamente al que utilizamos como entrada estándar).

SINTAXIS

```
tty [opciones]
```

Por ejemplo:

```
polifemo:~$ tty
/dev/ttyp1
```

3.6.4. Comando id

El comando `id` proporciona el nombre de usuario y número de identificación de usuario actual (UID), así como el nombre de los grupos y sus número de identificación (GID) a los que pertenezca.

SINTAXIS

```
id [opciones]
```

Por ejemplo:

```
polifemo:~$ id -a
uid=1000(llorente) gid=100(users) groups=100(users)
```

3.6.5. *Comando* date

Sirve para visualizar y cambiar la fecha y hora del computador UNIX. Sólo el administrador puede cambiar la fecha y hora.

SINTAXIS

```
date [opciones]
```

Por ejemplo:

```
polifemo:~$ date
Fri Oct 22 11:18:33 CEST 1999
```

3.6.6. *Comando* cal

El comando `cal` muestra un calendario del mes actual. Utilizando las opciones adecuadas pude mostrar el calendario del año actual o de cualquier mes de cualquier año. Si no se indica ningún año, muestra el calendario del mes actual; si especifica un año, pero no mes, se muestra el calendario completo del año especificado; y si se especifica tanto mes como año, se muestra sólo el mes indicado del año indicado.

SINTAXIS

```
date [opciones] [año [mes]]
```

Por ejemplo:

```
polifemo:~$ cal
October 2000
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
```

3.7. Comandos de control de sesión

Estos comandos permiten la entrada y salida del sistema.

3.7.1. Comando exit

El comando `exit` nos permite abandonar el shell, y con él la sesión actual. Si estamos en el shell inicial, saldremos completamente del sistema.

SINTAXIS

```
exit
```

3.7.2. Comando su

El comando `su` (*superuser*) nos permite entrar en el sistema como administrador, o como cualquier otro usuario especificando su nombre (suplantar a otro usuario).

SINTAXIS

```
su [usuario]
```

Cuando sea necesario, tras ejecutar el comando habrá que introducir la clave. Por ejemplo:

```
polifemo:~$ su chus
Password:
polifemo:/home/llorente#_
```

Ahora podemos ejecutar comandos como si fuéramos el nuevo usuario. Para finalizar el comando, hay que salir del nuevo shell en que se entra, con `exit`, con lo que conseguimos regresar a la sesión previa.

Habitualmente se emplea este comando para entrar como administrador temporalmente para realizar una tarea breve, como puede ser montar un disco, y retornar inmediatamente a lo que estábamos haciendo.

3.7.3. Comando login

Se trata de un comando parecido a `su`, que nos permite volver a entrar en el sistema, sin haber salido previamente del mismo. Este comando se ejecuta automáticamente cada vez que pretendemos entrar en el sistema, y es el que nos permite iniciar una sesión.

SINTAXIS

```
login [usuario [variables de entorno]]
```

A diferencia de `su`, con login se pierde la sesión anterior (si existiera), de manera que al invocar `exit`, se perderá la conexión con el sistema en lugar de regresar a la sesión anterior. Además, `login` hace una secuencia de entrada completa, incluyendo el cambio al directorio *home* del usuario, ejecución de los scripts de iniciación (ver apartado 5.5 *Scripts de iniciación* en la página 99), etc.

Existen más diferencias entre `login` y `su`, pero no vamos a estudiarlas.

3.8. Comandos de comunicación

3.8.1. *Comando* write

El comando `write` permite a un usuario enviar mensajes a otro. Para ello debe especificar el nombre de usuario y, opcionalmente, el nombre del dispositivo terminal (*tty*) en el que debe aparecer el mensaje.

SINTAXIS

```
write usuario[@host] [tty]
```

Se puede escribir mensajes a usuarios que se encuentren en otras máquinas UNIX, especificando el nombre completo de la máquina a continuación del nombre de usuario, (separándolo con @).

El mensaje enviado se toma de la entrada estándar, y el destinatario lo recibe en el terminal que esté utilizando (o el indicado explícitamente). Además, antes del mensaje recibirá una nota indicándole la recepción y el remitente. Por ejemplo, el usuario `llorente` escribe al usuario `chus`:

terminal del usuario llorente

```
polifemo:~$ write chus
Hola chus, estoy aquí.
^D
```

terminal del usuario chus

```
Message from llorente@polifemo.uhu.es on ttyp1 at 19:23 ...
Hola chus, estoy aquí.
EOF
```

Si el usuario de destino no se encuentra en el sistema, se producirá un error.

3.8.2. *Comando* mesg

El comando `mseg` (*messages*) permite activar y desactivar la recepción de mensajes enviados mediante `write`.

SINTAXIS

```
mesg [y] [n]
```

Con la opción `y`, se activa la recepción, y con `n`, se desactiva. Si no se especifican opciones, `mseg` indica el estado de activación por la salida estándar ("`is y`" para activado, "`is n`" para desactivado) y mediante el valor de finalización (0 para activado, 1 para desactivado)

3.8.3. Comando mail

El comando `mail` permite enviar mensajes de correo electrónico. Especificando un usuario, se envía el mensaje a ese usuario dentro de la máquina, pero además se puede especificar un nombre de máquina.

SINTAXIS

```
mail usuario[@host] [opciones]
```

El asunto del mensaje (*subject*) , así como el texto del mismo se toman de la entrada estándar. Si trabajamos desde el terminal, el asunto es preguntado por el comando antes de que se escriba el texto. En el caso de que trabajemos con redirección desde un archivo o tubería, el asunto se toma a partir de una línea (al principio del archivo) que comience con "`subject:`". El mensaje termina con una línea que únicamente contiene un punto (`.`). Este comportamiento se puede modificar mediante el uso de distintas opciones.

Por ejemplo, para enviar un mensaje a llorente@uhu.es:

SINTAXIS

```
polifemo:~$ mail llorente@uhu.es
Subject: hola
Hola. Acabo de llegar.
Saludos.
.
EOT
```

3.9. Comandos de cálculo

Este grupo de comandos permiten realizar operaciones matemáticas, lógicas, de comparación, etc.

3.9.1. Comando expr

El comando `expr` sirve evaluar expresiones aritméticas. El resultado de la evaluación se escribe en la salida estándar.

SINTAXIS

```
expr expresión
```

La `expresión` está compuesta por varios literales (números enteros o cadenas de caracteres) y operadores, separados siempre por espacios. Los operadores que coincidan con caracteres especiales para el shell (p.e. los paréntesis) deberán ser precedidos por una barra inversa o encerrados entre comillas para que pierdan su significado especial. La Tabla 46 contiene los operadores más importantes admitidos por el comando `expr`. Existen operadores aritméticos, lógicos y de manejo de cadenas. Los operadores de comparación devuelven 0 como falso y 1 como verdadero; los operadores lógicos utilizan el valor 0 como falso y cualquier número distinto de 0 como verdadero. Los operadores no se pueden utilizar como literales de tipo cadena.

Operador	Significado
`+`	Suma de enteros.
`-`	Resta de enteros.
`*`	Producto de enteros.
`/`	División de enteros.
`%`	Módulo (resto de la división) entre enteros.
`<`	Comparador *menor que*.
`<=`	Comparador *menor o igual*.
`=`	Comparador *igual a*.
`==`	Comparador *igual a* (idéntico a =).
`!=`	Comparador *distinto de*.
`>=`	Comparador *mayor o igual*.
`>`	Comparador *mayor*.
`|`	*OR* lógico.
`&`	*AND* lógico.
`()`	Permiten especificar el orden de las operaciones.
`substr`	Devuelve una subcadena. El formato es `substr` *`cadena posición longitud`*, y devuelve los *`longitud`* caracteres a partir de la posición *`posición`* de la cadena *`cadena`*.
`length`	Devuelve la longitud de una cadena. El formato es `length` *`cadena`*.
`index`	Devuelve la posición de un carácter en una cadena. El formato es `index` *`cadena muestra`*, y devuelve la posición en la cadena *`cadena`* de la primera aparición del primer carácter de la cadena *`muestra`*.

Tabla 46. Operadores válidos para el comando `expr`

El orden de precedencia es el siguiente: operador *OR*, operador *AND*, operadores de comparación, operadores de adición, operadores de multiplicación.

El siguiente ejemplo de uso de este comando sirve para sumar 1 a la variable `CONTADOR`:

```
CONTADOR=`expr $CONTADOR + 1`
```

3.9.2. Comando test

El comando `test` sirve para comprobar una condición. Como resultado, no ofrece ninguna salida. Si la condición comprobada es verdadera, termina con éxito (estado 0), y si no, termina con error (estado distinto de 0).

SINTAXIS

```
test expresión
```

Este comando se utiliza con las estructuras `if` y `while` que permiten construir bucles iterativos en la programación de *scripts* para el shell (lo estudiaremos en el apartado *5 Scripts en bash*, en la página 91). Para hacer más fácil su comprensión dentro de los de *scripts*, tiene una sintaxis alternativa. En este caso los corchetes no indican opcionalidad, sino que deben escribirse literalmente:

SINTAXIS ALTERNATIVA

```
[ expresión ]
```

En realidad el corchete izquierdo "`[`" es un programa, concretamente un enlace al comando `test`, situado en alguna parte del árbol de directorios. Por esta razón no debemos olvidar dejar un espacio de separación al menos entre el corchete izquierdo y el primer carácter de la expresión.

Las posibles expresiones se recogen en la Tabla 47. Todas las que se refieren a ficheros, son falsas si el fichero referido no existe. Además, se pueden combinar varias expresiones utilizando los operadores de la Tabla 48. A continuación veremos algunos ejemplos de uso de test:

- Comprobar la existencia de un fichero llamado `/tmp/stastus`.
- Comprobar si el directorio actual es el raíz.
- Comprobar si el número de usuarios conectados es mayor que 5[11].
- Comprobar si /tmp/a35627 es un directorio o un fichero (no un dispositivo, ni un enlace, etc.).
- Comprobar que el directorio actual en el *home*, y que nuestro nombre de usuario no es *root*.

```
[ -f /tmp/status ]
[ "/" = `pwd` ]
[ `who | wc -l` -gt 5 ]
[ -d /tmp/a35627 -o -f /tmp/a35627 ]
[ \( $HOME = `pwd` \) -a \( ! $USER = root \) ]
```

[11] Utilizar el comando `who` (página 63) para obtener un listado de usuarios conectados.

Expresión	La expresión es verdadera cuando...
`-r ruta`	*ruta* es un fichero con permiso de lectura (podemos acceder para leer).
`-w ruta`	*ruta* es un fichero con permiso de escritura (podemos acceder para escribir).
`-x ruta`	*ruta* es un fichero con permiso de ejecución (podemos ejecutarlo).
`-f ruta`	*ruta* es un fichero normal (no un directorio, enlace, dispositivo, etc.).
`-d ruta`	*ruta* es un directorio.
`-b ruta`	*ruta* es un dispositivo de bloques.
`-c ruta`	*ruta* es un dispositivo de caracteres.
`-L ruta`	*ruta* es un enlace simbólico.
`-s ruta`	*ruta* es un fichero con tamaño mayor que cero.
`-O ruta`	*ruta* es un fichero cuyo propietario somos nosotros.
`-G ruta`	*ruta* es un fichero cuyo grupo es el nuestro.
ruta1 `-ot` *ruta2*	*ruta1* es un fichero más antiguo que *ruta2*.
ruta1 `-nt` *ruta2*	*ruta1* es un fichero más reciente que *ruta2*.
`-z` *cadena*	*cadena* es una cadena de caracteres de longitud cero.
`-n` *cadena*	*cadena* es una cadena de caracteres de longitud mayor que cero.
cadena1 `=` *cadena2*	*cadena1* es una cadena igual a *cadena2*.
cadena1 `!=` *cadena2*	*cadena1* es una cadena diferente de *cadena2*.
número1 `-eq` *número2*	*número1* es un número entero igual a *número2*.
número1 `-ne` *número2*	*número1* es un número entero diferente de *número2*.
número1 `-gt` *número2*	*número1* es un número entero mayor que *número2*.
número1 `-ge` *número2*	*número1* es un número entero mayor o igual que *número2*.
número1 `-lt` *número2*	*número1* es un número entero menor que *número2*.
número1 `-le` *número2*	*número1* es un número entero menor o igual que *número2*.

Tabla 47. Expresiones válidas para el comando `test`

Operador	Significado
`!`	Negación.
`-a`	AND lógico.
`-o`	OR lógico.
`()`	Permiten especificar orden de precedencia (hay que utilizarlos con \ o con comillas para eliminar su significado especial para el shell)

Tabla 48. Operadores válidos para el comando `test`

3.10. Comandos de escritura

A continuación estudiamos algunos comandos que sirven para escribir, mostrar mensajes, borrar la pantalla, etc.

3.10.1. Comando echo

El comando `echo` sirve para escribir mensajes en la salida estándar. El mensaje se especifica como parámetro. Para escribir mensajes con caracteres especiales (p.e. espacios) debemos encerrarlo entre comillas.

SINTAXIS

```
echo [opciones] mensaje...
```

Un mensaje formado por varias palabras puede especificarse sin comillas, pero cada palabra se debe separar sólo por un espacio, ya que cada palabra se considerará un parámetro distinto, y `echo` lo que hace es escribir cada parámetro separado un espacio del anterior.

Después del mensaje, `echo` imprime un retorno de carro (comienza una nueva línea). Para evitar esto, podemos utilizar el parámetro `-n`.

Veamos los siguientes ejemplos. Observamos cómo en el último caso, el prompt aparece inmediatamente a la derecha del mensaje porque hemos empleado `-n`.

```
polifemo:~$ echo Esto son 3 letras: a  b  c
Esto son 3 letras: a b c
polifemo:~$ echo "Esto son 3 letras: a  b  c"
Esto son 3 letras: a  b  c
polifemo:~$ echo -n "Esto son 3 letras: a  b  c"
Esto son 3 letras: a  b  cpolifemo:~$
```

`echo` se utiliza a menudo para imprimir variables. Por ejemplo, para ver el contenido de la variable `PATH`:

```
polifemo:~$ echo "El path es $PATH"
El path es .:/usr/local/bin:/bin:/usr/bin:/usr/X11/bin:/usr/local/jdk/bin:/usr/o
penwin/bin:/usr/local/pascalfc/bin:/usr/lib/teTeX/bin
```

Modificador	Resultado
`-n`	Evita que se imprima un carácter de nueva línea al final del mensaje.

Tabla 49. Modificadores para el comando echo

3.10.2. Comando clear

El comando `clear` sirve para limpiar (borrar) la pantalla. No admite parámetros.

SINTAXIS

```
clear
```

3.10.3. Comando reset

El comando `reset` sirve para borrar la pantalla y reiniciar el terminal. No admite parámetros. Resulta útil cuando el termina pierde su configuración habitual, por ejemplo, cuando aborta o falla un programa que haya configurado el terminal de forma especial.

SINTAXIS

```
reset
```

3.10.4. *Comando* banner

Este comando permite construir pancartas formados por letras ampliadas.

SINTAXIS

```
banner [opciones] mensaje
```

Podemos especificar el tamaño del papel y el texto de la pancarta con `-w`*`tamaño`*. Por ejemplo:

```
polifemo:~$ banner -w 20 hola
     #            #
     ##############
            #
            #
     ########
        ##
      #######
     #
     #      #
      ######
     #            #
     ##############
       #
     ##### ##
     #   #
     ########
     #
```

3.11. Comandos de manejo de variables

Estudiamos en este apartado dos comandos utilizados para el manejo de variables en el shell. En la sección *2.1.5 Variables de entorno* (página 24) hemos estudiado el manejo de las variables.

3.11.1. Comando export

El comando `export` permite hacer que el valor de una variable del shell sea transmitido los programas que ejecute el shell. Admite como parámetro el conjunto de variables que queremos exportar.

SINTAXIS

```
export [variable...]
```

Por defecto, las variables definidas en un shell son locales a ese shell, y no se transmiten a los programas que ejecutemos desde él. Con este comando indicamos al shell la necesidad de que sí se transmitan a todos los programas que ejecutemos a partir de ese momento.

Si no se especifica ningún parámetro, `export` muestra una lista con las variables que se están exportando actualmente.

3.11.2. Comando set

El comando `set` permite ver el valor de todas las variables definidas. Además, permite activar y desactivar algunas opciones del shell (ver apartado 2.1.3 *Configuración de opciones de* bash en la página 22).

SINTAXIS

```
set
```

Por ejemplo:

```
polifemo:~$ set
BASH=/bin/bash
BASH_VERSION=1.14.7(1)
EUID=1000
HISTFILE=/home/llorente/.bash_history
HISTFILESIZE=500
HISTSIZE=500
HOME=/home/llorente
HOSTNAME=polifemo.uhu.es
HOSTTYPE=i386
HUSHLOGIN=FALSE
LOGNAME=llorente
LS_OPTIONS=--8bit --color=tty -F -b -T 0
OSTYPE=Linux
PATH=.:/usr/local/bin:/bin:/usr/bin:/usr/X11/bin:/usr/local/jdk/bin:/usr/openwin/bin:/usr/local
/pascalfc/bin:/usr/lib/teTeX/b
in:/usr/local/pgsql/bin
PGDATA=/usr/local/pgsql/data
PGLIB=/usr/local/pgsql/lib
PPID=27357
PS1=\h:\w\$
SHELL=/bin/bash
SHLVL=1
TERM=vt220
UID=1000
USER=llorente
```

3.11.3. Comando alias

El comando `alias` permite ver el valor de las opciones de shell, así como activarlas y desactivarlas, como hemos estudiado en el apartado 2.1.2 *Los alias* (página 21).

SINTAXIS

```
alias [nombre='comando [opciones]']
```

Para visualizar el conjunto de alias definidos, escribimos el comando sin opciones, y para definir un alias, escribimos como opción el nombre del alias seguido del símbolo = (sin dejar espacios en medio) y a continuación el comando equivalente con sus parámetros si fuera necesario, recomendablemente entre comillas para evitar sustituciones erróneas. Por ejemplo: `alias dir='ls -al'`.

3.12. Comandos de control de tiempo

Los comandos que veremos a continuación sirven para hacer pausas, medir el tiempo y ejecutar programas a horas específicas.

3.12.1. Comando sleep

El comando `sleep` sirve para hacer una pausa (detener la ejecución) durante un tiempo (inicialmente especificado en segundos).

SINTAXIS

```
sleep tiempo[unidad de tiempo]
```

El `tiempo` es un número entero. La `unidad de tiempo` es alguno de los siguientes caracteres: `s` (segundos), `m` (minutos), `h` (horas), `d` (días). Si se omite la unidad de tiempo se presupone `s` (segundos).

Ejemplo: hacer una pausa de 10 segundos: `sleep 10`.

3.12.2. *Comando time*

El comando `time` sirve para medir el tiempo de ejecución de otros comandos. Admite como parámetro otro comando. `time` ejecuta ese comando y después nos imprime el tiempo que ha tardado. Este tiempo se separa en tiempo de usuario, tiempo de núcleo, fallos de caché, etc.

SINTAXIS

```
time [opciones] comando [argumentos del comando]
```

Por ejemplo:

```
polifemo:~$ time sleep 2
0.00user 0.00system 0:02.00elapsed 0%CPU (0avgtext+0avgdata 0maxresident)k
0inputs+0outputs (62major+8minor)pagefaults 0swaps
```

3.12.3. *Comando at*

Este comando sirve para ejecutar comandos a una hora determinada.

SINTAXIS

```
at [opciones] comando [argumentos del comando]
```

El siguiente ejemplo ejecutará el programa `jesus1` a las 11 de la mañana del 23 de octubre:

```
polifemo:~$ at -f jesus1 11am Oct 23
Job 4 will be executed using /bin/sh
```

3.13. Comandos de ayuda

Estudiamos en este apartado comandos muy importantes que sirven para obtener información sobre otros comandos.

3.13.1. Comando man

El comando `man` (*manual*) sirve para visualizar las páginas del manual. Pasamos como parámetro el nombre del tema que queremos consultar. Si pasamos más de un tema, se mostrará la información de cada tema cada vez que terminemos con el anterior (tecla *q*).

SINTAXIS

```
man [sección] tema...
```

La visualización de temas se hace mediante el comando `less`, por lo que podemos utilizar las mismas teclas para desplazarnos por el texto.

El manual contiene información no sólo de comandos, sino de ficheros, funciones del lenguaje C, funciones de otros lenguajes, etc. Toda esta información se almacena en distintas secciones. Normalmente no nos importa en qué sección aparece un tema, pero a veces el mismo tema aparece en distintas secciones. Por ejemplo, la información del comando `mount` puede confundirse con la información de la rutina de C `mount()`. En tal situación quizá sea necesario indicar en qué sección del manual se encuentra el tema que nosotros queremos ver. Las secciones se identifican mediante un número. Por ejemplo:

- Para ver la página del comando `mount`: `man 8 mount` o simplemente `man mount`.
- Para ver la página de la rutina de C `mount()`: `man 2 mount`.

Dentro de la información del manual, a menudo se hace referencia a otras páginas del manual, indicando el nombre del tema y poniendo entre paréntesis el número de la sección.

Si no se especifica la sección, `man` buscará en todas las secciones y mostrará el tema de la primera sección que lo contenga. Si se especifica alguna sección, `man` sólo buscará en esa sección.

Las secciones más importantes para nosotros son:

- Sección 8: comandos.
- Sección 2: llamadas al sistema.
- Sección 3: rutinas del lenguaje C.

3.13.2. Comando whatis

El comando `whatis` (*what is?*) sirve para visualizar descripciones de palabras clave de temas relacionados con el shell, lenguajes de programación, comandos, etc.

SINTAXIS

```
whatis [opciones] tema...
```

Por ejemplo:

```
polifemo:~$ whatis ls
ls, dir, vdir (1)    - list contents of directories
ls-R (5)             - database for common TeX input files
```

En el listado aparecen, a la izquierda, un conjunto de temas que pueden ser consultados en el manual (`man`), con indicación de la sección correspondiente, y a la derecha, una breve descripción del tema. Al igual que ocurre con el manual, la información proporcionada por `whatis` se muestra mediante el comando `less`.

3.13.3. Comando apropos

El comando `apropos` es semejante a `whatis`, pero ofrece más información porque realiza una búsqueda más extensa.

SINTAXIS

```
whatis [opciones] tema...
```

3.14. Resumen de comandos

Comando	Significado	Categoría	Utilidad
`adduser`	*add user*	Administración	Añadir un nuevo usuario.
`alias`	*alias*	Manejo variables	Permite definir alias y mostrar los existentes.
`apropos`	*apropos*	Ayuda	Muestra información relacionada con un tema.
`at`	*at*	Control tiempo	Ejecuta un comando a una hora determinada.
`banner`	*banner*	Escritura	Imprime una pancarta.
`cal`	*calendar*	Información	Muestra un calendario.
`cat`	*concatenate*	Proceso ficheros	Sacar el contenido de ficheros (por pantalla).
`cd`	*change directory*	Manejo ficheros	Cambia el directorio por defecto.
`chfn`	*change full name*	Administración	Cambiar los datos personales de un usuario.
`chmod`	*change mode*	Manejo ficheros	Cambia los permisos de un fichero o directorio.
`chown`	*change owner*	Manejo ficheros	Cambia el propietario y grupo de un fichero o directorio.
`chsh`	*change shell*	Administración	Cambiar el shell que utiliza un usuario.
`clear`	*clear*	Escritura	Borra la pantalla.
`cmp`	*compare*	Proceso ficheros	Busca las diferencias entre dos ficheros.
`comm`	*common*	Proceso ficheros	Separa las líneas comunes y no comunes de dos ficheros ordenados.
`cp`	*copy*	Manejo ficheros	Copiar ficheros.
`date`	*date*	Información	Visualiza o cambiar la fecha y hora.
`df`	*disk free*	Discos	Indica la capacidad, estado y espacio libre de los discos del sistema.
`echo`	*echo*	Escritura	Escribe un mensaje en la salida estándar.
`exit`	*exit*	Control sesión	Finalizar la sesión con el shell.
`export`	*export*	Manejo variables	Hace que una variable se transmita desde el shell a los programas.
`expr`	*expression*	Cálculo	Evalúa una expresión aritmética.
`find`	*find*	Proceso ficheros	Buscar un fichero.
`grep`	*get regular expression and print*	Proceso ficheros	Mostrar las líneas de un fichero que contienen un patrón.
`halt`	*halt*	Administración	Detener el sistema.
`head`	*head*	Proceso ficheros	Mostrar las primeras líneas de un fichero.
`id`	*identification*	Información	Indica el número de identificación de usuario (UID) y grupo (GID).
`kill`	*kill*	Control procesos	Enviar una señal a un proceso.
`less`	*less (opposite of more)*	Proceso ficheros	Mostrar un fichero por páginas, permitiendo movimiento manual.
`ln`	*link*	Manejo ficheros	Crea enlaces entre ficheros o directorios.

Comando	Significado	Categoría	Utilidad
`login`	*login*	Control sesión	Entra en el sistema como otro usuario sin haber salido previamente.
`logout`	*log out*	Control sesión	Finalizar la sesión con el shell.
`lpr`	*line printer*	Proceso ficheros	Envía un fichero a la impresora.
`ls`	*list subdirectory*	Manejo ficheros	Listar subdirectorios.
`mail`	*mail*	Comunicación	Envía un mensaje de correo electrónico
`man`	*manual*	Ayuda	Ver una página del manual.
`mcopy`	*MS-DOS copy*	Discos	Copiar los ficheros de un disquete MS-DOS.
`mdir`	*MS-DOS directory*	Discos	Listar los ficheros de un disquete MS-DOS.
`mesg`	*message*	Comunicación	Activa o desactiva la recepción de mensajes.
`mkdir`	*make directory*	Manejo ficheros	Crear subdirectorios.
`mkfs`	*make file system*	Discos	Preparar un disquete con formato UNIX (Linux).
`more`	*more*	Proceso ficheros	Mostrar un fichero haciendo una pausa en cada pantalla.
`mount`	*mount*	Discos	Montar un dispositivo en una ruta.
`mv`	*move*	Manejo ficheros	Cambiar el nombre y/o ruta de ficheros (o directorios).
`passwd`	*password*	Administración	Cambia la clave de paso.
`ps`	*process status*	Control procesos	Listar los procesos actuales.
`pwd`	*print working directory*	Manejo ficheros	Muestra el directorio por defecto.
`reboot`	*reboot*	Administración	Iniciar el procedimiento de apagado y reiniciar.
`reset`	*reset*	Escritura	Reinicia el terminal y borra la pantalla.
`rm`	*remove*	Manejo ficheros	Borrar ficheros.
`rmdir`	*remove directory*	Manejo ficheros	Borrar subdirectorios.
`set`	*set*	Manejo variables	Muestra las variables y activa o desactiva opciones del shell.
`shutdown`	*shutdown*	Administración	Iniciar el procedimiento de apagado.
`sleep`	*sleep*	Control tiempo	Hacer una pausa.
`sort`	*sort*	Proceso ficheros	Ordenar las líneas de un fichero.
`su`	*super user*	Control sesión	Entra en el sistema como administrador, o como otro usuario.
`tail`	*tail*	Proceso ficheros	Mostrar las últimas líneas de un fichero.
`test`	*test*	Cálculo	Comprobar una condición.
`time`	*time*	Control tiempo	Cuanta el tiempo que tarda un comando en ejecutarse.
`top`	*top*	Control procesos	Mostrar los procesos actuales.
`tty`	*tty*	Información	Indica el nombre de dispositivo del terminal de entrada actual.
`umount`	*unmount*	Discos	Desmontar un dispositivo montado.

Comando	Significado	Categoría	Utilidad
`uniq`	*unique*	Proceso ficheros	Muestra un fichero sin repetir líneas iguales contiguas.
`wc`	*word count*	Proceso ficheros	Contar el número de líneas, palabras y caracteres de un fichero.
`whatis`	*what is?*	Ayuda	Muestra información relacionada con un tema.
`who`	*who*	Información	Lista los usuarios conectados a la máquina UNIX.
`whoami`	*who am I?*	Información	Devuelve el nombre de usuario.
`write`	*write*	Comunicación	Envía un mensaje a otro usuario.

Tabla 50. Resumen de comandos, ordenados alfabéticamente

Comando	Significado	Categoría	Utilidad
`adduser`	*add user*	Administración	Añadir un nuevo usuario.
`chfn`	*change full name*	Administración	Cambiar los datos personales de un usuario.
`chsh`	*change shell*	Administración	Cambiar el shell que utiliza un usuario.
`halt`	*halt*	Administración	Detener el sistema.
`passwd`	*password*	Administración	Cambia la clave de paso.
`reboot`	*reboot*	Administración	Iniciar el procedimiento de apagado y reiniciar.
`shutdown`	*shutdown*	Administración	Iniciar el procedimiento de apagado.
`apropos`	*apropos*	Ayuda	Muestra información relacionada con un tema.
`man`	*manual*	Ayuda	Ver una página del manual.
`whatis`	*what is?*	Ayuda	Muestra información relacionada con un tema.
`expr`	*expression*	Cálculo	Evalúa una expresión aritmética.
`test`	*test*	Cálculo	Comprobar una condición.
`mail`	*mail*	Comunicación	Envía un mensaje de correo electrónico
`mesg`	*message*	Comunicación	Activa o desactiva la recepción de mensajes.
`write`	*write*	Comunicación	Envía un mensaje a otro usuario.
`kill`	*kill*	Control procesos	Enviar una señal a un proceso.
`ps`	*process status*	Control procesos	Listar los procesos actuales.
`top`	*top*	Control procesos	Mostrar los procesos actuales.
`exit`	*exit*	Control sesión	Finalizar la sesión con el shell.
`login`	*login*	Control sesión	Entra en el sistema como otro usuario sin haber salido previamente.
`logout`	*log out*	Control sesión	Finalizar la sesión con el shell.
`su`	*super user*	Control sesión	Entra en el sistema como administrador, o como otro usuario.
`at`	*at*	Control tiempo	Ejecuta un comando a una hora determinada.
`sleep`	*sleep*	Control tiempo	Hacer una pausa.
`time`	*time*	Control tiempo	Cuanta el tiempo que tarda un comando en ejecutarse.
`df`	*disk free*	Discos	Indica la capacidad, estado y espacio libre de los discos del sistema.
`mcopy`	*MS-DOS copy*	Discos	Copiar los ficheros de un disquete MS-DOS.
`mdir`	*MS-DOS directory*	Discos	Listar los ficheros de un disquete MS-DOS.
`mkfs`	*make file system*	Discos	Preparar un disquete con formato UNIX (Linux).
`mount`	*mount*	Discos	Montar un dispositivo en una ruta.
`umount`	*unmount*	Discos	Desmontar un dispositivo montado.
`banner`	*banner*	Escritura	Imprime una pancarta.

Comando	Significado	Categoría	Utilidad
`clear`	*clear*	Escritura	Borra la pantalla.
`echo`	*echo*	Escritura	Escribe un mensaje en la salida estándar.
`reset`	*reset*	Escritura	Reinicia el terminal y borra la pantalla.
`cal`	*calendar*	Información	Muestra un calendario.
`date`	*date*	Información	Visualiza o cambiar la fecha y hora.
`id`	*identification*	Información	Indica el número de identificación de usuario (UID) y grupo (GID).
`tty`	*tty*	Información	Indica el nombre de dispositivo del terminal de entrada actual.
`who`	*who*	Información	Lista los usuarios conectados a la máquina UNIX.
`whoami`	*who am I?*	Información	Devuelve el nombre de usuario.
`cd`	*change directory*	Manejo ficheros	Cambia el directorio por defecto.
`chmod`	*change mode*	Manejo ficheros	Cambia los permisos de un fichero o directorio.
`chown`	*change owner*	Manejo ficheros	Cambia el propietario y grupo de un fichero o directorio.
`cp`	*copy*	Manejo ficheros	Copiar ficheros.
`ln`	*link*	Manejo ficheros	Crea enlaces entre ficheros o directorios.
`ls`	*list subdirectory*	Manejo ficheros	Listar subdirectorios.
`mkdir`	*make directory*	Manejo ficheros	Crear subdirectorios.
`mv`	*move*	Manejo ficheros	Cambiar el nombre y/o ruta de ficheros (o directorios).
`pwd`	*print working directory*	Manejo ficheros	Muestra el directorio por defecto.
`rm`	*remove*	Manejo ficheros	Borrar ficheros.
`rmdir`	*remove directory*	Manejo ficheros	Borrar subdirectorios.
`alias`	*alias*	Manejo variables	Permite definir alias y mostrar los existentes.
`export`	*export*	Manejo variables	Hace que una variable se transmita desde el shell a los programas.
`set`	*set*	Manejo variables	Muestra las variables y activa o desactiva opciones del shell.
`cat`	*concatenate*	Proceso ficheros	Sacar el contenido de ficheros (por pantalla).
`cmp`	*compare*	Proceso ficheros	Busca las diferencias entre dos ficheros.
`comm`	*common*	Proceso ficheros	Separa las líneas comunes y no comunes de dos ficheros ordenados.
`find`	*find*	Proceso ficheros	Buscar un fichero.
`grep`	*get regular expression and print*	Proceso ficheros	Mostrar las líneas de un fichero que contienen un patrón.
`head`	*head*	Proceso ficheros	Mostrar las primeras líneas de un fichero.
`less`	*less (opposite of more)*	Proceso ficheros	Mostrar un fichero por páginas, permitiendo movimiento manual.
`lpr`	*line printer*	Proceso ficheros	Envía un fichero a la impresora.
`more`	*more*	Proceso ficheros	Mostrar un fichero haciendo una pausa en cada pantalla.

Comando	Significado	Categoría	Utilidad
`sort`	*sort*	Proceso ficheros	Ordenar las líneas de un fichero.
`tail`	*tail*	Proceso ficheros	Mostrar las últimas líneas de un fichero.
`uniq`	*unique*	Proceso ficheros	Muestra un fichero sin repetir líneas iguales contiguas.
`wc`	*word count*	Proceso ficheros	Contar el número de líneas, palabras y caracteres de un fichero.

Tabla 51. Resumen de comandos, ordenados por categorías

4. Editores de texto

El Linux se incluyen multitud de editores de texto. Vamos a estudiar brevemente 3 de ellos, los más importantes dentro del mundo Linux:

- *joe*: se caracteriza por que utiliza las mismas combinaciones de teclas que los editores *Micro-Pro WordStar* o los editores de los IDE de Borland "*Turbo*"
- *emacs*: un editor muy grande, completo y extendido. Además puede funcionar bajo entorno gráfico.
- *vi*: bastante complicado de manejar, pero es importante conocerlo por que es el estándar en UNIX (lo encontraremos en cualquier máquina UNIX).
- *wpe*: un entorno de programación consistente en un editor similar a los editores de los IDE de Borland "*Turbo*" más modernos, que integra la invocación de distintos compiladores y un entorno de traza.
- *xwpe*: la versión gráfica bajo X-Window de *wpe*.

4.1. Editor joe

El editor *joe* se caracteriza por que utiliza las mismas combinaciones de teclas que los editores *Micro-Pro WordStar* o los editores de los IDE de Borland "*Turbo*", por lo que nos puede resultar muy familiar.

Al ejecutar el editor, admite como parámetro el nombre del fichero que queremos editar. No estudiaremos las opciones. No obstante, podemos conocerlas con ayuda del manual (`man joe`).

SINTAXIS

```
joe [opciones] [fichero]
```

La mayoría de las combinaciones de teclas que utiliza *joe* están formadas por dos pulsaciones consecutivas. Por ejemplo, con ^*KH* (pulsamos la tecla *Ctrl* y la tecla *K*, y luego la tecla *H*) *joe* nos muestra una pantalla de ayuda con las combinaciones de teclas más importantes. Otras combinaciones se hacen en una sola pulsación. Por ejemplo, con ^*Y* (pulsamos la tecla *Ctrl* y la tecla *Y*) borramos una línea del fichero.

La Tabla 52 contiene las principales combinaciones de teclas que se utilizan en el editor *joe*.

Combinación	Categoría	Significado
^KD	Archivos	Guarda el fichero actual
^KE	Archivos	Carga un fichero de disco para editarlo.
^KB	Bloques	Marca el inicio de un bloque.
^KC	Bloques	Copia el bloque marcado a la posición actual.
^KK	Bloques	Marca el final de un bloque.
^KM	Bloques	Mueve el bloque marcado a la posición actual.
^KR	Bloques	Inserta el contenido de un archivo en la posición del cursor.
^KW	Bloques	Guarda el bloque marcado en un fichero.
^KY	Bloques	Borrar el bloque marcado.
^KF	Búsqueda	Permite especificar un texto para ser buscado en el fichero.

Combinación	Categoría	Significado
^L	Búsqueda	Repite la última búsqueda realizada (busca la siguiente aparición).
^^	Edición	Vuelve a hace r los últimos cambios deshechos (en los teclados en español, esta tecla es ^' (*Ctrl* más la tecla de apóstrofe). En conexiones vía *telnet* posiblemente esta combinación no funcione.
^_	Edición	Deshace los últimos cambios. En conexiones vía *telnet* posiblemente esta combinación no funcione.
^D	Edición	Borra el carácter situado a la derecha del cursor.
^O	Edición	Borra la palabra situada a la izquierda del cursor.
^W	Edición	Borra la palabra situada a la derecha del cursor.
^Y	Edición	Borra la línea en la posición del cursor.
Del	Edición	Borra el carácter situado a la izquierda del cursor.
^A	Movimiento	Mueve el cursor al principio de la línea.
^E	Movimiento	Mueve el cursor al final de la línea.
^KL	Movimiento	Mueve el cursor a un número de línea que se introduce por teclado.
^KU	Movimiento	Mueve el cursor al principio del fichero.
^KV	Movimiento	Mueve el cursor al final del fichero.
^U	Movimiento	Retrocede una página.
^V	Movimiento	Avanza una página.
^X	Movimiento	Mueve el cursor una palabra a la derecha.
^Z	Movimiento	Mueve el cursor una palabra a la izquierda.
^C	Varios	Termina y sale al *shell*.
^KH	Varios	Muestra u oculta la ventana de ayuda sobre el teclado.
^KX	Varios	Guarda el fichero actual, termina y sale al *shell*.
^KZ	Varios	Sale al *shell* para ejecutar comandos (se puede volver con exit).
^T	Varios	Accede al menú de configuración de modo de edición, que permite cambiar modo de inserción, modo de indentación, modo de corte de palabras, tamaño de las tabulaciones, márgenes, etc.

Tabla 52. Combinaciones de teclas del editor joe

Podemos crear un fichero ~/.joerc con las opciones predefinidas del editor, entre ellas, el contenido de la ventana de ayuda y la disposición del teclado (combinaciones de teclas). Podemos utilizar como muestra el archivo /usr/lib/joe/joerc.

4.2. Editor emacs

El editor *emacs* es parecido a *joe*, pero mucho más completo y complejo. Estudiaremos únicamente su funcionamiento como editor básico de textos, pero en realidad su utilidad es mucho más amplia. La instalación completa puede ocupar mucho espacio en disco, por lo que suelen existir opciones especiales en la instalación de Linux para incluir o no los componentes de *emacs*.

Al ejecutar el editor, admite como parámetro el nombre del fichero que queremos editar.

SINTAXIS

```
emacs [fichero]
```

Algunas de las combinaciones de teclas que utiliza *emacs* están formadas por dos pulsaciones consecutivas. Por ejemplo, con *^HH* (pulsamos la tecla *Ctrl* y la tecla *H*, y luego la tecla *H*) *emacs* nos muestra una pantalla de ayuda, de la que podemos salir con *^XL*. Otras combinaciones se hacen en una sola pulsación. Por ejemplo, con *^A* (pulsamos la tecla *Ctrl* y la tecla *A*) saltamos al principio de una línea. También hay combinaciones que utilizan dos o más veces la tecla *Ctrl*. Finalmente, algunas combinaciones comienzan con la tecla *Escape* seguida de una pulsación. Por ejemplo, *Escape F* (pulsamos la tecla *Escape* y luego la tecla *F*)

Combinación	**Categoría**	**Significado**
`^X^F`	Archivos	Carga un fichero en otra ventana.
`^X^R`	Archivos	Carga un fichero, eliminando el contenido actual.
`^X^S`	Archivos	Guarda el fichero actual.
`^X^W`	Archivos	Guarda el fichero actual con otro nombre.
`^XN`	Archivos	Guarda el fichero actual con otro nombre.
`^Espacio`	Bloques	Marca el principio de un bloque[12].
`^W`	Bloques	Elimina el bloque marcado.
`^X^I`	Bloques	Inserta un archivo en la posición del cursor.
`^Y`	Bloques	Pega el último bloque cortado (borrado con algún comando) o copiado al portapapeles.
`Escape w`	Bloques	Copia el bloque marcado al portapapeles.
`^R`	Búsqueda	Busca una cadena hacia atrás.
`^S`	Búsqueda	Busca una cadena.
`^XR`	Búsqueda	Repite la búsqueda hacia atrás.
`^XS`	Búsqueda	Repite la búsqueda.
`Escape ^R`	Búsqueda	Busca y reemplaza de forma interactiva.
`Escape r`	Búsqueda	Busca y reemplaza (sin preguntar).
`^D`	Edición	Borra un carácter a la derecha.
`^K`	Edición	Borra hasta el final de la línea.
`^T`	Edición	Intercambia el carácter actual con el que le sigue.
`^X^T`	Edición	Intercambia la línea actual con la que le sigue.
`Escape c`	Edición	Pone en mayúsculas los caracteres necesarios.
`Escape d`	Edición	Borra una palabra a la derecha.
`Escape l`	Edición	Pone una palabra en minúsculas (todos los caracteres).
`Escape t`	Edición	Intercambia la palabra sobre la que está el cursor con la que le sigue.
`Escape u`	Edición	Pone una palabra en mayúsculas (todos los caracteres).
`^A`	Movimiento	Salta al principio de una línea.
`^B`	Movimiento	Mueve el cursor a la izquierda (←).
`^E`	Movimiento	Salta al final de una línea.
`^F`	Movimiento	Mueve el cursor a la derecha (→).
`^N`	Movimiento	Mueve el cursor abajo (↓).
`^P`	Movimiento	Mueve el cursor arriba (↑).

[12] Trabajar con bloques, marcamos el principio del bloque con `^Espacio`, a continuación movemos el cursor para marcar el final, y por último efectuamos alguna operación con el bloque, por ejemplo, con `^W` borramos el bloque.

Combinación	Categoría	Significado
^V	Movimiento	Avance de página.
^X]	Movimiento	Avance de página.
Escape :	Movimiento	Inicio del documento (primera línea).
Escape ;	Movimiento	Fin del documento (última línea).
Escape a	Movimiento	Salta a una línea.
Escape b	Movimiento	Mueve el cursor una palabra a la izquierda.
Escape f	Movimiento	Mueve el cursor una palabra a la derecha.
Escape v	Movimiento	Retroceso de página.
^X^C	Varios	Termina *emacs*.
^X^U	Varios	Deshace el último cambio.
^XB	Varios	Cambia a otra ventana (otro archivo en edición)
^XX	Varios	Cambia a la siguiente otra ventana (otro archivo en edición)
Escape ^N	Varios	Cambia el nombre de una ventana.
Escape z	Varios	Guarda todos los ficheros y termina.

Tabla 53. Teclas y comandos del editor emacs

En los comandos de búsqueda, mientras se teclea la palabra buscada, *emacs* realiza la búsqueda (esto se llama *búsqueda incremental*). La búsqueda con reemplazamiento interactiva nos permite decidir qué hacer en cada aparición del patrón buscado. Entonces podemos responder con las opciones de la Tabla 54.

Opción	Acción
^G	Cancelar.
?	Obtener una lista de opciones.
.	Sustituir y terminar, devolviendo el cursor a la posición inicial.
,	Sustituir el resto del documento (pasa a modo no interactivo).
y	Sustituir y buscar la siguiente aparición.
Espacio	Sustituir y buscar la siguiente aparición (igual que y).
n	No sustituir y buscar la siguiente aparición.

Tabla 54. Opciones de búsqueda y sustitución interactiva

Se puede repetir varias veces de forma automática un comando pulsando *Escape* y un número, y a continuación ejecutando el comando.

Podemos crear un archivo llamado .emacs en nuestro directorio *home* para definir las opciones del editor.

emacs puede funcionar con un entorno de menús. Para ello es necesario trabajar bajo un entorno gráfico. Por esta razón, no podemos utilizar esta facilidad a través de una conexión de tipo *telnet*. En cambio, podemos emplear sistemas de conexión mediante gráficos como *eXcursion*, en lugar de telnet o cualquier tipo de conexión basada en texto.

4.3. Editor vi

vi es el editor de pantalla más extendido en UNIX. Es bastante difícil de manejar, pero conviene conocerlo puesto que es el que nos podemos encontrar en cualquier máquina UNIX. En los sistemas Linux, *vi* conserva el funcionamiento original, pero se le han añadido nuevas combinaciones de teclas para hacerlo más intuitivo, al menos, a los usuarios de MS-DOS/Windows. Nosotros estudiaremos el funcionamiento original. El editor se invoca pasando como parámetro, opcionalmente, el nombre del fichero que queremos editar.

SINTAXIS

```
vi [fichero]
```

vi funciona con 3 modos: modo *shell*, modo *comando* y modo *inserción*. Podemos pasar de un modo a otro en cualquier momento pulsando las combinaciones de teclas o comandos adecuados. La Figura 1 ilustra en forma de autómata estos tres modos.

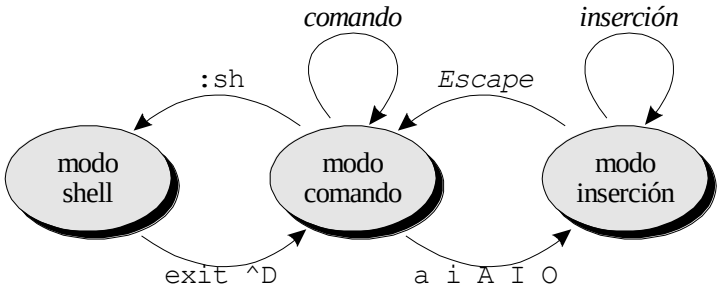

Figura 1. Modos de funcionamiento de vi

- El modo *shell* permite escribir y ejecutar comandos del shell. Una vez dentro de *vi*, podemos hacer una salida momentánea al shell, ejecutar algún comando, y regresar de nuevo a *vi* con `exit`.
- El modo *comando* nos permite trabajar con el texto: mover el cursor de edición, insertar líneas en blanco, borrar líneas, copiar líneas, borrar palabras, deshacer/rehacer, buscar textos, guardar, salir, etc. Los comandos se invocan mediante la pulsación de alguna tecla (distinguiendo entre mayúsculas y minúsculas) o bien mediante una palabra clave precedida por algún símbolo, por ejemplo, dos puntos (`:`).
- El modo *inserción* sirve para añadir texto al documento. Pulsando *Escape*, volvemos al modo de comandos.

La Tabla 55 contiene los principales comandos que nos permiten utilizar el editor *vi*.

Combinación	Categoría	Significado
`:w`	Archivos	Guarda el fichero en edición.
`:w` *`fichero`*	Archivos	Guarda el fichero en edición con el nombre *`fichero`*.
`^G`	Archivos	Muestra información sobre el fichero que estamos editando.
`##``Y`	Bloques	Copia al portapapeles la línea actual y las *##-1* siguientes (para duplicarla con `P` o `p`).
`:r` *`fichero`*	Bloques	Inserta en la posición del cursor el contenido del fichero *`fichero`*.
`p`	Bloques	Inserta el último bloque borrado (`dd`) o copiado al portapapeles (`Y`) en la siguiente línea.
`P`	Bloques	Inserta el último bloque borrado (`dd`) o copiado al portapapeles (`Y`) en la línea actual.
`##``G`	Búsqueda	Salta a la línea número *##*, siendo *##* cualquier número.

Combinación	Categoría	Significado
/*patrón*/	Búsqueda	Salta a la próxima ocurrencia del *patrón*.
:##	Búsqueda	Salta a la línea número ##.
?*patrón*?	Búsqueda	Salta a la anterior ocurrencia del *patrón*.
n	Búsqueda	Repite la última búsqueda.
N	Búsqueda	Repite la última búsqueda, pero en dirección inversa.
~	Edición	Cambia el carácter sobre el cursor de mayúsculas a minúsculas, o viceversa.
a	Edición	Pasa al modo de inserción, situando el cursor una posición a la derecha (*Escape* para terminar).
A	Edición	Pasa al modo de inserción, situando el cursor al final de la línea (*Escape* para terminar).
C	Edición	Elimina el resto de la línea a la derecha y pasa al modo de inserción (*Escape* para terminar).
cw	Edición	Elimina el resto de la palabra a la derecha y pasa al modo de inserción (*Escape* para terminar).
D	Edición	Borra hasta el final de la línea.
dd	Edición	Borra la línea actual (la pasa al portapapeles).
dw	Edición	Borra a la derecha del cursor hasta el final de palabra.
i	Edición	Pasa al modo de inserción, dejando el cursor en la posición actual (*Escape* para terminar).
I	Edición	Pasa al modo de inserción, situando el cursor al principio de la línea (*Escape* para terminar).
J	Edición	Une la línea actual con la siguiente.
o	Edición	Pasa al modo de inserción, situando el cursor en la línea siguiente (*Escape* para terminar).
rx	Edición	Reemplaza el carácter sobre el cursor por el carácter *x*.
x	Edición	Borra un carácter a la derecha (el situado sobre el cursor).
X	Edición	Borra un carácter a la izquierda (el situado antes del cursor).
:w	Ficheros	Guarda el fichero en edición.
-	Movimiento	Salta a la primera posición de la línea anterior.
$	Movimiento	Salta al final de la línea actual.
^B	Movimiento	Retroceso de página.
^D	Movimiento	Avance de media página.
^E	Movimiento	Avance de una línea (↓)
^F	Movimiento	Avance de página.
^U	Movimiento	Retroceso de media página.
^Y	Movimiento	Retroceso de una línea (↑)
+	Movimiento	Salta a la primera posición de la línea siguiente.
0 (cero)	Movimiento	Mueve el cursor hasta el principio de la línea actual.
b	Movimiento	Retrocede el cursor a la palabra anterior.
B	Movimiento	Retrocede el cursor a la palabra anterior, contando los signos de puntuación como parte de la palabra.
Delete	Movimiento	Retroceso de un carácter (←)
e	Movimiento	Mueve el cursor hasta el final de la palabra actual.
E	Movimiento	Mueve el cursor hasta el final de la palabra, contando los signos de puntuación como parte de la palabra.
Enter	Movimiento	Salta a la primera posición de la línea siguiente.
Espacio	Movimiento	Avance de un carácter (→)
H	Movimiento	Saltar al inicio del documento.

Combinación	Categoría	Significado
L	Movimiento	Saltar al final del documento.
M	Movimiento	Situar el cursor en el centro de la pantalla.
w	Movimiento	Avanza el cursor a la palabra siguiente.
W	Movimiento	Avanza el cursor a la palabra siguiente, contando los signos de puntuación como parte de la palabra.
z	Movimiento	Centrar la pantalla de manera que la posición actual del cursor quede en la parte superior de la pantalla.
z.	Movimiento	Centrar la pantalla en la posición actual del cursor.
:!*comando*	Shell	Ejecuta el comando *comando* del shell.
:sh	Shell	Pasa a modo shell para que podamos ejecutar comandos (exit para regresar a *vi*).
:q	Varios	Abandona *vi*.
:q!	Varios	Abandona *vi* sin guardar los cambios.
:set	Varios	Muestra el estado de las opciones principales.
:set all	Varios	Muestra el estado de todas las opciones.
:wq	Varios	Guarda el fichero en edición y abandona *vi*.
:x	Varios	Guarda el fichero en edición y abandona *vi*.
u	Varios	Deshace los últimos cambios.
U	Varios	Rehace los últimos cambios deshechos.

Tabla 55. Teclas y comandos del editor vi

Todas las operaciones de *vi* pueden ser antepuestas por un número, lo que hace que la operación se repita ese número de veces. Por ejemplo, 3dd borra 3 líneas de una vez, 23P pega el último texto borrado 23 veces, etc.

Los comandos de búsqueda utilizan el punto (.) como comodín para representar cualquier carácter. Para buscar un punto debe anteponerse el carácter "\". Por ejemplo, para buscar la cadena "etc." debemos escribir el siguiente comando: "/etc\.".

vi tiene varias opciones de configuración, que pueden ser consultadas con el comando :set. Este comando también permite establecer las opciones. Por ejemplo, para activar la opción de auto-indentación podemos poner ":set ai" o ":set autoindent", y para desactivarla, ":set noai" o ":set noautoindent". Otra opción importante que podemos activar es "nu" (o "number"), que muestra los números de línea a la izquierda de la pantalla. En el mismo comando set se pueden establecer varias opciones.

Las opciones también pueden establecerse en un archivo de configuración, en el directorio *home* de cada usuario. Este fichero debe llamarse ".exrc" y debe contener una secuencia de ordenes para vi. Por ejemplo:

.exrc

```
set autoindent autowrite report=2 showmatch
```

4.4. Entornos wpe y xwpe

wpe (*window programming environment*) es un entorno de programación (no sólo un editor) semejante a los IDE de Borland (Borland Pascal, C, etc.). El aspecto (ver Figura 2) es similar y tiene aproximadamente las mismas funciones incorporadas:

- **Facilidades de edición**: manejo de bloques, portapapeles, etc.

- **Editor orientado a sintaxis**: utilización de distintos colores para identificar las secciones de los programas.
- **Invocación al compilador**: sin necesidad de teclear desde línea de comando la llamada al compilador y sus parámetros.
- **Depurador integrado**: con las opciones habituales de traza: ejecución paso a paso, inspección de objetos (variables), puntos de ruptura, etc.

Por ahora sólo nos interesa su funcionamiento como editor. En general podemos utilizar las mismas teclas que en *joe* y que en los editores de Borland para MS-DOS, además de los menús.

Existe una versión para entornos gráficos (X-Window) llamada *xwpe*, idéntica a *wpe*, pero más vistosa. La Figura 2 muestra el aspecto de *xwpe*.

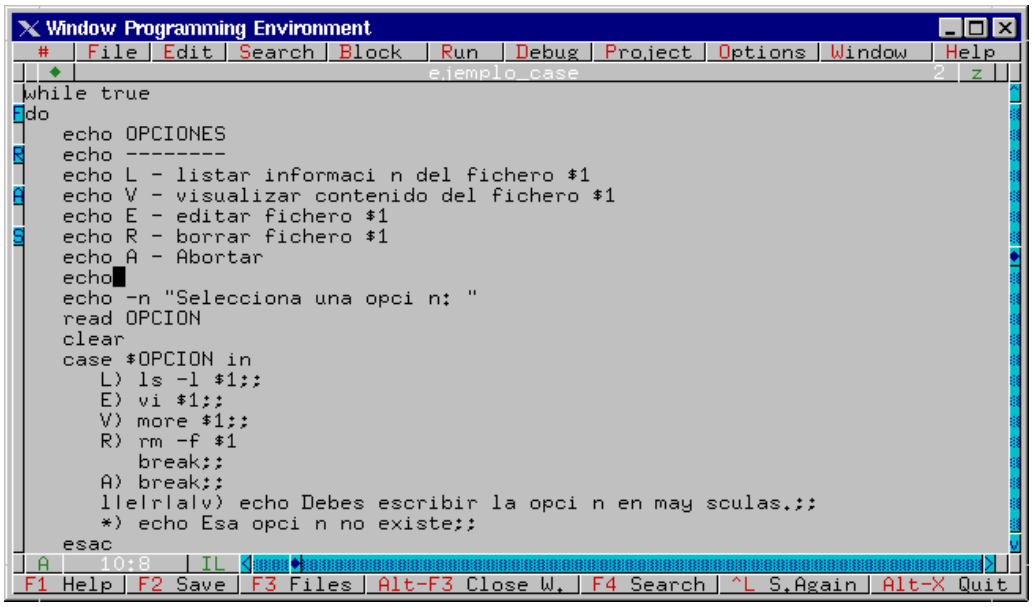

Figura 2. Aspecto del editor xwpe

xwpe necesita trabajar bajo entorno gráfico, por lo que no podemos utilizarlo a través de una conexión de tipo *telnet*. Necesitamos algún sistema de conexión mediante gráficos como *eXcursion*, o, por supuesto, trabajar en la consola principal bajo *X-window*.

5. Scripts en bash

Los shell de UNIX, además de permitir el uso de comandos, permiten crear programas formados a base de comandos, llamados de muchas formas: *shell-scripts*, *scripts*, *guiones*, etc. Se parecen a los ficheros de proceso por lotes (*batch*) de MS-DOS (ficheros **.bat*), pero son mucho más potentes.

Además de los comandos, en los *scripts* podemos utilizar muchas de las estructuras típicas de la programación tradicional: estructuras iterativas, estructuras de control, etc.

Los *scripts* se comportan como auténticos programas (comandos), y admiten parámetros para obtener resultados más completos. Algunos de estos *scripts* tienen utilidades específicas.

5.1. Creación de scripts

Un *script* es, esencialmente, un archivo de texto que contiene en cada línea un comando, una línea en blanco, o un comentario. Los comentarios comienzan por el carácter `#`, y se pueden incluir en cualquier parte de una línea de un script. Para poder ejecutar un *script* es necesario que establezcamos previamente el permiso de ejecución del fichero correspondiente.

IMPORTANTE:

El shell puede determinar si un fichero es o no de texto examinando la primera línea del fichero; si ésta contiene únicamente caracteres de texto, considerará que todo el fichero es de texto y tratará de ejecutarlo como un script. Sin embargo, si aparece algún carácter que no sea puramente de texto, intentará ejecutarlo como un programa compilado (binario). Esto nos puede ocurrir si en la primera línea de un script escribimos caracteres *extraños*, como vocales acentuadas, eñes, símbolos especiales, etc,

El siguiente ejemplo es un script que podemos utilizar para conocer el estado del sistema: nos muestra la fecha, un calendario y el listado de usuarios conectados. Guardaremos el fichero con el nombre `ejemplo_basico`:

ejemplo_basico

```
date     # MUESTRA LA FECHA
cal      # MUESTRA UN CALENDARIO DE ESTE MES
who      # MUESTRA LA LISTA DE USUARIOS CONECTADOS
```

El resultado de su ejecución es el siguiente:

```
polifemo:~$ ./ejemplo_basico
Tue Oct 26 10:05:15 CEST 1999
    October 1999
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
hans     ttyrc    Oct 26 09:52 (192.168.1.14)
javier   ttyp4    Oct 26 09:49 (192.168.1.29)
amg      ttyr9    Oct 26 09:51 (192.168.1.19)
jupamodo ttyrd    Oct 26 10:00 (192.168.1.11)
```

Habitualmente en los scripts se utilizan variables para hacerlos más flexibles. Además del uso de las variables que ya conocemos, tenemos nuevas posibilidades al utilizar scripts:

- Lectura del valor de una variable desde teclado.
- Utilización de parámetros.
- Variables especiales.

Para leer un valor por teclado utilizamos el comando **read**, con la siguiente sintaxis:

SINTAXIS

```
read variable
```

`variable` es el nombre de una variable (sin $). Por ejemplo:

ejemplo_read

```
echo Escriba algo:
read ALGO
echo Ha escrito lo siguiente: \"$ALGO\"
```

```
polifemo:~$ ./ejemplo_read
Escriba algo:
En un lugar de la mancha...
Ha escrito lo siguiente: "En un lugar de la mancha..."
```

La utilización de parámetros la estudiaremos en el punto *5.2 Parámetros* (página 93). Por otro lado, las siguientes variables están presentes cuando utilizamos scripts:

- **$!**: contiene el valor del PID del último proceso ejecutado en segundo plano. Por ejemplo:

ejemplo_pid

```
sleep 3 &     # Ejecuta algo en segundo plano
echo El proceso anterior tiene PID $!
ps            # Muestra los PID de todos los procesos, para poder comparar
```

```
polifemo:~$ ./ejemplo_pid
El proceso anterior tiene PID 7515
  PID TTY STAT  TIME COMMAND
 6650  r8 S    0:00 -bash
 7514  r8 S    0:00 -bash
 7515  r8 S    0:00 sleep 3
 7516  r8 R    0:00 ps
```

- **$?**: contiene el valor de salida del último comando ejecutado. Cuando un programa termina, termina con un estado, que es un número. Si el programa termina de forma normal, este valor será cero. Y si termina con algún tipo de error, el valor devuelto será distinto de cero. Muchos comandos utilizan este mecanismo. La única forma de conocer el valor de retorno de un comando es mediante el uso de esta variable. Ejemplo:

ejemplo_test

```
test 0 -eq 1   # Debe terminar con error
echo $?
```

```
test 0 -eq 0   # Debe terminar correctamente
echo $?
```

```
polifemo:~$ ./ejemplo_test
1
0
```

En realidad estas variables también se pueden utilizar desde la línea de comandos (sin script), cuando escribimos varios comandos en la misma orden.

Podemos hacer que nuestros scripts terminen con un estado de normalidad o de error. El comando `exit` (que hasta ahora nos ha servido para salir del sistema) permite terminar prematuramente un script, con un estado determinado. La sintaxis es:

SINTAXIS

```
exit [estado]
```

estado es un número que indica el estado de finalización, que debe ser 0 para representar un estado correcto y un valor distinto de cero para un estado de error (normalmente cada comando utiliza el número de estado erróneo para indicar un tipo concreto de error). Si no se indica *estado*, se presupone 0. El siguiente ejemplo termina con error de tipo 17:

ejemplo_exit

```
echo Hola
exit 17
echo Esto nunca se escribe!!!
```

Si lo ejecutamos y a continuación analizamos el código de error, observamos que la parte final del programa no se ejecuta, y que terminó con estado 17:

```
polifemo:~$ ./ejemplo_exit ; echo $?
Hola
17
```

5.2. Parámetros

Los parámetros pasados como argumentos a un script pueden ser consultados como si fueran variables con los nombres de la Tabla 56.

Variable	Significado
`$1`	Primer parámetro pasado.
`$2`	Segundo parámetro pasado.
`$3`	Tercer parámetro pasado.
`...`	
`${10}`	Décimo parámetro pasado.
`${11}`	Undécimo parámetro pasado.
`...`	
`$*`	Todos los parámetros pasados (separados por el primer carácter de la variable `IFS`).
`$0`	Nombre del archivo ejecutado (ruta completa).
`$#`	Número de parámetros pasados.

Tabla 56. Variables de acceso a los parámetros

Si consultamos el valor de un parámetro no pasado (por ejemplo, `$4` cuando sólo hay un parámetro), el valor devuelto será una cadena vacía.

Para utilizar los parámetros con dos o más cifras numéricas (es decir, `$10`, `$11`, `$12`, etc.) tenemos que encerrar entre llaves el número (es decir, `${10}`, `${11}`, `${12}`, etc.), ya que de lo contrario se confundirán con alguno de los 10 primeros parámetros seguidos de algún número (es decir, lo equivalente a `${1}0`, `${1}1`, `${1}2`, etc.)

El comando **`shift`** nos permite desechar el parámetro primero ($1) y desplazar una posición el resto de parámetros, de manera que $2 pasa a ocupar la posición de $1, $3 la de $2, y así sucesivamente. Esto nos permite utilizar los parámetros con las estructuras de control iterativas.

Veamos un ejemplo de uso de los parámetros y el comando `shift`. Sea el siguiente script, que llamaremos `ejemplo_par`:

ejemplo_par

```
echo Hay $# argumentos, que son: $*
echo Argumentos 1, 2 y 3:
echo $1 $2 $3
shift
echo Argumentos 2, 3 y 4:
echo $1 $2 $3
```

La siguiente salida muestra un ejemplo de su ejecución:

```
polifemo:~$ ./ejemplo_par A B C D E
Hay 5 argumentos, que son: A B C D E
Argumentos 1, 2 y 3:
A B C
Argumentos 2, 3 y 4:
B C D
```

5.3. Sentencias de control

Dentro de un script podemos utilizar algunas de las estructuras habituales de los lenguajes de programación imperativos: estructuras iterativas (bucles *for*, *while*, etc.) y estructuras del control (alternativa, alternativa múltiple, etc.).

5.3.1. Estructura iterativa for

Permite ejecutar una secuencia de comandos varias veces, y en cada una de ellas asigna un valor a una variable, tomado de un conjunto de posibles valores.

SINTAXIS

```
for variable in conjunto
do
   comandos
done
```

variable es el nombre de una variable (sin $). *conjunto* es un conjunto de valores que se irán asignando a la variable en cada iteración. Podemos utilizar para definir este conjunto máscaras de nombres de ficheros (p.e. `*.txt`), conjuntos de valores (p.e. `$*`), o una mezcla de ambas cosas (p.e. `*.txt $1 fichero1 palabra1 /tmp/*.c`). Finalmente *comandos* es la secuencia de líneas con comandos que se ejecutarán en cada iteración. Un ejemplo: mostrar sucesivamente el contenido los archivos pasados como parámetro, haciendo una pausa entre fichero y fichero:

ejemplo_for

```
for I in $*
do
   clear
   echo Contenido del fichero $I
   echo ---------------------------------------------
   cat $I
   sleep 1
done
```

Entre los comandos encerrados por el bucle (entre `do` y `done`) se puede incluir alguna de las siguientes sentencias:

- **`break`**: permite romper el bucle, de manera que la ejecución continúa por la instrucción inmediatamente posterior al `done` más interno (igual que el *break* de C).
- **`continue`**: permite iniciar la siguiente iteración de forma inmediata (sin necesidad de llegar al `done`), de forma que el próximo comando que se ejecuta es la asignación de un nuevo valor a la variable de control (igual que el *continue* de C).

5.3.2. Estructura iterativa while

Permite repetir la ejecución de una secuencia de comandos mientras se dé una condición.

SINTAXIS

```
while condición
do
   comandos
done
```

condición es un comando que decide si se continúa iterando. Si el comando termina con éxito, se continúa iterando; por el contrario, si termina con error, termina el bucle. *comandos* es la secuencia de líneas con comandos que se ejecutarán en cada iteración.

Veamos un ejemplo: escribir los números del 1 al 30:

ejemplo_while

```
NUM=1
while [ $NUM -le 30 ]
do
   echo $NUM
   NUM=`expr $NUM + 1`
done
```

Existen dos comandos muy útiles para ser utilizados en este tipo de bucles. Se trata de los comandos **true** y **false**:

- `true` es un comando que no hace nada, pero que termina con un estado de éxito (0).
- `false` tampoco hace nada, pero termina con un estado de error (distinto de 0).

Por lo tanto, podemos utilizar estos comandos como expresión que siempre se evalúa a verdadero (`true`) o a falso (`false`). Esto permite, entre otras muchas posibilidades, crear bucles infinitos.

En la `wihle` estructura también podemos utilizar las sentencias `break` y `continue`:

- **break**: permite romper el bucle, de manera que la ejecución continúa por la instrucción inmediatamente posterior al `done` más interno (igual que el *break* de C).
- **continue**: permite iniciar la siguiente iteración de forma inmediata (sin necesidad de llegar al `done`), de forma que el próximo comando que se ejecuta es la evaluación de la condición de continuación (igual que el *continue* de C).

5.3.3. Estructura iterativa until

Permite repetir la ejecución de una secuencia de comandos hasta que se dé una condición.

SINTAXIS

```
until condición
do
   comandos
done
```

El funcionamiento es idéntico al de la estructura *while*, excepto por que la condición de salida del bucle es contraria a la de éste. Podemos verlo en el ejemplo de uso de `if` (a continuación).

5.3.4. Estructura alternativa simple (if)

Permite bifurcar la ejecución de dos ramas alternativas, según el valor de una condición.

SINTAXIS

```
if condición
then
   comandos
[else
```

```
   comandos_alternativos]
fi
```

condición es un comando que decide qué rama se ejecuta, la *then*, o la *else*. Si el comando termina con éxito, se ejecutan los comandos de la rama *then*; por el contrario, si termina con error, se ejecutan los comandos de la rama *else*.

comandos es la secuencia de líneas con comandos que se ejecutarán en la rama *then*. *comandos_alternativos* es la secuencia de la rama *else*. La parte *else* es opcional.

Veamos un ejemplo: el juego de adivinar el número de palabras de un fichero pasado por parámetro:

ejemplo_if

```
TAMANO=`wc -w <$1`   # Numero de palabras
NUMERO=-1            # Iniciamos de manera que entre en el bucle

until [ $TAMANO -eq $NUMERO ]
do
   echo Adivina el numero de palabras del fichero $1
   read NUMERO
   if [ $NUMERO -gt $TAMANO ]
   then
      echo El fichero $1 tiene menos palabras.
   else
      if [ $NUMERO -lt $TAMANO ]
      then
         echo El fichero $1 tiene mas palabras.
      fi
   fi
done
echo HAS ACERTADO!!!!
```

5.3.5. Estructura alternativa múltiple (case)

Permite bifurcar la ejecución entre varias ramas alternativas, según el valor de una expresión.

SINTAXIS

```
case expresion in
   valor1) comandos1
           comandos1;;
[  valor2) comandos2
           comandos2;;]
   ...
[  *)      otros_comandos
           otros_comandos;;]
esac
```

expresión es un comando que decide qué rama se ejecuta. Este comando debe escribir por la salida estándar un valor que será comparado con cada unas de las opciones *valor1*, *valor2*, etc. En el caso de que coincida con alguno de estas opciones, ejecutará los comandos situados a la derecha de "`)`" y en las líneas sucesivas hasta encontrar "`;;`". Si no coincide con ninguno de los valores propuestos, y existe una opción `*)`, entonces ejecuta los comandos asociados a dicha opción.

Cada valor propuesto puede ser un único valor o un conjunto de valores, separados por "|", y entonces se entrará en la opción cuando la expresión coincida con cualquiera de los valores.

El siguiente ejemplo muestra un menú de opciones para manejar un fichero pasado como parámetro. Se puede mostrar información, su contenido, borrarlo, etc.

ejemplo_case

```
while true
do
   echo OPCIONES
   echo --------
   echo L - listar información del fichero $1
   echo V - visualizar contenido del fichero $1
   echo E - editar fichero $1
   echo R - borrar fichero $1
   echo A - Abortar
   echo
   echo -n "Selecciona una opción: "
   read OPCION
   clear
   case $OPCION in
      L) ls -l $1;;
      E) vi $1;;
      V) more $1;;
      R) rm -f $1
         break;;
      A) break;;
      l|e|r|a|v) echo Debes escribir la opción en mayúsculas.;;
      *) echo Esa opción no existe;;
   esac
done
```

5.4. Depuración de scripts

Cuando se está ejecutando un script, no podemos ver la secuencia de comandos que se ejecuta, a menos que activemos la opción –v del bash, utilizando para ello el comando `set` (ver apartado 2.1.3 *Configuración de opciones de* bash en la página 22). Esto nos puede resultar muy útil para saber qué es lo que realmente está ocurriendo en el script.

Veamos por ejemplo la ejecución del siguiente script utilizando la opción `-v`:

script cuenta

```
I=$1
while [ $I -gt 0 ]
do
        echo $I
        I=`expr $I - 1`
done
```

```
polifemo:~$ set -v
polifemo:~$ cuenta 5
cuenta 5
I=$1
while [ $I -gt 0 ]
do
        echo $I
        I=`expr $I - 1`
done
5
```

```
expr $I - 1
4
expr $I - 1
3
expr $I - 1
2
expr $I - 1
1
expr $I - 1
```

En lugar de –v, podemos utilizar –x, que nos muestra lo que realmente se ejecuta en cada instrucción:

```
polifemo:~$ set +v
set +v
polifemo:~$ set -x
polifemo:~$ cuenta 5
+ cuenta 5
++ I=5
++ [ 5 -gt 0 ]
++ echo 5
5
+++ expr 5 - 1
++ I=4
++ [ 4 -gt 0 ]
++ echo 4
4
+++ expr 4 - 1
++ I=3
++ [ 3 -gt 0 ]
++ echo 3
3
+++ expr 3 - 1
++ I=2
++ [ 2 -gt 0 ]
++ echo 2
2
+++ expr 2 - 1
++ I=1
++ [ 1 -gt 0 ]
++ echo 1
1
+++ expr 1 - 1
++ I=0
++ [ 0 -gt 0 ]
```

5.5. Scripts de iniciación

Existen algunos scripts que se ejecutan de forma automática cada vez que entramos en el sistema. Se suelen utilizar para definir algunas variables importantes (como el *path*) y ejecutar comandos de iniciación. La Tabla 57 recoge algunos de ellos. Los que están situados en los directorios *home* de los usuarios son personalizables por cada usuario; sin embargo, lo que tiene rutas absolutas son compartidos por todos los usuarios.

Fichero	Utilidad
`~/.bash_profile`	Se ejecuta cada vez que entra en el sistema, teniendo `bash` como shell predefinido.
`~/.bash_login`	Se ejecuta cada vez que entra en el sistema, teniendo `bash` como shell predefinido, siempre y cuando no exista `~/.bash_profile`.
`~/.profile`	Se ejecuta cada vez que entra en el sistema, con cualquier shell predefinido (si el shell es `bash`, sólo se ejecuta `.profile` si no existen `~/.bash_profile` ni `~/.bash_login`).
`/etc/profile`	Se ejecuta cada vez que entra en el sistema, teniendo `bash` como shell predefinido.
`~/.bashrc`	Se ejecuta cada vez que se ejecuta `bash` como un comando (sin parámetros) o cuando se entra como otro usuario con `su`.

Tabla 57. Scripts de iniciación

6. Bibliografía complementaria

6.1. Libros de Sistemas Operativos y UNIX

[Dei93] Deitel, H.M., "***Sistemas operativos. 2ª edición.***", Addison-Wesley, 1993.

[Mil88] Milenkovic, M., "***Sistemas Operativos. Conceptos y Diseño***", McGraw-Hill, 1988.

[Mil88] Milenkovic, M., "***Sistemas operativos***", McGrawHill, 1988.

[Mor89] Morgan, R., McGilton, H., "***Introducción al UNIX sistema V***", McGraw-Hill, 1989.

[Sil94] Silberschatz, A., Peterson, J., Galvin, P., "***Sistemas operativos. Conceptos fundamentales. 3ª edición***", Addison-Wesley, 1994.

[Sta97] Stallings, W., "***Sistemas operativos. 2ª edición***", Prentice Hall, 1997.

[Tan98] Tanenbaum, A.S., "***Sistemas Operativos. Diseño e Implementación***", Perentice-Hall, 1998.

6.2. Libros de Linux en la Biblioteca

[Bec97] Beck, M., "***Linux Kernel Internals***", Addison-Wesley, 1997 (681.3.066 Linux).

[Bla96] Blanco, V.J., "***Linux: Instalación, Administración y Uso del Sistema***", RA-MA, 1996 (681.3.066 Linux).

[Car97] Card, R. Dumas, E., Mvel, F., "***Programación Linux 2.0: API de Sistema y Funcionamiento de Núcleo***", Gestión 2000, 1997 (681.3.066 Linux).

[Mar98] Martín Pérez, C., Pérez Crespo, I., "***Linux***", Anaya Multimedia, 1998 (681.3.066 Linux).

[Pet98] Petersen, R., "***Linux: the Complete Reference***", Osborne, 1998 (681.3.066 Linux).

[Tac96] Tackett, J., Gunter, D., "***Utilizando Linux***", Prentice Hall, 1996 (681.3.066 Linux).

[Tac98] Tackett, J., Gunter, D., "***Linux: Edición Especial***", Prentice Hall, 1998 (681.3.066 Linux).

[Wel96] Welsh, M., Kaufma, L., "***Running Linux***", O'Reilly, 1996 (681.3.066 Linux).

6.3. Libros de UNIX en la Biblioteca

[Art94] Arthur, L.J., Burns, T., "***UNIX Shell Programming***", John Wiley & Sons, 1994 (681.3.066 UNIX)

[Chr90] Christian, K., "***Diccionario de C y UNIX***", Anaya Multimedia, 1990 (681.3.066 C, 681.3.066 UNIX)

[Chr92] Christian, K., "***UNIX***", Paraninfo, 1992 (681.3.066 UNIX)

[Cof90] Coffin, S., "***UNIX: Manual de Referencia: incluye Sistema V versión 3***", Osborne/McGraw-Hill, 1990 (681.3.066 UNIX)

[Hav94] Haviland, K. Salama, B., "***UNIX^{tm} System Programming***", Addison-Wesley, 1994 (681.3.066 UNIX)

[Ker94] Kernighan, B.W., Pike, R., "***El Entorno de Programación UNIX***", Prentice-Hall Hispanoamericana, 1994 (681.3.066 UNIX)

[LeB90] LeBlond, G.T., Blust, S.R., Modes, W., "***Using UNIX System V: release 3***", Osborne/MacGraw-Hill, 1990 (681.3.066 UNIX)

[Nem95] Nemeth, E., "***UNIX System Administration Handbook***", Prentice Hall, 1995 (681.3.066 UNIX)

[Nov97] Novo, A., "***El Entorno UNIX***", Abeto, 1997 (681.3.066 UNIX)

[Pra89] Prata, S., Martin, D., "***UNIX Sistema V: Manual de Referencia de todos los Comandos y Utilidades***", Anaya Multimedia, 1989 (681.3.066 UNIX)

[Ros97] Rosen, K.H., "***UNIX: Sistema V, versión 4***", Osborne/McGraw-Hill, 1997 (681.3.066 UNIX)

[Sán96] Sánchez, S., "***UNIX: Guía del Usuario***", RA-MA, 1996 (681.3.066 UNIX)

[Tho85] Thomas, R., Yates, J., "***Sistema Operativo UNIX: Guía del Usuario***", McGraw-Hill, 1985 (681.3.066 UNIX)

[Wai86] Waite, M., Martin, D., Prate, S., "***Introducción al UNIX sistema V***", Anaya Multimedia, 1986 (681.3.066 UNIX)